Desarrollo de un Sistema de Información Web para la Gestión de Inventarios, Ingresos y Egresos en el Establecimiento Comercial Minorista Britny Max del Municipio de Rionegro Santander

Britny Mayerith Romero Sepulveda

Nasly Brendy Kenaira Rojas Garcia

Trabajo de grado para optar al título de Ingeniería de Sistemas

Modalidad: Proyecto de Investigación Aplicado

Director /ra/ res
Nombres y Apellidos completos de quién dirige el trabajo
Doctor / Magíster / Especialista en

UNICIENCIA

Corporación Universitaria de Ciencia y Desarrollo

Facultad Ingeniería

Programa académico Ingeniería de Sistemas

Bucaramanga

2026

Carta de Cesión de Derechos de Autor

Ciudad, fecha

Señores
Corporación Universitaria de Ciencia y Desarrollo – Uniciencia
Ciudad

Estimados señores:

Yo (nosotros), _____, identificado(s) con C. C. No. _____, autor(es) del trabajo de grado titulado_____ presentado en el año _____ como requisito para optar al título de _____; manifiesto (amos) en este documento la voluntad de ceder a la Corporación Universitaria de Ciencia y Desarrollo – UNICIENCIA los derechos patrimoniales consagrados; así mismo, autorizo lo siguiente, siempre y cuando sea con fines académicos:

- Ponga el contenido de este trabajo a disposición de los usuarios en EL REPOSITORIO INSTITUCIONAL. SI: ____ NO: ____

- Permita la consulta a los usuarios interesados en el contenido de este trabajo para todos los usos que tengan finalidad académica, ya sea formato impreso, CD-ROM o digital desde la Internet. SI: ____ NO: ____
- Todos los usos que tengan finalidad académica; de manera especial la divulgación a través de redes de información académica. SI: ____ NO: ____

De conformidad con lo establecido en el artículo 30 de la Ley 23 de 1982 y sobre el artículo 11 de la Decisión Andina 351 de 1993, "Los derechos morales sobre el trabajo son propiedad de los autores", los cuales son irrenunciables, imprescriptibles, inembargables e inalienables. Atendiendo lo anterior, siempre que se consulte la obra, mediante cita bibliográfica se debe dar crédito al trabajo y a su(s) autor(es).

Firma
_____
_
Nombres y apellidos
Número de documento de identidad

_____
_
Nombres y apellidos
Número de documento de identidad
Firma

Dedicatoria

Dedico este logro a nuestro señor, por darme el privilegió de tener la oportunidad de poder estudiar, por darme las fuerzas, la paciencia para poder avanzar a este punto. A mi tía Miriam Sepulveda por brindarme el apoyo económico para estudiar, a Leidy Yohana y José Bautista por ayudar a que se hiciera realidad. A mi abuela María Suarez quien fue y es mi fuerza para todo el camino y aunque no esté presente, su memoria siempre estará presente. A mi madre Ruth Sepulveda, a mis compañeros de clases que hicieron que este proceso fuera más tranquilo, Finalmente a mí misma por ser fuerte y no rendirme a pesar de las adversidades.

Britny Mayerith Romero Sepulveda

Dedico la culminación de este proceso, primero, a Dios, por permitirnos disfrutar de vida, salud y de las oportunidades junto a los desafíos de aprendizaje necesarios para llevarlo a cabo; a mi madre, Johanna García Páez, y a mi padre, Edward Rojas Montaguth, por enseñarme y educarme con amor, día a día, por cuidar mis pasos y por brindarme los recursos y las bases requeridas para prepararme y crecer personal, profesional y laboralmente. Desde siempre ha sido un privilegio para mí que sean mis padres, y deseo, día a día, lograr todo aquello que los haga sentir orgullosos de mí.

Nasly Brendy Kenaira Rojas Garcia

Agradecimientos

Agradezco a la Corporación Universitaria de Ciencia y Desarrollo- Uniciencia, por haber sido importante en mi proceso de formación, por permitirme adquirir experiencias y descubrir capacidades que antes no sabía que tenía. Agradezco a cada una de las personas que hicieron parte de este camino que me dejaron una enseñanza que llevare en mi vida Profesional. Agradezco a Kenaira Rojas Garcia mi compañera de proyecto de grado con la que hemos estado desde el primer semestre. Gracias por compartir momentos difíciles, experiencias, peleas, felicidades que nos hacen más que compañeras. Este logro representa varios años de esfuerzo que hoy me permite mirar atrás con Orgullo y Gratitud.

Britny Mayerith Romero Sepulveda

Agradezco a Dios, a mi padre, Edward Rojas Montaguth, y a mi madre, Johanna García Páez, por permitirme llevar a cabo mi proceso de estudio, por suplir cada necesidad y por acompañar cada momento de mi proceso desde el primer día; a la Corporación Universitaria de Ciencia y Desarrollo – UNICIENCIA; a los docentes que deciden ejercer con vocación su labor y que brindan enseñanzas y señalan caminos en pro del futuro de sus estudiantes; y a mi compañera de proyecto de grado, Britny Romero Sepúlveda, por su apoyo y complicidad, por su paciencia en cada situación presentada y por su amistad, cariño y compañía, no solo en este proceso final, sino en todo el camino recorrido juntas.

Nasly Brendy Kenaira Rojas García

Resumen

El presente trabajo describe el desarrollo de un sistema de información web para la gestión de inventarios, ingresos y egresos de la tienda Britny Max, ubicada en Rionegro, Santander, que llevaba su control en un cuaderno físico. El objetivo fue optimizar los procesos administrativos y operativos del establecimiento mediante una herramienta accesible desde el navegador. El proyecto siguió un enfoque cuantitativo de alcance descriptivo y el modelo en cascada: se levantaron los requerimientos con la propietaria, se diseñaron los casos de uso y el modelo de datos, y se construyó la aplicación con Vue y Tailwind CSS sobre Supabase y PostgreSQL. El sistema registra productos, lee códigos de barras, controla ventas, entradas de mercancía, conteos físicos, ingresos y egresos, genera reportes y protege la información con cuatro niveles de acceso cuyos permisos asigna la administradora, políticas de seguridad en la base de datos y cierre de sesión tras cinco minutos de inactividad, y quedó publicado en la web mediante Vercel. Se aplicó un plan de pruebas con 61 pruebas automatizadas, todas aprobadas, recorridos funcionales en Chrome y Edge, verificaciones de seguridad sobre la base de datos en la nube y mediciones de rendimiento, en las que las operaciones evaluadas respondieron en menos de 0,3 segundos, frente al límite de 2 segundos definido. Se concluye que el sistema cumple los requerimientos especificados y que la prueba de aceptación en la tienda, con la propietaria, terminó con resultados conformes.

Palabras clave: sistema de información web, gestión de inventarios, tiendas de barrio, control de acceso, pruebas de software.

Abstract

This work describes the development of a web-based information system for managing inventory, income and expenses at Britny Max, a neighborhood store in Rionegro, Santander, Colombia, which kept its records in a paper notebook. The objective was to optimize the store's administrative and operational processes through a tool accessible from a web browser. The project followed a quantitative, descriptive approach and the waterfall model: requirements were gathered with the owner, use cases and the data model were designed, and the application was built with Vue and Tailwind CSS on top of Supabase and PostgreSQL. The system registers products, reads barcodes, controls sales, stock receipts, physical counts, income and expenses, generates reports and protects information with four access levels whose permissions are assigned by the administrator, database security policies and automatic sign-out after five minutes of inactivity, and it was published on the web through Vercel. A test plan was applied, including 61 automated tests, all of which passed, functional walkthroughs in Chrome and Edge, security checks on the cloud database and performance measurements, in which the evaluated operations responded in less than 0.3 seconds against the 2-second limit defined. It is concluded that the system meets the specified requirements and that the acceptance test at the store, carried out with the owner, ended with satisfactory results.

Keywords: web information system, inventory management, neighborhood stores, access control, software testing.

Tabla de Contenido

Introducción 18

Planteamiento del Problema 20

Pregunta de Investigación 22

Objetivos 23

Objetivo General 23

Objetivos Específicos 23

Justificación 24

Marco Referencial 26

Estado del Arte 26

Marco Teórico Conceptual 28

Marco Legal 30

Metodología 31

Análisis y Diseño del Sistema 33

Descripción del Proceso Actual 33

Especificación de Requerimientos 34

Requerimientos Funcionales 35

Requerimientos No Funcionales 40

Diseño del Sistema 45

Diagramas de Casos de Uso 46

Diagrama de Clases 55

Desarrollo del Sistema 57

Arquitectura del Sistema 57

Tecnologías Utilizadas 58
Modelo de Datos 59
Seguridad y Niveles de Acceso 61
Despliegue y Puesta en Producción 67
Módulos Implementados 69
Acceso al Sistema 69
Panel de Inicio 70
Gestión de Productos 71
Lectura de Código de Barras 73
Registro de Ventas 74
Ingresos y Egresos 76
Entradas de Mercancía y Conteo Físico 78
Reportes 80
Respaldo de Información 81
Diseño Adaptable 82
Plan de Pruebas y Resultados 84
Objetivo y Alcance 84
Niveles y Tipos de Prueba 84
Entorno y Datos de Prueba 86
Criterios de Aceptación 86
Casos de Prueba y Resultados 87
Resultados de las Pruebas Automatizadas 89
Resultados de Rendimiento 91

Resultados de Seguridad y Compatibilidad 92

Pruebas de Aceptación del Usuario 93

Prueba de Aceptación en la Tienda 94

Análisis de Resultados 95

Conclusiones 97

Recomendaciones 99

Referencias 100

Anexo A. Listado de Pruebas Automatizadas 105

Anexo B. Instalación y Puesta en Marcha 108

Anexo C. Script de la Base de Datos 109

Anexo D. Código Fuente y Trazabilidad 127

Lista de Tablas

Tabla 1 *Requerimiento Funcional RF01: Autenticación de Usuario* 36

Tabla 2 *Requerimiento Funcional RF02: Registro de Productos* 36

Tabla 3 *Requerimiento Funcional RF03: Consulta y Búsqueda de Productos* 37

Tabla 4 *Requerimiento Funcional RF04: Edición y Eliminación de Productos* 37

Tabla 5 *Requerimiento Funcional RF05: Lectura de Código de Barras* 38

Tabla 6 *Requerimiento Funcional RF07: Registro de Salidas por Venta* 38

Tabla 7 *Requerimiento Funcional RF11: Registro de Ingresos y Egresos* 39

Tabla 8 *Requerimiento Funcional RF15: Generación de Reportes* 39

Tabla 9 *Requerimiento No Funcional RNF01: Tiempo de Respuesta* 40

Tabla 10 *Requerimiento No Funcional RNF02: Seguridad de la Información* 41

Tabla 11 *Requerimiento No Funcional RNF03: Facilidad de Uso* 42

Tabla 12 *Requerimiento No Funcional RNF04: Disponibilidad del Sistema* 42

Tabla 13 *Requerimiento No Funcional RNF05: Compatibilidad* 43

Tabla 14 *Requerimiento No Funcional RNF07: Integridad de los Datos* 43

Tabla 15 *Requerimiento No Funcional RNF08: Escalabilidad* 44

Tabla 16 *Requerimiento No Funcional RNF09: Respaldo de Información* 45

Tabla 17 *Tecnologías Utilizadas en el Desarrollo* 59

Tabla 18 *Tablas de la Base de Datos* 60

Tabla 19 *Matriz Inicial de Permisos por Nivel de Acceso* 62

Tabla 20 *Entornos de Funcionamiento del Sistema* 68

Tabla 21 *Cuentas de Usuario Creadas en Producción* 69

Tabla 22 *Niveles y Tipos de Prueba del Plan* 85

Tabla 23 *Entorno de Ejecución de las Pruebas* 86

Tabla 24 *Casos de Prueba y Resultados Obtenidos* 87

Tabla 25 *Resultados de las Pruebas Automatizadas* 89

Tabla 26 *Tiempos de Respuesta Medidos* 92

Tabla 27 *Escenarios de la Prueba de Aceptación del Usuario* 93

Tabla 28 *Resultados de la Prueba de Aceptación en la Tienda* 95

Tabla 29 *Pruebas Automatizadas del Sistema* 105

Tabla 30 *Matriz de Trazabilidad de Requerimientos* 128

Lista de Figuras

Figura 1 *Diagrama de Casos de Uso del RF01: Autenticación de Usuario* 47

Figura 2 *Diagrama de Casos de Uso del RF02: Registro de Productos* 48

Figura 3 *Diagrama de Casos de Uso del RF03: Consulta y Búsqueda de Productos* 49

Figura 4 *Diagrama de Casos de Uso del RF04: Edición y Eliminación de Productos* 50

Figura 5 *Diagrama de Casos de Uso del RF05: Lectura de Código de Barras* 51

Figura 6 *Diagrama de Casos de Uso del RF07: Registro de Salidas por Venta* 52

Figura 7 *Diagrama de Casos de Uso del RF11: Registro de Egresos* 53

Figura 8 *Diagrama de Casos de Uso del RF11: Registro de Ingresos* 53

Figura 9 *Diagrama de Casos de Uso del RF15: Generación de Reportes* 54

Figura 10 *Diagrama de Clases del Sistema* 56

Figura 11 *Arquitectura del Sistema Britny Max* 58

Figura 12 *Modelo Entidad-Relación de la Base de Datos* 61

Figura 13 *Módulo de Usuarios y Niveles de Acceso* 63

Figura 14 *Permisos por Nivel Editables por la Administradora* 64

Figura 15 *Acceso Restringido para el Nivel Cajero* 65

Figura 16 *Aviso de Cierre de Sesión por Inactividad* 67

Figura 17 *Pantalla de Inicio de Sesión* 70

Figura 18 *Panel de Inicio del Nivel Administrador* 71

Figura 19 *Módulo de Productos* 72

Figura 20 *Registro de Producto con Mensajes de Validación* 73

Figura 21 *Lectura de Código de Barras* 74

Figura 22 *Registro de Salida por Venta* 75

Figura 23 *Historial de Ventas* 76

Figura 24 *Módulo de Ingresos* 77

Figura 25 *Módulo de Egresos* 78

Figura 26 *Registro de Entrada de Mercancía* 79

Figura 27 *Conteo Físico con Detección de Diferencias* 80

Figura 28 *Módulo de Reportes* 81

Figura 29 *Módulo de Respaldo y Cuenta* 82

Figura 30 *Vista del Sistema en un Celular* 83

Figura 31 *Ejecución de las Pruebas Unitarias* 90

Figura 32 *Ejecución de las Pruebas de Base de Datos* 91

Figura 33 *Cambio de Permisos Pendiente de Guardar* 94

Figura 34 *Estructura de Carpetas del Proyecto* 128

Figura 35 *Punto de Entrada de la Aplicación (src/main.js)* 130

Figura 36 *Protección de las Pantallas (src/router/index.js)* 131

Figura 37 *Carga de Permisos en la Sesión (src/stores/auth.js)* 132

Figura 38 *Reglas de Permisos (src/lib/permisos.js)* 133

Figura 39 *Cierre de Sesión por Inactividad (src/composables/useInactividad.js)* 134

Figura 40 *Llamadas a las Operaciones de la Base de Datos (src/services/supabaseBackend.js)* 135

Figura 41 *Registro Transaccional de una Venta (supabase/schema.sql)* 136

Figura 42 *Verificación de Permisos en la Base de Datos (supabase/schema.sql)* 137

Figura 43 *Validación de Productos (src/lib/validaciones.js)* 138

Lista de Anexos

Anexo A. Listado de Pruebas Automatizadas 105

Anexo B. Instalación y Puesta en Marcha 108

Anexo C. Script de la Base de Datos 109

Anexo D. Código Fuente y Trazabilidad 127

Introducción

El presente trabajo aborda el desarrollo de un sistema de información web para la gestión de inventarios, ingresos y egresos en la tienda Britny Max, ubicada en el municipio de Rionegro, Santander. Las tiendas de barrio constituyen un eslabón esencial de la distribución minorista en Colombia; sin embargo, su operación se caracteriza por una amplia informalidad administrativa, con un control de existencias que continúa realizándose de forma manual mediante cuadernos, libretas o registros escritos. En el caso específico de la tienda Britny Max, la propietaria no dispone de una herramienta que le permita conocer con exactitud las unidades disponibles de cada producto, identificar los artículos de mayor rotación ni determinar el momento oportuno de reabastecimiento, lo que impide detectar pérdidas por vencimiento, hurto, errores de cobro o manejo inadecuado de la mercancía.

Frente a esta problemática, se planteó como pregunta de investigación: ¿de qué manera el desarrollo de un sistema de información web para la gestión de inventarios, ingresos y egresos optimiza los procesos administrativos y operativos de la tienda Britny Max, en el municipio de Rionegro, Santander? Para responderla, establecimos como objetivo general desarrollar un sistema de información web para la gestión de inventarios, ingresos y egresos en la tienda Britny Max, con el fin de optimizar los procesos administrativos y operativos del establecimiento, a través de tres objetivos específicos orientados al análisis de requerimientos, la implementación de los módulos del sistema y la evaluación de su funcionamiento en el entorno operativo real.

El proyecto se desarrolló bajo un enfoque cuantitativo de alcance descriptivo, siguiendo el modelo en cascada de la ingeniería de software. La interfaz se construyó con Vue y Tailwind CSS, la autenticación y la persistencia de datos se apoyaron en Supabase sobre PostgreSQL, y se integró la lectura de códigos de barras mediante lector o cámara para agilizar el registro de las transacciones. El documento se organiza en el planteamiento del problema, los objetivos y la justificación; el marco referencial; la metodología; el análisis y diseño del sistema; el desarrollo, con capturas de las pantallas implementadas; el plan de pruebas y sus resultados; y, finalmente, las conclusiones y recomendaciones. El sistema está publicado en https://britny-max.vercel.app y su código fuente se incluye en el Anexo D.

Planteamiento del Problema

Las tiendas de barrio constituyen un eslabón esencial de la distribución minorista en Colombia y una de las formas más extendidas de la economía popular. De acuerdo con la Encuesta de Micronegocios del Departamento Administrativo Nacional de Estadística [DANE] (2025), en 2024 el país registró 5.297.252 micronegocios, de los cuales el 23,9 % corresponde a la actividad de comercio y reparación de vehículos automotores y motocicletas, la rama económica con mayor número de unidades. Estos micronegocios ocuparon a 6.879.489 personas, lo que evidencia su peso en el tejido productivo del país. Dentro de este grupo, el DANE (2026) estimó la existencia de 540.233 micronegocios de panaderías y tiendas de barrio en 2024, el 89,5 % de ellos dedicados al comercio al por menor.

Pese a su relevancia, su operación se caracteriza por una marcada informalidad administrativa. Según el DANE (2025), el 77,6 % de los micronegocios no contaba con Registro Único Tributario en 2024 y el 89,5 % no estaba inscrito en cámara de comercio. Esta informalidad se extiende a la gestión interna del negocio: el 89,1 % de los micronegocios no usó dispositivos electrónicos para el desarrollo de sus actividades durante 2024. En el caso particular de las panaderías y tiendas de barrio, el 43,4 % no contaba con ningún registro para llevar sus cuentas y el 46,1 % las llevaba de manera informal en un cuaderno, una hoja de Excel o una máquina registradora (DANE, 2026), de modo que el control de existencias continúa realizándose, en su mayoría, de forma manual.

Ese modelo presenta limitaciones críticas a medida que aumentan el volumen de productos y la frecuencia de transacciones: es imposible conocer en tiempo real el estado del inventario, se dificulta la detección oportuna de mermas, no existen registros confiables para las decisiones de compra y aumenta la exposición a errores humanos en el conteo de mercancías. La situación se agrava fuera de los centros urbanos, donde la brecha digital es más amplia: en 2024 el 83 % de la población urbana del mundo usaba internet, frente al 48 % de la población rural (International Telecommunication Union [ITU], 2024). En la misma línea, Sarmiento Suárez et al. (2024) evidencian un bajo nivel de madurez digital en las mipymes colombianas, que adoptan principalmente tecnologías básicas como las redes sociales, la banca digital y las páginas web.

En el caso específico de la tienda Britny Max, ubicada en el municipio de Rionegro, Santander, la propietaria no dispone de una herramienta que le permita conocer con exactitud las unidades disponibles de cada producto, identificar los artículos de mayor rotación ni determinar el momento oportuno de reabastecimiento. Al cierre de cada mes resulta inviable comparar el inventario físico con las ventas registradas, lo que impide detectar pérdidas por vencimiento, hurto, errores de cobro o manejo inadecuado de la mercancía.

Frente a esta problemática, un sistema de información web ofrece una solución accesible: automatiza el registro de existencias, genera reportes en tiempo real y alerta sobre discrepancias entre el inventario físico y el registrado, e incorpora la lectura de códigos de barras para agilizar la operación. No obstante, su adaptación en municipios pequeños enfrenta barreras como el desconocimiento tecnológico de los propietarios, la percepción de costo o complejidad y la escasez

de herramientas diseñadas para el contexto del pequeño negocio. Por ello se hace necesario implementar una solución ajustada a las capacidades operativas de este tipo de establecimientos, que resuelva el control de existencias de manera sencilla, funcional y sostenible.

Pregunta de Investigación

¿De qué manera el desarrollo de un sistema de información web para la gestión de inventarios, ingresos y egresos optimiza los procesos administrativos y operativos de la tienda Britny Max, en el municipio de Rionegro, Santander?

Objetivos

Objetivo General

Desarrollar un sistema de información web para la gestión de inventarios, ingresos y egresos en la tienda Britny Max, del municipio de Rionegro, Santander, con el fin de optimizar los procesos administrativos y operativos del establecimiento.

Objetivos Específicos

Analizar los procesos operativos de gestión de inventario, ingresos y egresos de la tienda Britny Max para establecer los requerimientos funcionales y no funcionales del sistema de información web.

Implementar los módulos de registro de productos, control de entradas y salidas de mercancía, gestión de ingresos y egresos, e integración de la lectura de código de barras que conforman el sistema de información web.

Evaluar el funcionamiento del sistema de información web en el entorno operativo real de la tienda Britny Max mediante pruebas funcionales que validen el control de existencias y la detección de discrepancias de inventario.

Justificación

El comercio minorista de barrio representa una de las actividades económicas más comunes e importantes en las comunidades locales colombianas. A pesar de su relevancia social y económica, muchos de estos negocios se administran de manera informal, lo que puede limitar su crecimiento y desarrollo. La tienda Britny Max no es ajena a esta realidad, ya que su propietaria controla las existencias de forma manual, sin una herramienta que le permita conocer con precisión el estado del inventario ni detectar pérdidas de manera oportuna.

La propuesta se justifica por varias razones. En primer lugar, desde el punto de vista práctico, responde a una necesidad cotidiana de la tienda, ya que permitirá a la propietaria consultar en cualquier momento el estado de las existencias, anticipar la reposición de productos y verificar al final de cada mes la correspondencia entre el inventario físico y los registros del sistema. Esto facilitará la toma de decisiones relacionadas con las compras y el abastecimiento.

Desde el aspecto económico, un control adecuado del inventario contribuye directamente a mejorar la rentabilidad del negocio, ya que permite identificar los productos de mayor rotación, reducir pérdidas por desperdicio o vencimiento y reconocer los periodos de mayor demanda.

En el aspecto tecnológico, el proyecto demuestra que las herramientas informáticas también pueden implementarse en pequeños negocios. El desarrollo de una aplicación web adaptada a las necesidades de una tienda de barrio puede servir como referencia para otros establecimientos similares que deseen mejorar sus procesos mediante el uso de la tecnología.

Finalmente, desde el aspecto académico, el proyecto integra conocimientos de ingeniería de sistemas, bases de datos y sistemas de información en el desarrollo de un sistema funcional que resuelve una problemática real. Además, contribuye al estudio de la transformación digital en pequeños negocios de municipios pequeños de Colombia, mostrando cómo la tecnología puede convertirse en una herramienta de apoyo para mejorar la gestión empresarial.

Marco Referencial

Estado del Arte

La gestión de inventarios en pequeños negocios ha sido objeto de creciente estudio durante los últimos años. A escala internacional, Nirmala et al. (2022) aplicaron los análisis ABC y VED para clasificar los artículos según su valor y su criticidad y, con ello, orientar el control del inventario; Maheshwari et al. (2025) mostraron, a partir de dos casos de estudio en micro, pequeñas y medianas empresas del sector de alimentos, que la adopción del internet de las cosas se relaciona con menores costos de inventario, aunque estas empresas se resisten a adoptarlo por la percepción de altos costos de implementación; y Sbai et al. (2022) propusieron un enfoque de decisión multicriterio para seleccionar sistemas de inventario en redes de distribución, aplicado a la cadena de suministro farmacéutica del sector público de Marruecos.

En el contexto latinoamericano, Corella Parra y Olea Miranda (2023) desarrollaron un sistema de control de inventario para una comercializadora de sistemas de riego en Sonora, México, que combinó herramientas físicas y digitales para mejorar el flujo de materiales y la exactitud de los registros; Galaviz-Magallanes et al. (2024) implementaron un método de control de inventario en una microempresa familiar de Chihuahua, México, a partir de una base de datos de ventas y del método de máximos y mínimos, y encontraron excesos de inventario en tres de los cinco productos analizados; Flores Tapia y Flores Cevallos (2023) compararon los sistemas de revisión continua (Q) y periódica (P) y demostraron que la elección del sistema modifica la cantidad económica de pedido, el inventario de seguridad y los costos totales; y Ccallo Rodríguez

et al. (2024), mediante una revisión sistemática, analizaron la efectividad del Lean Warehousing para optimizar la gestión de inventarios en pequeñas y medianas empresas. Estos trabajos coinciden en que la sistematización del control de existencias incide directamente en la eficiencia operativa y la rentabilidad de los pequeños negocios.

En el contexto colombiano, Rueda Vera et al. (2022) analizaron la incidencia de los sistemas de información en el control de inventarios de 300 micro, pequeñas y medianas empresas comerciales de Cúcuta y concluyeron que dicha relación es positiva, al identificar doce correlaciones entre ambas variables. Palma Cardoso et al. (2023) estudiaron microempresas del sector farmacéutico del Tolima, encontraron un manejo de inventarios que lleva a decisiones alejadas de las necesidades reales de los negocios y propusieron un modelo de control basado en las filosofías Just in Time y 5S. Guerra-Naranjo (2024), en una investigación bibliográfica publicada en la revista *Reflexiones Contables*, examinó la influencia de la gestión de inventarios en el rendimiento empresarial de organizaciones colombianas y concluyó que la gerencia debe implementar una gestión estratégica de los inventarios para tomar decisiones acertadas. De manera más cercana al presente proyecto, Pardo Morales et al. (2026) diseñaron, junto con tenderos del sector Balmoral de Fusagasugá, un prototipo web para la gestión de inventarios en tiendas de barrio; el proceso participativo evidenció el interés de los comerciantes por las herramientas digitales y permitió definir requerimientos ajustados a sus condiciones tecnológicas y operativas, lo que refuerza la necesidad de desarrollar herramientas sencillas, asequibles y acompañadas de procesos de apropiación.

La relevancia de estas soluciones se sustenta en evidencia oficial. La Encuesta de Micronegocios (DANE, 2025) muestra que el comercio concentra el mayor número de micronegocios del país (23,9 %) y que el 89,1 % de estas unidades no utiliza dispositivos electrónicos en su operación, mientras que el boletín dedicado a las panaderías y tiendas de barrio (DANE, 2026) reporta que, entre las que no llevan contabilidad formal, el 14,2 % no sabe cómo llevar registros. Sarmiento Suárez et al. (2024) reportan un bajo nivel de madurez digital en las mipymes colombianas, con predominio de tecnologías básicas, y la ITU (2024) advierte que la brecha de uso de internet entre zonas urbanas y rurales no muestra avances, salvo en el grupo de países de menores ingresos. Este panorama justifica el desarrollo de soluciones tecnológicas asequibles y adaptadas al contexto del pequeño comercio, como la propuesta en el presente proyecto.

Marco Teórico Conceptual

Un sistema de información se entiende como un conjunto organizado de personas, datos, procesos y recursos que captura, procesa y distribuye información para apoyar las actividades de una organización y mejorar la toma de decisiones. La gestión de inventarios comprende los procesos orientados a controlar las existencias de un negocio para disponer de los productos necesarios en el momento oportuno, en las cantidades adecuadas y al menor costo posible (Flores Tapia y Flores Cevallos, 2023; Palma Cardoso et al., 2023). El código de barras es una representación gráfica de datos legible mediante dispositivos ópticos, que permite automatizar el registro de transacciones y actualizar el inventario en tiempo real. Los ingresos corresponden a las

entradas de dinero por ventas y los egresos a las salidas asociadas a compras y gastos operativos; su registro sistemático permite conocer la situación financiera del negocio.

El sistema se desarrolló como una aplicación web de una sola página. La interfaz de usuario se construyó con Vue, framework progresivo de JavaScript para crear interfaces basadas en componentes reactivos (Vue.js, 2026), y su apariencia se resolvió con Tailwind CSS, marco de estilos basado en clases utilitarias que facilita el diseño adaptable a computadores y celulares (Tailwind Labs, 2026). Vite se utilizó como herramienta de desarrollo y construcción (Vite, 2026) y Pinia como almacén del estado compartido de la aplicación (Pinia, 2025). La autenticación, la API y la persistencia de los datos se soportaron en Supabase, plataforma de backend como servicio cuya biblioteca cliente permite consumir sus servicios desde el navegador (Supabase, 2026) y que utiliza PostgreSQL, sistema de gestión de bases de datos relacional de código abierto que incorpora políticas de seguridad a nivel de fila para restringir qué registros puede consultar o modificar cada usuario (PostgreSQL Global Development Group, 2025). La lectura de códigos de barras se resolvió de dos maneras: con lectores USB, que funcionan como un teclado, y con la cámara del dispositivo mediante la biblioteca ZXing (ZXing-js, 2024).

En materia de seguridad, el control de acceso basado en niveles o roles asigna a cada usuario un conjunto de permisos de acuerdo con sus funciones, de modo que solo pueda ejecutar las operaciones que le corresponden. El estándar de verificación de seguridad de aplicaciones de OWASP recomienda, además, aplicar las reglas de autorización en una capa de servicio confiable y no solo en la interfaz, y definir un tiempo de inactividad tras el cual la sesión exija una nueva

autenticación (OWASP Foundation, 2025). La calidad del producto se valoró a partir de las características de adecuación funcional, eficiencia de desempeño, compatibilidad, capacidad de interacción, fiabilidad y seguridad descritas en la norma ISO/IEC 25010 (International Organization for Standardization [ISO], 2023), y las pruebas se organizaron con los conceptos de proceso, niveles y tipos de prueba de la norma ISO/IEC/IEEE 29119-1 (ISO, 2022). El proceso de construcción se orientó por el modelo en cascada de la ingeniería de software, apropiado para proyectos con requisitos definidos desde el inicio, que organiza el trabajo en fases secuenciales con entregables verificables.

Marco Legal

El desarrollo se enmarca en la Ley 1581 de 2012 y su Decreto Reglamentario 1377 de 2013, que establecen los principios para el tratamiento de datos personales en Colombia, y en la Ley 527 de 1999, que reconoce la validez jurídica de los mensajes de datos y respalda el uso del sistema como soporte de las transacciones del negocio.

En coherencia con estas normas, el sistema solo almacena los datos personales necesarios de sus usuarios (nombre, correo de acceso y nivel), restringe su consulta al nivel administrador y no expone identificadores ni credenciales en las direcciones web. Cada venta, entrada, ajuste, ingreso y egreso conserva la fecha y el usuario que lo registró, de modo que los mensajes de datos generados por el sistema sirvan como soporte de las operaciones del negocio.

Metodología

El proyecto se desarrolló bajo un enfoque cuantitativo de alcance descriptivo, orientado al análisis de requerimientos, el diseño, el desarrollo y la evaluación de un sistema de información web para la gestión de inventarios de la tienda Britny Max. El enfoque cuantitativo se justifica porque los resultados son medibles: el número de productos registrados, la precisión del inventario, las discrepancias detectadas y las pérdidas identificadas constituyen variables cuantificables antes y después de la implementación. El proceso siguió el modelo en cascada, que organiza el trabajo en fases secuenciales con entregables definidos:

Fase 1. Análisis de requerimientos (julio, semanas 1-4). Se levantaron los requerimientos funcionales y no funcionales mediante observación directa, entrevistas semiestructuradas con la propietaria y revisión de los registros manuales existentes. Entregable: documento de especificación de requerimientos.

Fase 2. Diseño (agosto, semanas 1-3). Se modeló la base de datos en PostgreSQL y se diseñaron las interfaces de usuario y los flujos de información. Entregable: documento de diseño con los diagramas de casos de uso, el diagrama de clases y los prototipos de interfaz elaborados en Figma.

Fase 3. Desarrollo (agosto semana 3 a septiembre semana 4). Se implementaron los módulos de autenticación con cuatro niveles de acceso, registro y consulta de productos, lectura de códigos de barras, ventas, entradas de mercancía, conteo físico, ingresos y egresos, reportes y respaldo, con la interfaz en Vue y Tailwind CSS, la persistencia en Supabase sobre PostgreSQL y

la integración del lector de códigos de barras. Entregable: software funcional publicado en Vercel e instalado como servicio en el computador de la tienda.

Fase 4. Pruebas y evaluación (octubre). Se ejecutó el plan de pruebas descrito en el capítulo correspondiente: pruebas automatizadas unitarias y de base de datos, recorridos funcionales por nivel de usuario, pruebas de seguridad sobre la base de datos en la nube, mediciones de tiempo de respuesta y pruebas de compatibilidad. También se realizó la prueba de aceptación en el entorno real de la tienda con la propietaria. Entregable: informe de pruebas y sistema listo para su operación.

Las unidades de análisis son los productos comercializados y los registros de entrada y salida generados durante la implementación. Los instrumentos incluyen fichas de observación, guías de entrevista y formatos de registro de inventario físico.

Análisis y Diseño del Sistema

Descripción del Proceso Actual

La tienda Britny Max, ubicada en el municipio de Rionegro, Santander, lleva actualmente el control de su operación de manera completamente manual. El registro de los productos, las entradas y salidas de mercancía, y los movimientos económicos del negocio se realizan en un cuaderno físico, sin ningún apoyo tecnológico que garantice la organización, confiabilidad o permanencia de la información.

El proceso de registro de productos se hace de forma básica, anotando el nombre, el código, la categoría y el precio de compra de cada artículo. La actualización de las cantidades disponibles no se realiza de manera inmediata, sino aproximadamente una vez por semana, lo que genera un desfase entre el inventario real y el registrado. Esta situación dificulta saber con exactitud cuántas unidades de cada producto hay disponibles en un momento dado y cuándo es necesario realizar un nuevo pedido.

En cuanto al control de los movimientos económicos, la propietaria registra manualmente los ingresos por ventas, los egresos por compras de mercancía, los pagos de servicios públicos y otros gastos generales del negocio. Sin embargo, al no contar con un sistema organizado, resulta difícil consultar esta información de manera ágil o hacer un seguimiento claro del comportamiento financiero del negocio durante un período determinado.

Otro aspecto crítico del proceso actual es la vulnerabilidad de la información. Al estar registrada únicamente en un cuaderno físico, existe el riesgo permanente de pérdida por deterioro,

extravío o cualquier eventualidad que afecte el soporte físico. La propietaria ha manifestado que esta situación le genera preocupación, especialmente porque no existe ningún mecanismo que permita recuperar la información en caso de que se pierda.

Finalmente, la consulta del estado del negocio también presenta limitaciones. Para conocer qué productos están disponibles o agotados, revisar las entradas y salidas de mercancía, o evaluar cómo estuvo funcionando el negocio durante un período específico, la propietaria debe revisar manualmente las anotaciones del cuaderno, un proceso que consume tiempo y que puede generar errores de interpretación. Esta dificultad para acceder a información organizada y confiable es uno de los principales factores que motivaron la necesidad de desarrollar un sistema de información adaptado a las condiciones reales de la tienda.

Especificación de Requerimientos

La definición de los requerimientos del sistema constituye una etapa fundamental en el proceso de desarrollo de software, ya que permite establecer con claridad qué debe hacer el sistema y bajo qué condiciones debe funcionar. En el caso de la tienda Britny Max, esta especificación parte directamente de las dificultades identificadas en el proceso actual: el registro manual de productos, la pérdida de información, la dificultad para consultar el estado del inventario y la ausencia de un control organizado de los movimientos económicos. Definir los requerimientos con precisión garantiza que el sistema a desarrollar responda a las necesidades reales del negocio y no a suposiciones.

Los requerimientos se clasifican en dos categorías. Los requerimientos funcionales describen las funciones específicas que el sistema debe ser capaz de realizar. Los requerimientos no funcionales establecen las condiciones de calidad bajo las cuales el sistema debe operar, como su velocidad, seguridad, facilidad de uso y disponibilidad. Ambos tipos de requerimientos fueron identificados a partir de la entrevista realizada con la propietaria de la tienda y de la observación del proceso actual.

Requerimientos Funcionales

A continuación se presentan los requerimientos funcionales del sistema, los cuales describen las funciones que este debe ejecutar para responder a las necesidades operativas de la tienda Britny Max. Cada requerimiento se deriva de una necesidad concreta identificada durante el proceso de diagnóstico.

Tabla 1

Requerimiento Funcional RF01: Autenticación de Usuario

Identificación del requerimiento:	RF01
Nombre del Requerimiento:	Autenticación de usuario
Características:	El sistema permitirá el acceso mediante usuario y contraseña, validando las credenciales del usuario autorizado.
Descripción del requerimiento:	El sistema permitirá iniciar sesión para acceder a los módulos de gestión de Britny Max.
Requerimiento NO funcional:	RNF01, RNF02
Prioridad del requerimiento:	Alta

Nota. Elaboración propia.

Tabla 2

Requerimiento Funcional RF02: Registro de Productos

Identificación del requerimiento:	RF02
Nombre del Requerimiento:	Registro de productos
Características:	El sistema permitirá registrar los productos comercializados en Britny Max.
Descripción del requerimiento:	El sistema permitirá registrar nombre, código de barras, categoría, precio de compra, precio de venta y cantidad inicial del producto.
Requerimiento NO funcional:	RNF01, RNF03, RNF07
Prioridad del requerimiento:	Alta

Nota. Elaboración propia.

Tabla 3

Requerimiento Funcional RF03: Consulta y Búsqueda de Productos

Identificación del requerimiento:	RF03
Nombre del Requerimiento:	Consulta y búsqueda de productos
Características:	El sistema permitirá consultar y localizar los productos registrados.
Descripción del requerimiento:	El usuario podrá buscar productos mediante nombre, categoría o código de barras y consultar su información y cantidad disponible.
Requerimiento NO funcional:	RNF01, RNF03, RNF05
Prioridad del requerimiento:	Alta

Nota. Elaboración propia.

Tabla 4

Requerimiento Funcional RF04: Edición y Eliminación de Productos

Identificación del requerimiento:	RF04
Nombre del Requerimiento:	Edición y eliminación de productos
Características:	El sistema permitirá modificar o retirar productos registrados.
Descripción del requerimiento:	El usuario podrá modificar los datos de un producto existente o eliminarlo del catálogo cuando sea necesario.
Requerimiento NO funcional:	RNF02, RNF03, RNF07
Prioridad del requerimiento:	Media

Nota. Elaboración propia.

Tabla 5

Requerimiento Funcional RF05: Lectura de Código de Barras

Identificación del requerimiento:	RF05
Nombre del Requerimiento:	Lectura de código de barras
Características:	El sistema permitirá identificar productos mediante su código de barras.
Descripción del requerimiento:	El sistema permitirá utilizar un lector de código de barras para identificar productos y facilitar las operaciones de consulta y registro.
Requerimiento NO funcional:	RNF01, RNF03, RNF05
Prioridad del requerimiento:	Alta

Nota. Elaboración propia.

Tabla 6

Requerimiento Funcional RF07: Registro de Salidas por Venta

Identificación del requerimiento:	RF07
Nombre del Requerimiento:	Registro de salidas por venta
Características:	El sistema permitirá registrar las ventas realizadas y descontará del inventario las unidades vendidas.
Descripción del requerimiento:	El sistema registrará los productos, cantidades y valores de cada venta, actualizará las existencias y generará el ingreso correspondiente; la venta no se registrará si alguna cantidad supera las existencias disponibles.
Requerimiento NO funcional:	RNF01, RNF03, RNF04, RNF07, RNF08, RNF09
Prioridad del requerimiento:	Alta

Nota. Elaboración propia.

Tabla 7

Requerimiento Funcional RF11: Registro de Ingresos y Egresos

Identificación del requerimiento:	RF11
Nombre del Requerimiento:	Registro de ingresos y egresos
Características:	El sistema permitirá registrar los movimientos económicos realizados en el negocio.
Descripción del requerimiento:	El usuario podrá registrar los ingresos generados por las ventas y los egresos relacionados con compras y gastos del negocio.
Requerimiento NO funcional:	RNF01, RNF02, RNF07
Prioridad del requerimiento:	Alta

Nota. Elaboración propia.

Tabla 8

Requerimiento Funcional RF15: Generación de Reportes

Identificación del requerimiento:	RF15
Nombre del Requerimiento:	Generación de reportes
Características:	El sistema permitirá consultar información relacionada con las operaciones del negocio.
Descripción del requerimiento:	El sistema permitirá generar reportes relacionados con inventario, ventas, movimientos de mercancía e ingresos y egresos en un periodo determinado.
Requerimiento NO funcional:	RNF01, RNF03, RNF07
Prioridad del requerimiento:	Media

Nota. Elaboración propia.

Requerimientos No Funcionales

Los requerimientos no funcionales establecen los criterios de calidad que debe cumplir el sistema para garantizar una experiencia de uso adecuada en el contexto real de la tienda Britny Max. Estos requerimientos no describen funciones específicas, sino las condiciones bajo las cuales el sistema debe operar, como su velocidad, seguridad, facilidad de uso y capacidad de crecimiento.

Durante el desarrollo, el requerimiento RNF02 se amplió para incluir cuatro niveles de acceso, el cierre automático de la sesión tras cinco minutos de inactividad y la protección de las direcciones web, tal como se registra en la Tabla 10.

Tabla 9

Requerimiento No Funcional RNF01: Tiempo de Respuesta

Identificación del requerimiento:	RNF01
Nombre del Requerimiento:	Tiempo de respuesta
Características:	El sistema deberá responder rápidamente durante las operaciones normales.
Descripción del requerimiento:	Las operaciones principales de consulta, registro y actualización deberán completarse en un tiempo máximo de 2 segundos bajo condiciones normales.
Requerimiento funcional relacionado:	RF02, RF03, RF05, RF07, RF11, RF15
Prioridad del requerimiento:	Alta

Nota. Elaboración propia.

Tabla 10

Requerimiento No Funcional RNF02: Seguridad de la Información

Identificación del requerimiento:	RNF02
Nombre del Requerimiento:	Seguridad de la información
Características:	El sistema deberá proteger la información y restringir el acceso según cuatro niveles de seguridad: administrador, supervisor, cajero y consulta.
Descripción del requerimiento:	El acceso requerirá usuario y contraseña; cada nivel tendrá permisos definidos que se aplicarán en la interfaz y en la base de datos; la sesión se cerrará automáticamente tras 5 minutos de inactividad y las direcciones web no expondrán tokens ni identificadores.
Requerimiento funcional relacionado:	RF01, RF04, RF07, RF11
Prioridad del requerimiento:	Alta

Nota. Elaboración propia.

Tabla 11

Requerimiento No Funcional RNF03: Facilidad de Uso

Identificación del requerimiento:	RNF03
Nombre del Requerimiento:	Facilidad de uso
Características:	La interfaz deberá ser clara, sencilla e intuitiva.
Descripción del requerimiento:	La propietaria deberá poder utilizar las funciones principales del sistema después de una capacitación inicial.
Requerimiento funcional relacionado:	RF02, RF03, RF04, RF05, RF07, RF15
Prioridad del requerimiento:	Alta

Nota. Elaboración propia.

Tabla 12

Requerimiento No Funcional RNF04: Disponibilidad del Sistema

Identificación del requerimiento:	RNF04
Nombre del Requerimiento:	Disponibilidad del sistema
Características:	El sistema deberá estar disponible mediante un navegador web durante el horario de operación de la tienda.
Descripción del requerimiento:	El sistema podrá ser utilizado durante el horario comercial, salvo interrupciones programadas o fallas externas.
Requerimiento funcional relacionado:	RF01, RF02, RF03, RF05, RF07, RF11, RF15
Prioridad del requerimiento:	Media

Nota. Elaboración propia.

Tabla 13

Requerimiento No Funcional RNF05: Compatibilidad

Identificación del requerimiento:	RNF05
Nombre del Requerimiento:	Compatibilidad
Características:	El sistema deberá funcionar correctamente en navegadores web actuales y diferentes tamaños de pantalla.
Descripción del requerimiento:	Las funciones principales deberán funcionar correctamente en navegadores como Chrome, Edge y Firefox y en dispositivos de escritorio y móviles.
Requerimiento funcional relacionado:	RF03, RF05
Prioridad del requerimiento:	Media

Nota. Elaboración propia.

Tabla 14

Requerimiento No Funcional RNF07: Integridad de los Datos

Identificación del requerimiento:	RNF07
Nombre del Requerimiento:	Integridad de los datos
Características:	El sistema deberá mantener la consistencia de la información registrada.
Descripción del requerimiento:	Los movimientos registrados deberán actualizar correctamente la información relacionada y evitar registros incompletos o inconsistentes.
Requerimiento funcional relacionado:	RF02, RF04, RF07, RF11, RF15
Prioridad del requerimiento:	Alta

Nota. Elaboración propia.

Tabla 15

Requerimiento No Funcional RNF08: Escalabilidad

Identificación del requerimiento:	RNF08
Nombre del Requerimiento:	Escalabilidad
Características:	El sistema deberá permitir el crecimiento de la información almacenada sin afectar significativamente su funcionamiento.
Descripción del requerimiento:	La base de datos PostgreSQL gestionada mediante Supabase deberá soportar el crecimiento del número de productos y movimientos registrados.
Requerimiento funcional relacionado:	RF02, RF07, RF11, RF15
Prioridad del requerimiento:	Media

Nota. Elaboración propia.

Tabla 16

Requerimiento No Funcional RNF09: Respaldo de Información

Identificación del requerimiento:	RNF09
Nombre del Requerimiento:	Respaldo de información
Características:	La información almacenada deberá contar con mecanismos que permitan su recuperación.
Descripción del requerimiento:	Los datos deberán poder recuperarse mediante los mecanismos de respaldo disponibles en la plataforma utilizada.
Requerimiento funcional relacionado:	RF02, RF07, RF11, RF15
Prioridad del requerimiento:	Media

Nota. Elaboración propia.

Diseño del Sistema

En esta sección se presenta el diseño del sistema de información web propuesto para la gestión de inventarios, ingresos y egresos de la tienda Britny Max, ubicada en el municipio de Rionegro, Santander. Para realizar este diseño se tuvieron en cuenta los requerimientos funcionales y no funcionales definidos durante el análisis de las necesidades del establecimiento.

El diseño comprende los diagramas de casos de uso correspondientes a los requerimientos funcionales seleccionados, el diagrama de clases y los prototipos de las interfaces elaborados en Figma, que sirvieron como base visual de las pantallas finales presentadas en el capítulo de desarrollo. Estos elementos permiten representar las funciones que tendrá el sistema, la

organización de sus elementos principales y la manera en que los usuarios interactúan con la aplicación.

Los diagramas se desarrollaron de forma individual para representar los requerimientos funcionales seleccionados. Por esta razón, cada uno se explica por separado, conservando su identificación y propósito dentro del sistema.

Diagramas de Casos de Uso

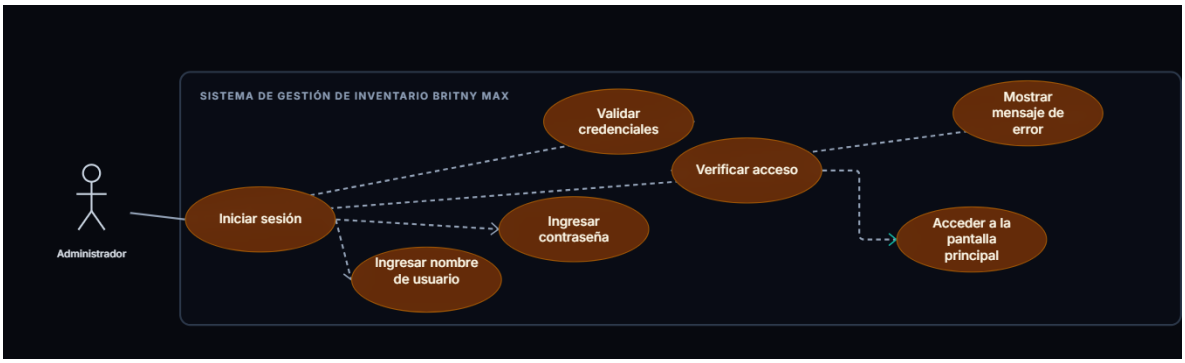
Los diagramas de casos de uso permiten representar las acciones que podrá realizar el usuario dentro del sistema y la relación entre el administrador y las funcionalidades de la aplicación. Para Britny Max, estos diagramas se organizaron de acuerdo con los requerimientos funcionales definidos durante la etapa de análisis.

Diagrama de Casos de Uso del RF01: Autenticación de Usuario. Este diagrama representa el acceso al sistema de información web mediante el ingreso de un usuario y una contraseña. Su propósito es establecer un mecanismo de autenticación que permita el ingreso del administrador autorizado.

A través de esta funcionalidad, el usuario podrá ingresar sus credenciales para que el sistema las valide. Como resultado, se permitirá el acceso cuando los datos sean correctos o se informará que las credenciales no son válidas. La autenticación constituye el punto de entrada a las demás funciones administrativas del sistema.

Figura 1

Diagrama de Casos de Uso del RF01: Autenticación de Usuario



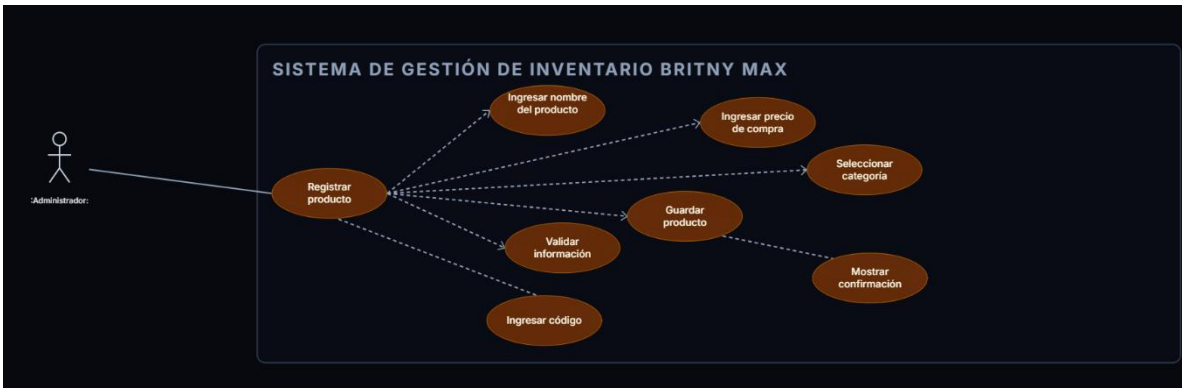
Nota. Elaboración propia.

Diagrama de Casos de Uso del RF02: Registro de Productos. Este diagrama representa las acciones relacionadas con el registro de los productos que comercializa la tienda Britny Max. Su finalidad es permitir que la información de la mercancía quede almacenada de manera organizada para facilitar su administración y consulta.

El registro contempla datos como el nombre del producto, el código de barras, la categoría, el precio de compra, el precio de venta y la cantidad inicial. Esta información servirá como base para identificar los productos y llevar el control de las existencias dentro del sistema.

Figura 2

Diagrama de Casos de Uso del RF02: Registro de Productos



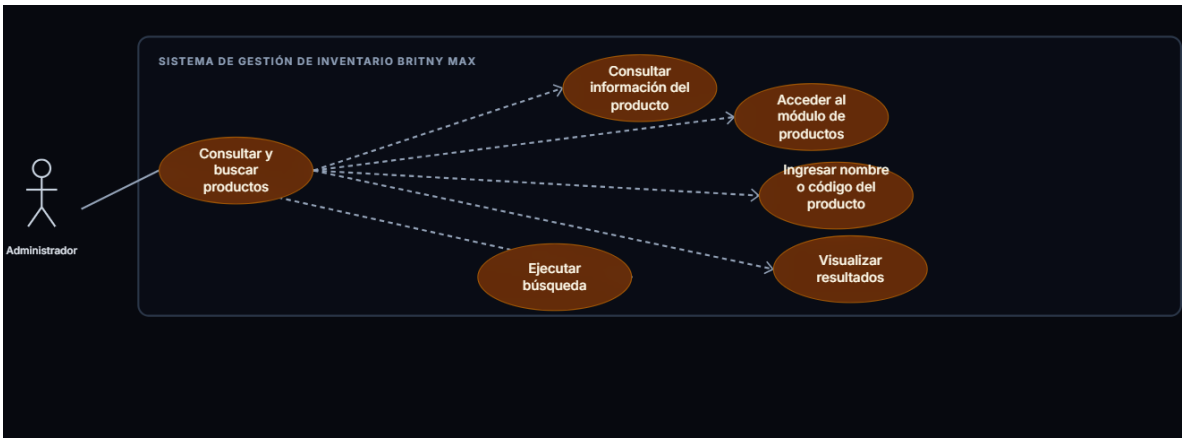
Nota. Elaboración propia.

Diagrama de Casos de Uso del RF03: Consulta y Búsqueda de Productos. Este diagrama representa las acciones destinadas a localizar y consultar la información de los productos registrados. La funcionalidad busca facilitar la revisión de los datos del catálogo sin tener que depender exclusivamente de los registros manuales utilizados actualmente en la tienda.

La consulta podrá realizarse mediante el nombre, la categoría o el código de barras del producto. De esta manera, el administrador podrá identificar los artículos registrados y consultar información relacionada con ellos, incluida su cantidad disponible.

Figura 3

Diagrama de Casos de Uso del RF03: Consulta y Búsqueda de Productos



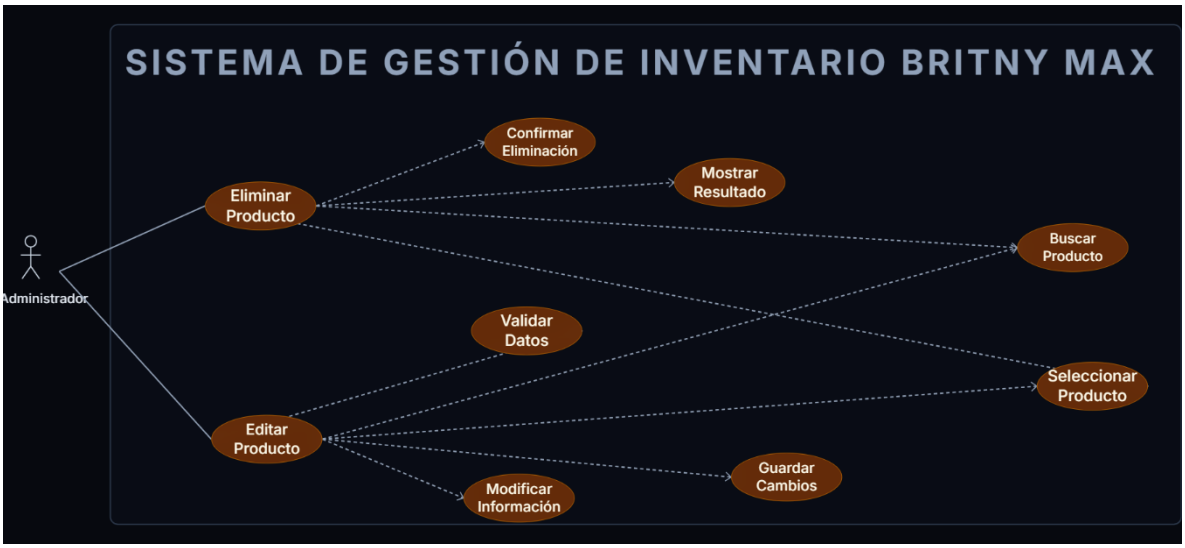
Nota. Elaboración propia.

Diagrama de Casos de Uso del RF04: Edición y Eliminación de Productos. Este diagrama representa las acciones relacionadas con la administración de los productos que ya se encuentran registrados. Su propósito es permitir que la información del catálogo pueda mantenerse actualizada cuando sea necesario realizar modificaciones o retirar un producto.

La funcionalidad contempla la selección del producto correspondiente, la modificación de sus datos y su eliminación del catálogo cuando proceda. Con ello, se busca mantener organizada la información de los artículos que maneja la tienda.

Figura 4

Diagrama de Casos de Uso del RF04: Edición y Eliminación de Productos



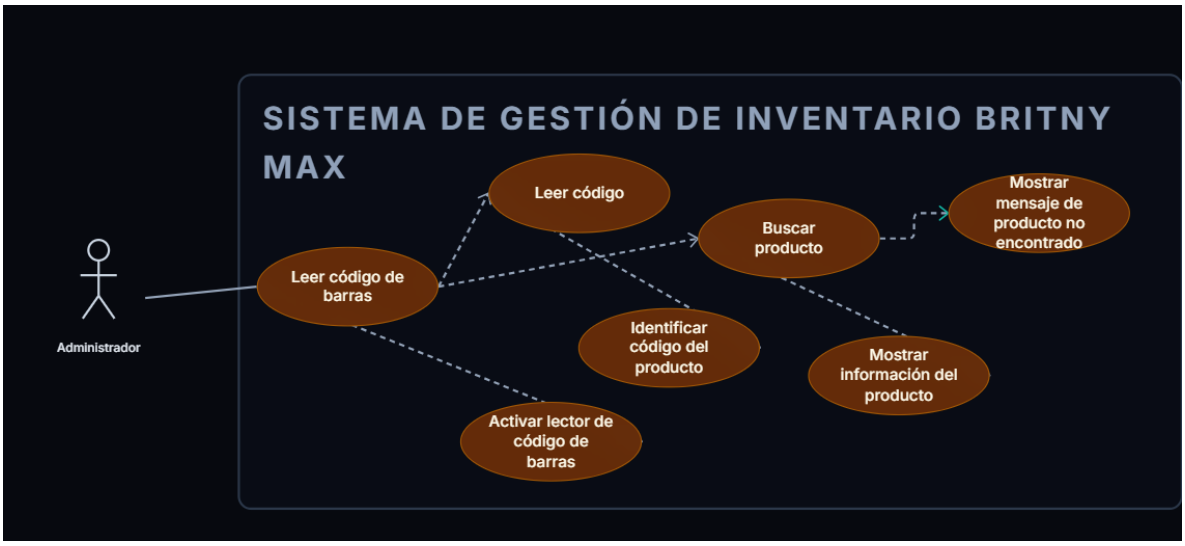
Nota. Elaboración propia.

Diagrama de Casos de Uso del RF05: Lectura de Código de Barras. Este diagrama representa la identificación de productos mediante la lectura de su código de barras. La funcionalidad se plantea para facilitar las operaciones de consulta y las actividades relacionadas con las entradas y salidas de mercancía.

El sistema utilizará un lector de código de barras para reconocer el código del producto y recuperar la información asociada. Esta función busca reducir la necesidad de buscar manualmente cada artículo y agilizar las operaciones que requieren identificar productos.

Figura 5

Diagrama de Casos de Uso del RF05: Lectura de Código de Barras



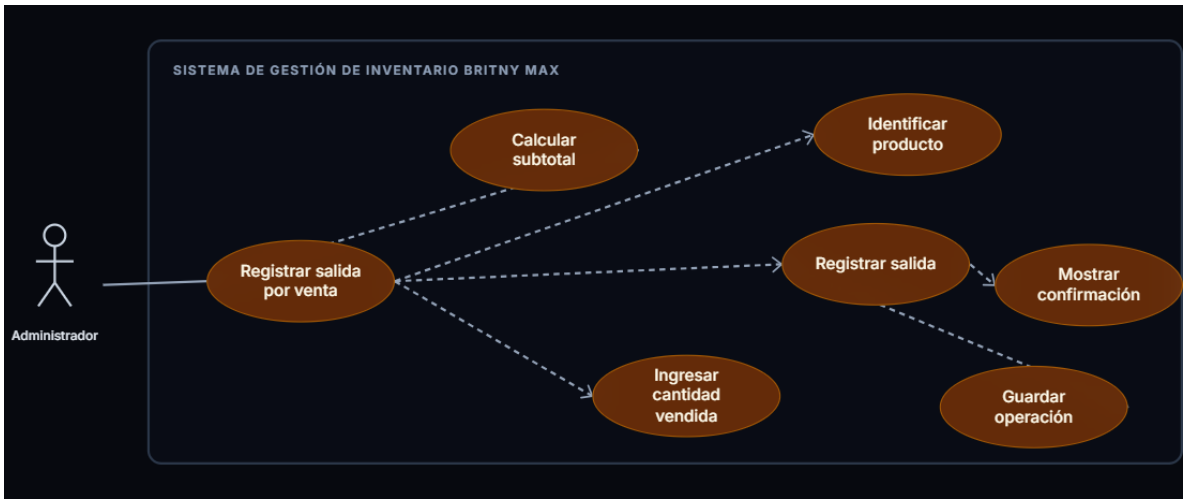
Nota. Elaboración propia.

Diagrama de Casos de Uso del RF07: Registro de Salidas por Venta. Este diagrama representa las acciones relacionadas con el registro de las ventas y la salida de los productos del inventario. Su propósito es mantener actualizadas las cantidades disponibles a medida que se registran las operaciones de venta.

El proceso contempla la identificación del producto, el registro de la cantidad vendida, el valor de la venta y la fecha correspondiente. Una vez registrada la operación, el sistema deberá actualizar las existencias de acuerdo con las unidades vendidas y conservar la información de la venta para su posterior consulta.

Figura 6

Diagrama de Casos de Uso del RF07: Registro de Salidas por Venta



Nota. Elaboración propia.

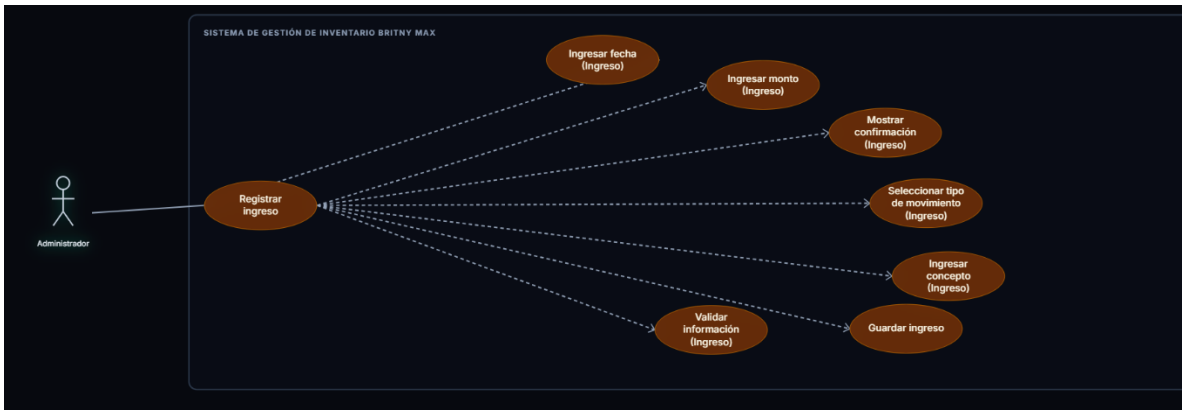
Diagrama de Casos de Uso del RF11: Registro de Ingresos y Egresos. Este diagrama representa las acciones relacionadas con el registro de los movimientos económicos de la tienda. Su finalidad es organizar la información de los ingresos y egresos para facilitar el seguimiento de los recursos económicos del establecimiento.

La funcionalidad contempla el registro del tipo de movimiento, su concepto, el valor y la fecha. Los ingresos estarán relacionados con el dinero recibido por las ventas, mientras que los egresos incluirán las compras de mercancía y otros gastos del negocio, como los pagos de servicios públicos.

El sistema contará con opciones separadas para consultar y registrar los ingresos y los egresos, de manera que cada tipo de movimiento pueda gestionarse de forma independiente.

Figura 7

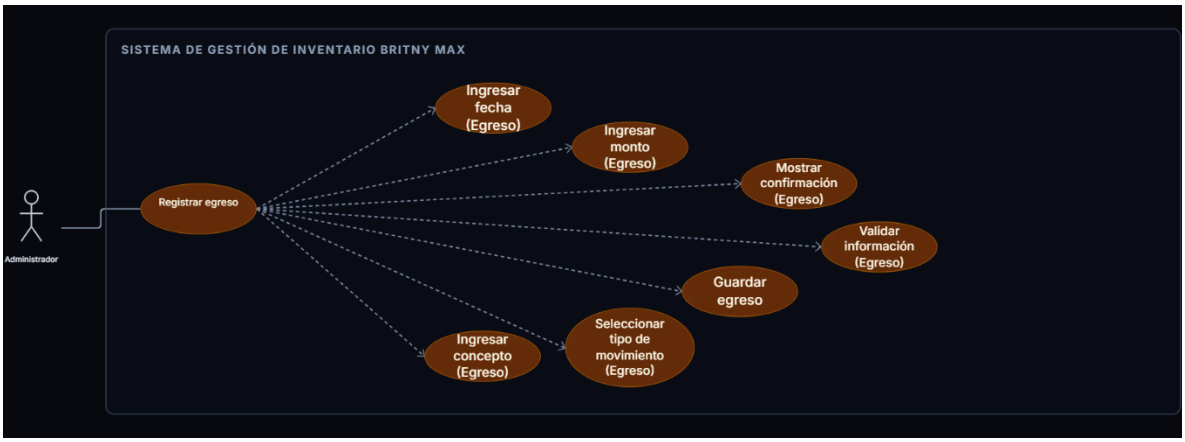
Diagrama de Casos de Uso del RF11: Registro de Egresos



Nota. Elaboración propia.

Figura 8

Diagrama de Casos de Uso del RF11: Registro de Ingresos



Nota. Elaboración propia.

Diagrama de Casos de Uso del RF15: Generación de Reportes. Este diagrama representa las acciones relacionadas con la consulta de información resumida sobre las operaciones

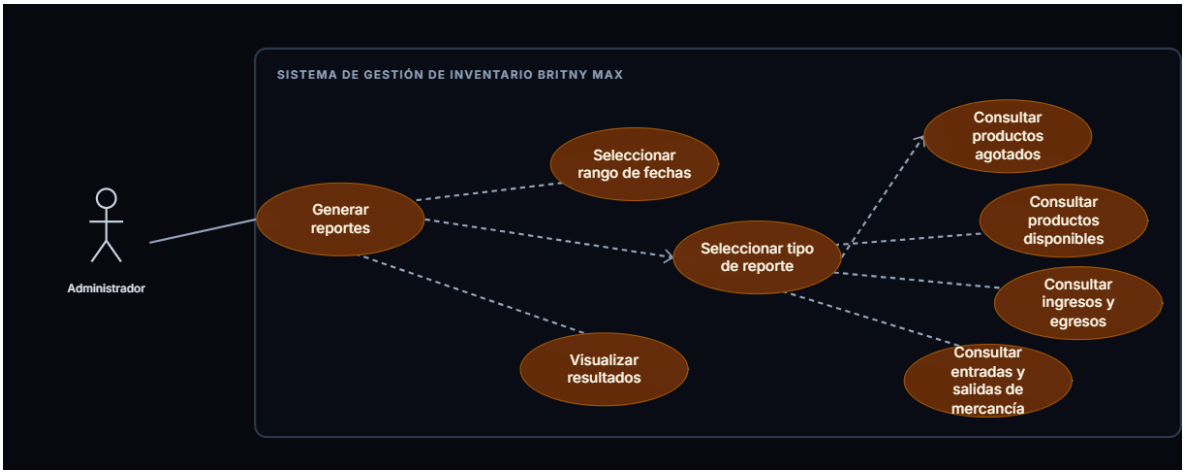
de Britny Max. Su objetivo es facilitar al administrador la revisión de los datos registrados y apoyar la toma de decisiones sobre el funcionamiento de la tienda.

Los reportes previstos abarcan información sobre el inventario, las ventas, los movimientos de mercancía y los ingresos y egresos. Para los reportes económicos se contempla la selección de un periodo determinado, lo que permitirá consultar los movimientos registrados durante las fechas seleccionadas.

Esta funcionalidad busca ofrecer una forma más organizada de revisar la información que actualmente se registra de manera manual.

Figura 9

Diagrama de Casos de Uso del RF15: Generación de Reportes



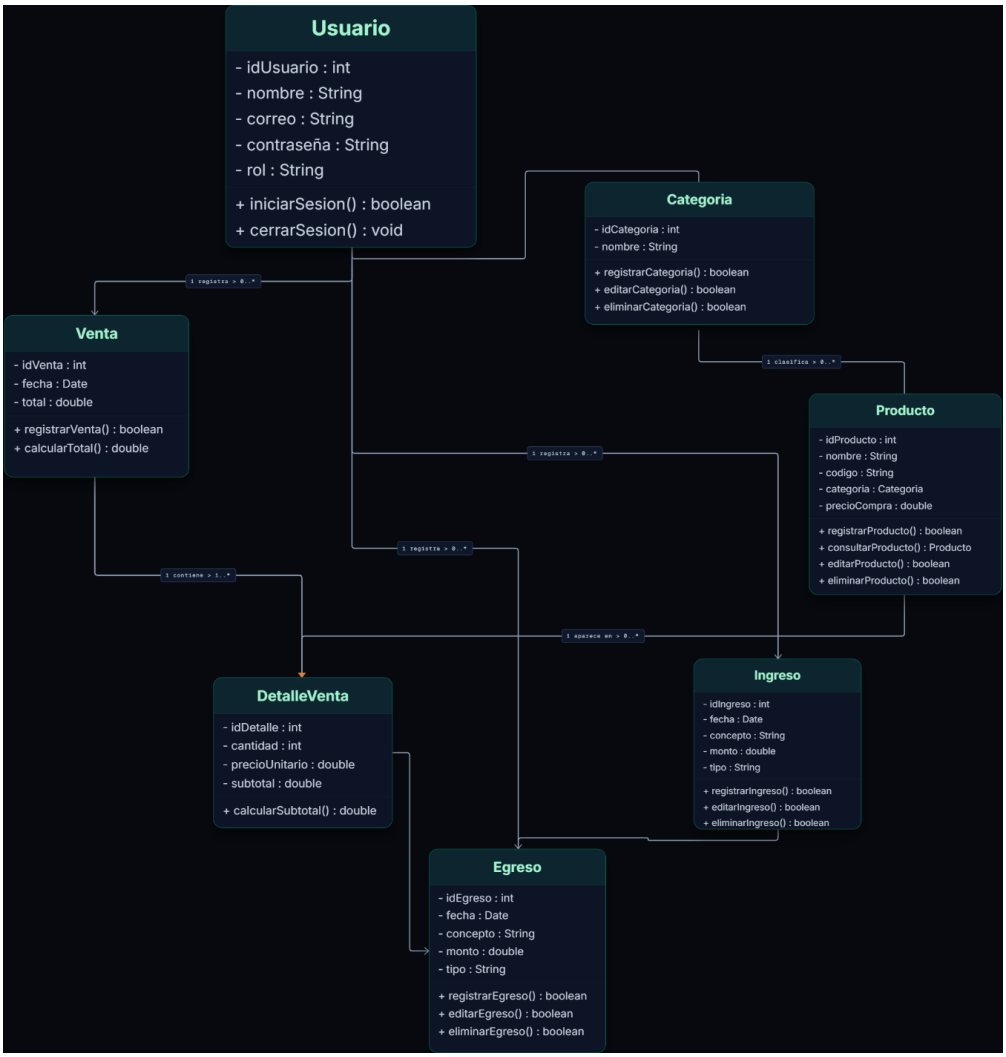
Nota. Elaboración propia.

Diagrama de Clases

El diagrama de clases (Figura 10) representa las entidades principales del sistema —usuario, categoría, producto, venta, detalle de venta, ingreso y egreso— y sus relaciones. Durante la implementación, este modelo se tradujo en las tablas de la base de datos descritas en el capítulo de desarrollo, a las que se añadió una tabla de movimientos de inventario para conservar la trazabilidad de cada entrada, salida y ajuste.

Figura 10

Diagrama de Clases del Sistema



Nota. Elaboración propia.

Desarrollo del Sistema

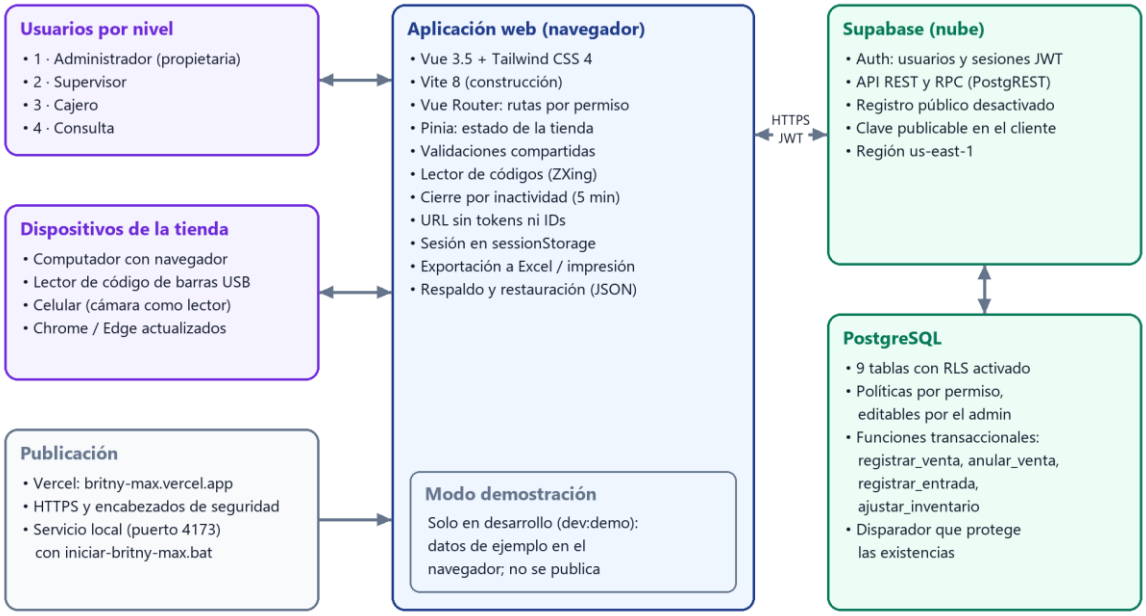
En este capítulo se presenta la construcción del sistema de información web a partir del diseño anterior. Se describen la arquitectura, las tecnologías empleadas, el modelo de datos implementado, los mecanismos de seguridad y los módulos desarrollados, acompañados de capturas de las pantallas del sistema en funcionamiento. Las capturas se tomaron con los datos de ejemplo del modo de demostración, que aplica las mismas reglas de negocio y los mismos niveles de acceso que la versión conectada a Supabase; en ellas no aparecen los avisos propios de ese modo, que no forman parte de la versión publicada.

Arquitectura del Sistema

El sistema es una aplicación web de una sola página que se ejecuta en el navegador y se comunica por HTTPS con Supabase (Figura 11). La aplicación se sirve desde el computador de la tienda mediante un servicio local que se inicia con el archivo iniciar-britny-max.bat y queda en segundo plano, de modo que también puede abrirse desde otros equipos o celulares conectados a la misma red. Las reglas que protegen la información no dependen solo de la interfaz: se repiten en la base de datos mediante políticas de seguridad a nivel de fila y funciones transaccionales, de forma que ninguna operación pueda saltarse las validaciones aunque se intente acceder directamente a la API.

Figura 11

Arquitectura del Sistema Britny Max



Nota. Elaboración propia.

Tecnologías Utilizadas

La Tabla 17 resume las herramientas empleadas en la construcción del sistema y la versión instalada en el proyecto al cierre de las pruebas.

Tabla 17

Tecnologías Utilizadas en el Desarrollo

Componente	Tecnología	Versión	Función en el sistema
Interfaz de usuario	Vue.js	3.5.44	Componentes reactivos de las pantallas
Estilos	Tailwind CSS	4.3.3	Diseño adaptable a computador y celular
Construcción	Vite	8.3.5	Servidor de desarrollo, compilación y servicio local
Estado y rutas	Pinia / Vue Router	3.0.4 / 4.6.4	Estado compartido y rutas protegidas por permiso
Backend	Supabase (supabase-js)	2.118.0	Autenticación, API REST y llamadas a funciones
Base de datos	PostgreSQL	Gestionada por Supabase	Tablas, políticas RLS, funciones y disparadores
Lectura de códigos	ZXing (@zxing/browser)	0.1.5	Lectura con la cámara del dispositivo
Íconos	Lucide	1.0.0	Iconografía de la interfaz
Pruebas	node:test y PGlite	Node.js 24.18 / 0.5.8	Pruebas unitarias y de base de datos automatizadas

Nota. Las versiones corresponden al archivo package-lock.json del proyecto. Elaboración propia.

Modelo de Datos

La base de datos se implementó en PostgreSQL con nueve tablas (Tabla 18). Las existencias de los productos solo cambian mediante funciones transaccionales —crear_producto, registrar_venta, anular_venta, registrar_entrada y ajustar_inventario— que validan los datos, actualizan el inventario, registran el movimiento y generan el ingreso o egreso correspondiente en una sola transacción; si alguna validación falla, no se guarda ningún cambio. Un disparador impide modificar las existencias por fuera de estas funciones, y las restricciones de la base de datos repiten

los mismos límites de las validaciones de la interfaz (por ejemplo, precios positivos, cantidades enteras y códigos de barras únicos entre productos activos).

Tabla 18

Tablas de la Base de Datos

Tabla	Información que almacena
usuarios	Perfil de cada persona autorizada: nombre, nivel de acceso y estado. El correo y la contraseña los administra Supabase Auth.
permisos_rol	Permisos que la administradora asigna a los niveles supervisor, cajero y consulta.
categorias	Clasificación de los productos (granos, lácteos, bebidas, aseo, entre otras).
productos	Catálogo con código de barras, categoría, precios de compra y venta, existencias y stock mínimo.
ventas	Encabezado de cada venta: fecha, total, usuario que la registró y estado (registrada o anulada).
detalle_venta	Productos, cantidades, precios unitarios y subtotales de cada venta.
movimientos_inventario	Trazabilidad de entradas, salidas por venta, anulaciones y ajustes por conteo físico, con las existencias resultantes.
ingresos	Dinero recibido por ventas y por otros conceptos.
egresos	Compras de mercancía, servicios públicos, arriendo y otros gastos del negocio.

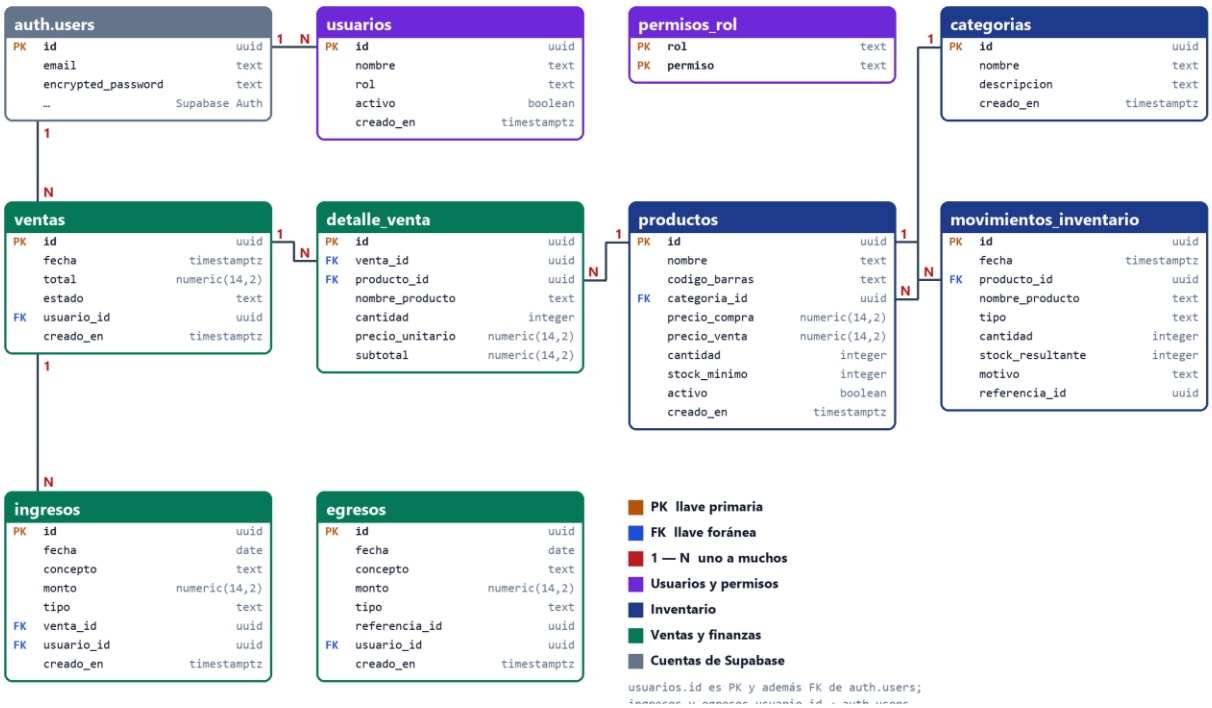
Nota. Elaboración propia a partir del archivo supabase/schema.sql.

La Figura 12 presenta el modelo entidad-relación implementado, con las llaves primarias y foráneas de cada tabla. La tabla usuarios amplía las cuentas de Supabase Auth (auth.users) con el nombre, el nivel y el estado de cada persona; una venta tiene uno o muchos detalles, cada detalle y cada movimiento de inventario se relacionan con un producto, y el ingreso de una venta conserva la referencia a ella para retirarlo si se anula. Respecto al diagrama de clases del diseño (Figura 10), se agregaron las tablas movimientos_inventario, para la trazabilidad de las existencias, y

permisos_rol, para los permisos que asigna la administradora. El script completo que crea la base de datos, con sus restricciones, políticas de seguridad y funciones, está en el Anexo C.

Figura 12

Modelo Entidad-Relación de la Base de Datos



Nota. Elaboración propia a partir del archivo supabase/schema.sql.

Seguridad y Niveles de Acceso

El sistema maneja cuatro niveles de acceso: el nivel 1, administrador, corresponde a la propietaria y tiene control total, incluida la gestión de usuarios y la restauración de respaldos; el nivel 2, supervisor, opera el negocio completo pero no administra usuarios ni restaura información; el nivel 3, cajero, registra ventas y consulta el inventario, sin acceso a las finanzas ni a la edición

de productos; y el nivel 4, consulta, puede revisar el inventario, las ventas, las finanzas y los reportes, pero no modificar nada. La Tabla 19 presenta la matriz inicial de permisos y la Figura 13 muestra el módulo en el que la administradora crea las cuentas y asigna los niveles.

Tabla 19

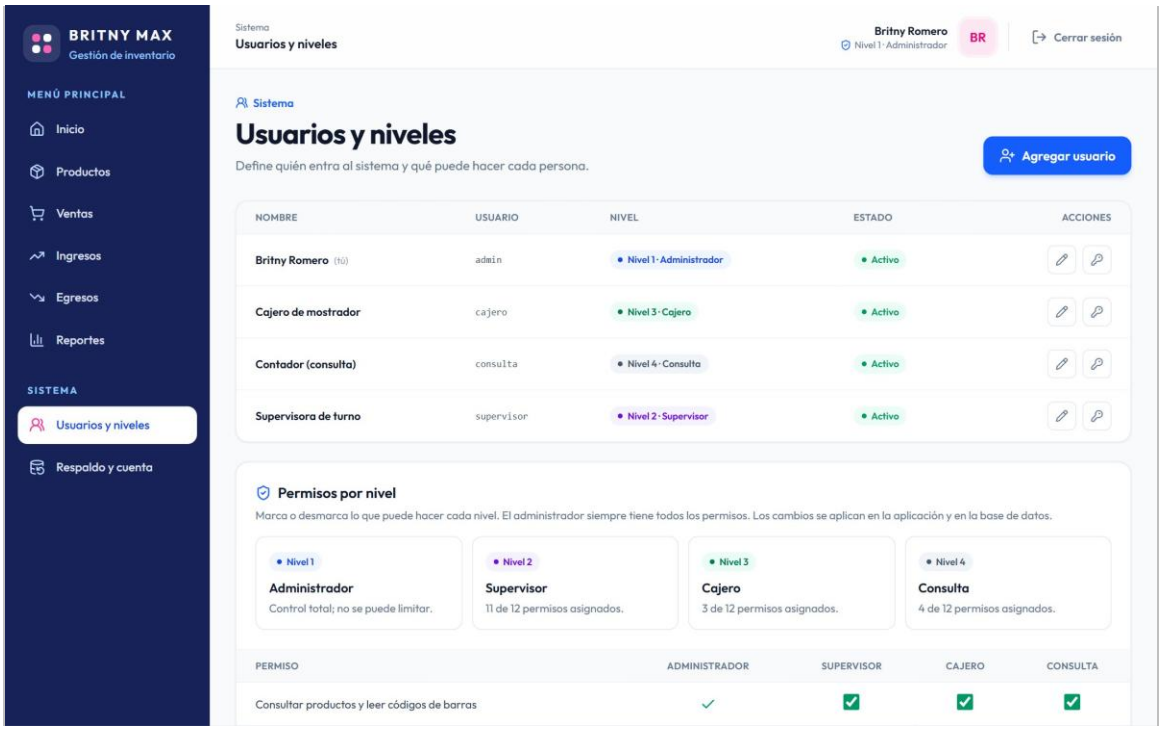
Matriz Inicial de Permisos por Nivel de Acceso

Permiso	Administrador	Supervisor	Cajero	Consulta
Ver inventario y productos	Sí	Sí	Sí	Sí
Crear y editar productos y categorías	Sí	Sí	No	No
Registrar entradas de mercancía	Sí	Sí	No	No
Hacer conteo físico y ajustes	Sí	Sí	No	No
Registrar ventas	Sí	Sí	Sí	No
Ver historial de ventas	Sí	Sí	Sí	Sí
Anular ventas	Sí	Sí	No	No
Ver ingresos y egresos	Sí	Sí	No	Sí
Registrar y editar ingresos y egresos	Sí	Sí	No	No
Ver reportes	Sí	Sí	No	Sí
Descargar respaldo	Sí	Sí	No	No
Restaurar respaldo	Sí	No	No	No
Gestionar usuarios y niveles	Sí	No	No	No

Nota. Matriz inicial definida en src/lib/permisos.js y en la tabla permisos_rol; la administradora puede modificarla. Elaboración propia.

Figura 13

Módulo de Usuarios y Niveles de Acceso



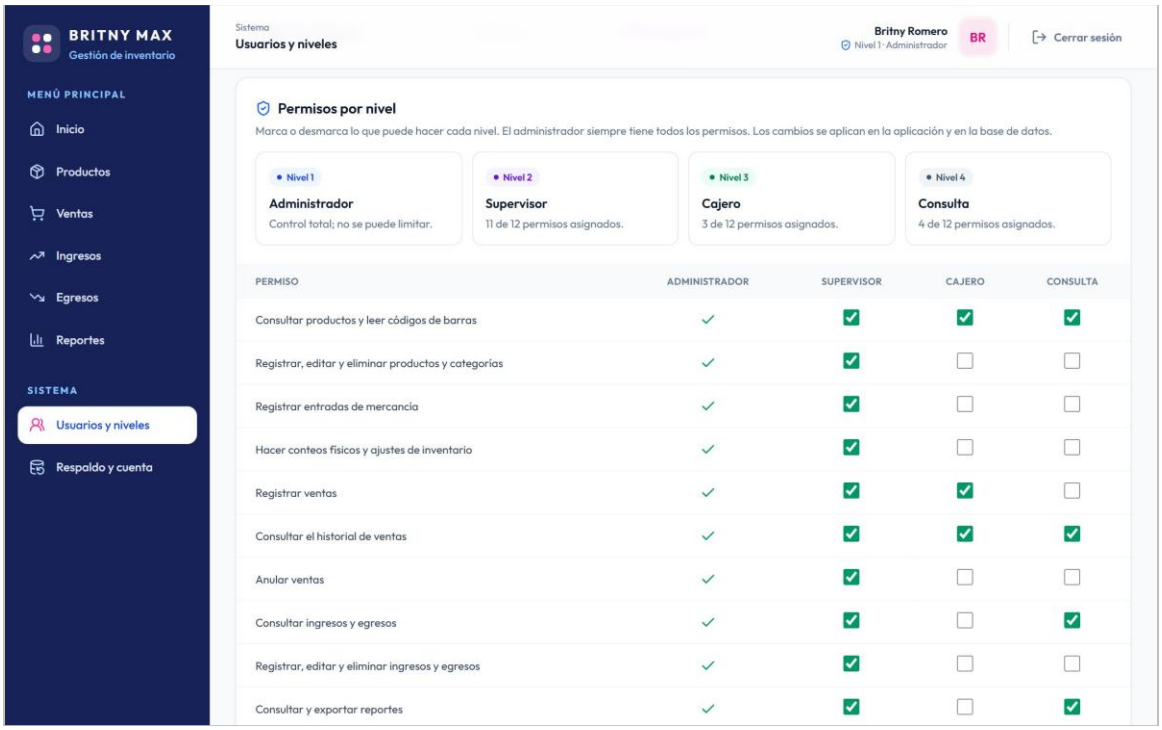
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

A solicitud del usuario, los permisos de los niveles 2 a 4 se volvieron configurables: desde «Usuarios y niveles», la administradora marca o desmarca cada permiso y lo guarda (Figura 14). La matriz se almacena en la tabla permisos_rol, y las políticas y funciones de la base de datos la consultan, de modo que un permiso retirado queda bloqueado tanto en la interfaz como en la API. Administrar usuarios es un permiso exclusivo del administrador, y al marcar un permiso se agregan los que necesita para funcionar (por ejemplo, registrar ventas exige consultar productos). Los

cambios llegan a las personas conectadas, como máximo, 30 segundos después, al cambiar de pantalla y sin cerrar su sesión.

Figura 14

Permisos por Nivel Editables por la Administradora



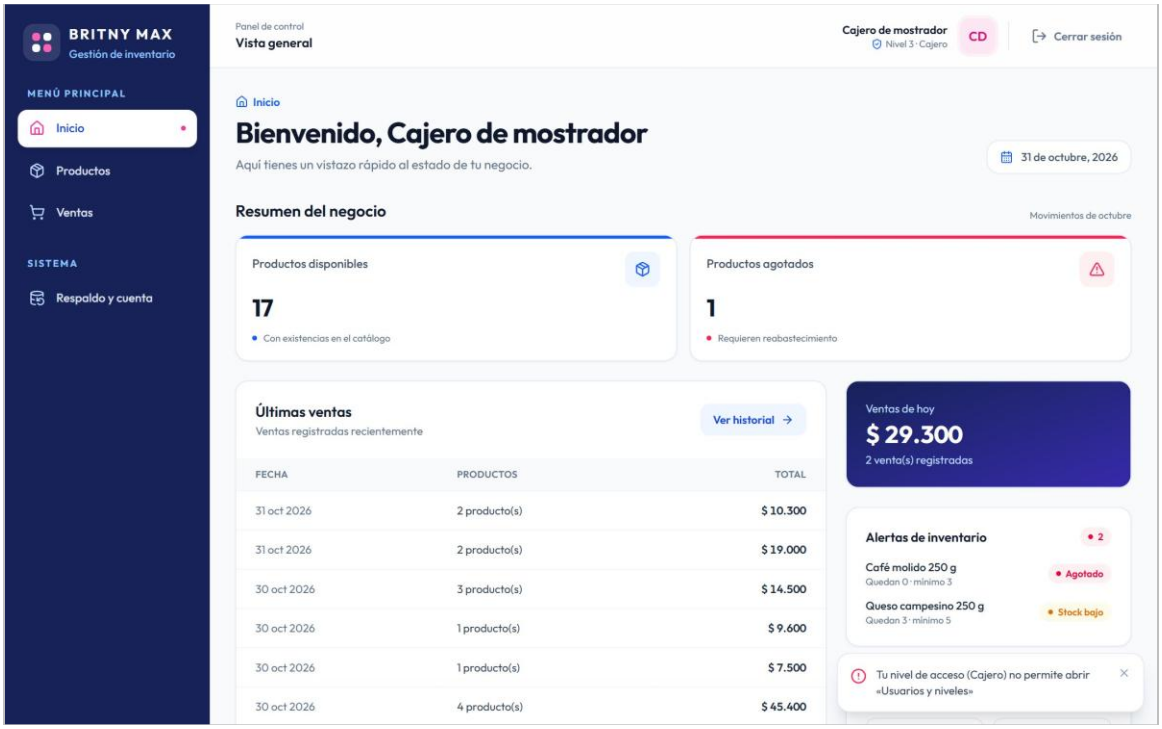
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Los permisos se aplican en dos capas. En la interfaz, el menú solo muestra las opciones permitidas y cada ruta verifica el permiso antes de abrirse: si un cajero intenta entrar a «Usuarios y niveles», el sistema lo devuelve al inicio con un aviso (Figura 15). En la base de datos, 24 políticas de seguridad a nivel de fila y las funciones transaccionales verifican de nuevo los

permisos del usuario, de modo que una persona sin permiso tampoco puede leer ni modificar información llamando directamente a la API. El registro público de usuarios está desactivado y la aplicación solo utiliza la clave publicable de Supabase; la clave secreta nunca se incluye en el navegador.

Figura 15

Acceso Restringido para el Nivel Cajero



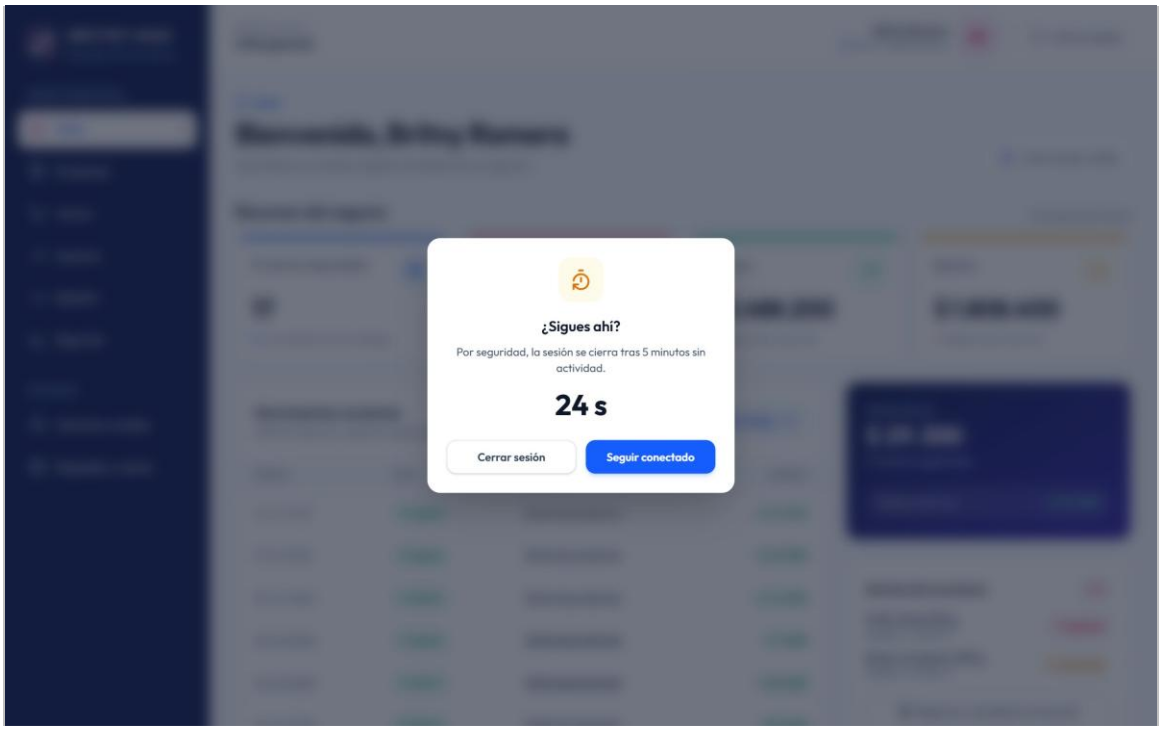
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Para cumplir el RNF02, la sesión se cierra automáticamente tras cinco minutos sin actividad. Treinta segundos antes, el sistema muestra un aviso con cuenta regresiva que permite

seguir conectado (Figura 16); la hora de la última actividad se guarda con la hora real del equipo, por lo que el cierre también ocurre si el computador se suspende o la pestaña queda oculta. La sesión se guarda solo en el almacenamiento de la pestaña, que se borra al cerrar el navegador, y la opción «Recordar usuario» conserva únicamente el nombre de usuario. Además, las direcciones web no muestran identificadores de registros y, si llegan con tokens de acceso, el sistema los elimina antes de cargar la aplicación; el servicio local agrega encabezados de seguridad como la política de seguridad de contenido.

Figura 16

Aviso de Cierre de Sesión por Inactividad



Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Despliegue y Puesta en Producción

El sistema quedó publicado en Vercel, en la dirección https://britny-max.vercel.app, a partir del repositorio privado del proyecto en GitHub. Cada vez que se sube un cambio al repositorio, Vercel compila la aplicación con Vite y la publica con HTTPS y con los encabezados de seguridad definidos en el archivo vercel.json. La conexión con Supabase se configura mediante tres variables de entorno —VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY y

VITE_AUTH_DOMAIN— y solo se utiliza la clave pública del proyecto; la clave secreta nunca se incluye en la aplicación. La Tabla 20 resume los entornos en los que funciona el sistema.

Tabla 20

Entornos de Funcionamiento del Sistema

Entorno	Dirección o comando	Uso
Producción (Vercel)	https://britny-max.vercel.app	Uso diario de la tienda desde el computador o el celular, con los datos en Supabase
Servicio local	http://localhost:4173 (iniciar-britny-max.bat)	Alternativa en el computador de la tienda y en su red local, conectada a la misma base de datos
Desarrollo	npm run dev:demo	Pruebas con datos de ejemplo guardados en el navegador; no se publica

Nota. Elaboración propia.

Para evitar que una versión mal configurada guarde información en el navegador, la compilación se detiene con un mensaje explícito si faltan las variables de Supabase. Durante la puesta en producción este control detectó una variable registrada con un nombre incorrecto en Vercel, y la publicación no se realizó hasta corregirla. Por la misma razón, el modo de demostración solo puede activarse en el equipo de desarrollo y no forma parte de la versión publicada.

En la base de datos se crearon cuatro cuentas, una por nivel de acceso (Tabla 21). Gracias a la variable VITE_AUTH_DOMAIN, cada persona ingresa con un nombre de usuario corto en lugar del correo completo. Las contraseñas iniciales se entregan por separado y deben cambiarse desde el módulo «Respaldo y cuenta» antes de la operación real.

Tabla 21

Cuentas de Usuario Creadas en Producción

Usuario	Nombre	Nivel	Estado
admin	Britny Romero (propietaria)	1 · Administrador	Activa
supervisor	Supervisora de turno	2 · Supervisor	Activa
cajero	Cajero de mostrador	3 · Cajero	Activa
consulta	Contador (consulta)	4 · Consulta	Activa

Nota. Las contraseñas no se incluyen en este documento por seguridad. Elaboración propia.

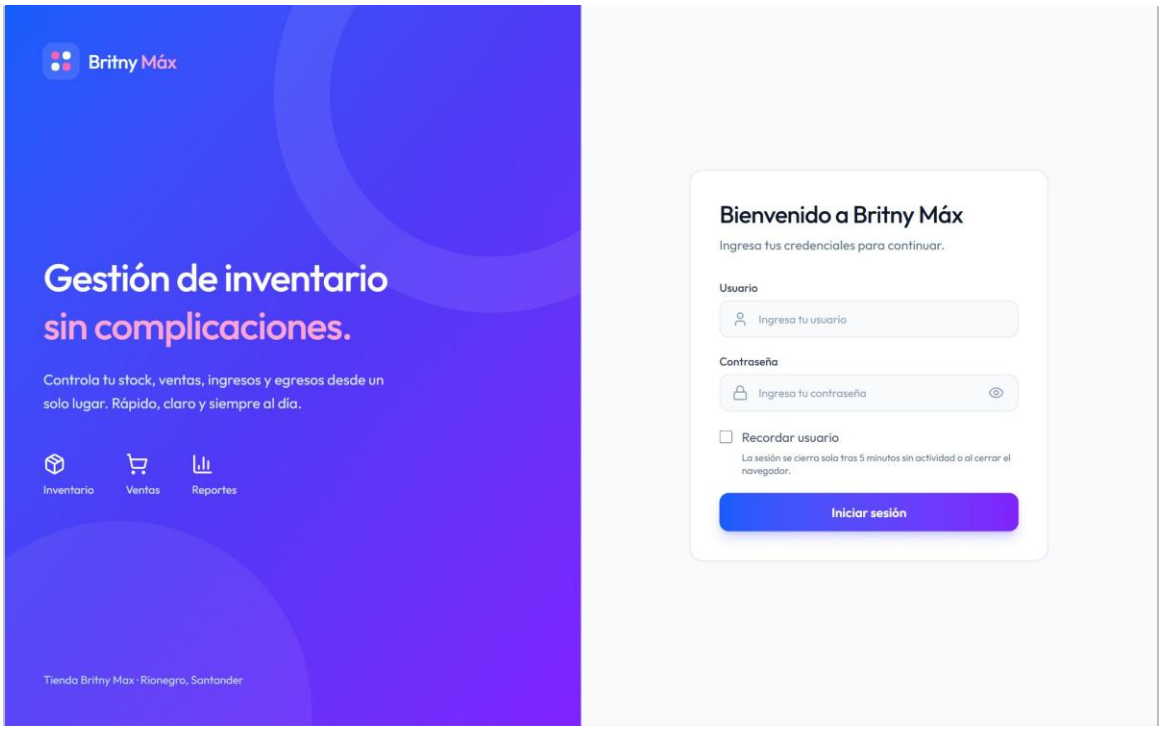
Módulos Implementados

Acceso al Sistema

La pantalla de inicio de sesión (Figura 17) solicita el usuario y la contraseña, valida que los campos no estén vacíos e informa cuando las credenciales no son válidas, en cumplimiento del RF01. En el modo de demostración se muestran cuentas de ejemplo; en la versión publicada solo ingresan las personas que la administradora autoriza.

Figura 17

Pantalla de Inicio de Sesión



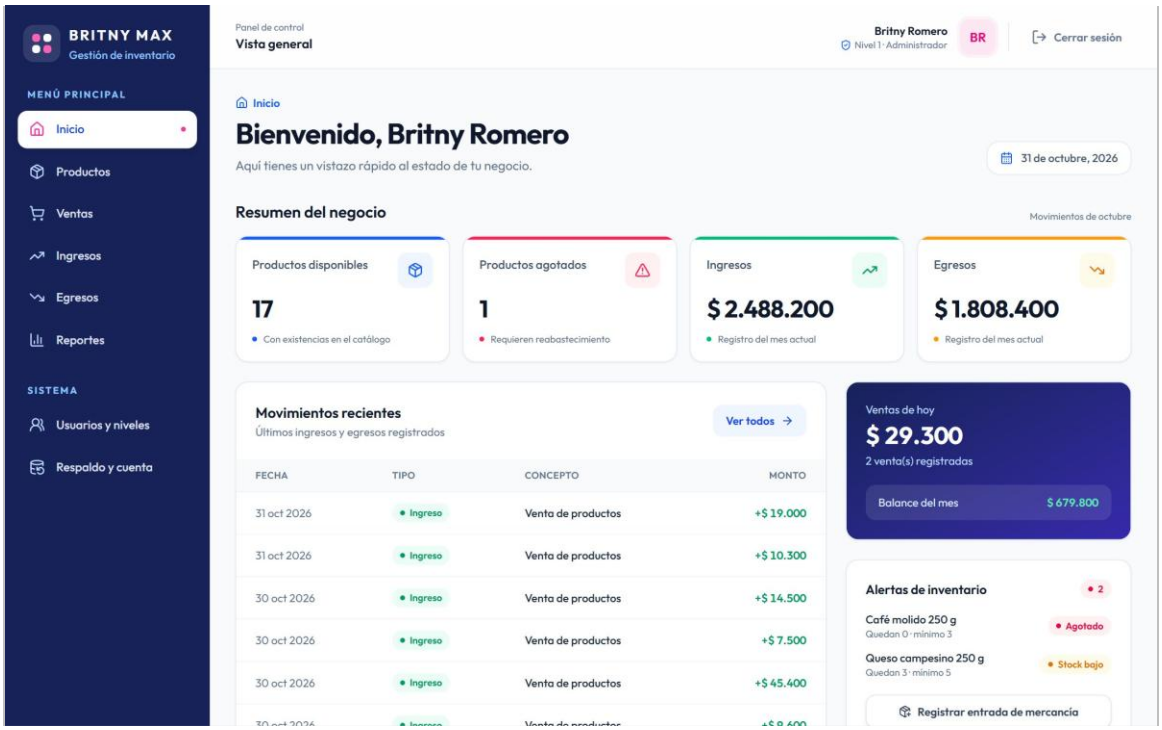
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Panel de Inicio

Al ingresar, cada usuario ve un resumen del negocio adaptado a su nivel: productos disponibles y agotados, ingresos y egresos del mes, ventas del día, movimientos recientes y alertas de inventario de los productos agotados o con existencias por debajo del mínimo (Figura 18).

Figura 18

Panel de Inicio del Nivel Administrador



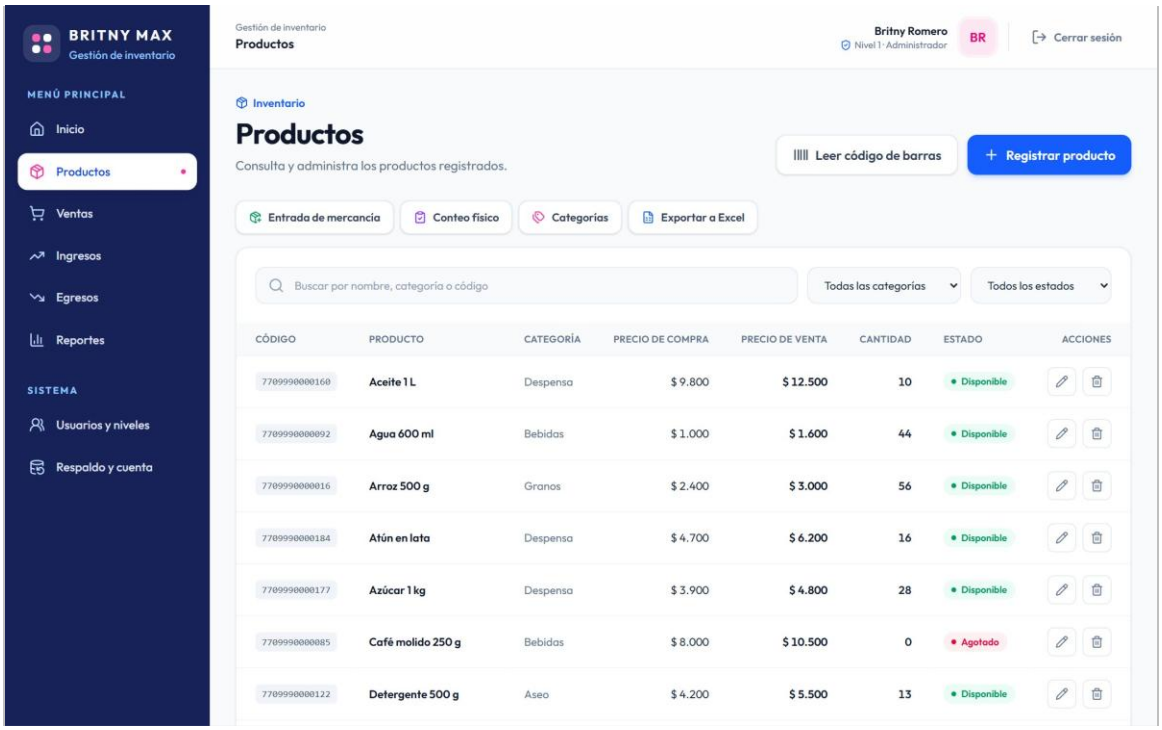
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Gestión de Productos

El módulo de productos (Figura 19) lista el catálogo con su código de barras, categoría, precios, existencias y estado, y permite buscar por nombre, categoría o código, filtrar, exportar a Excel, editar y retirar productos (RF03 y RF04). El formulario de registro (Figura 20) valida cada campo antes de guardar e informa el error junto al dato correspondiente (RF02).

Figura 19

Módulo de Productos

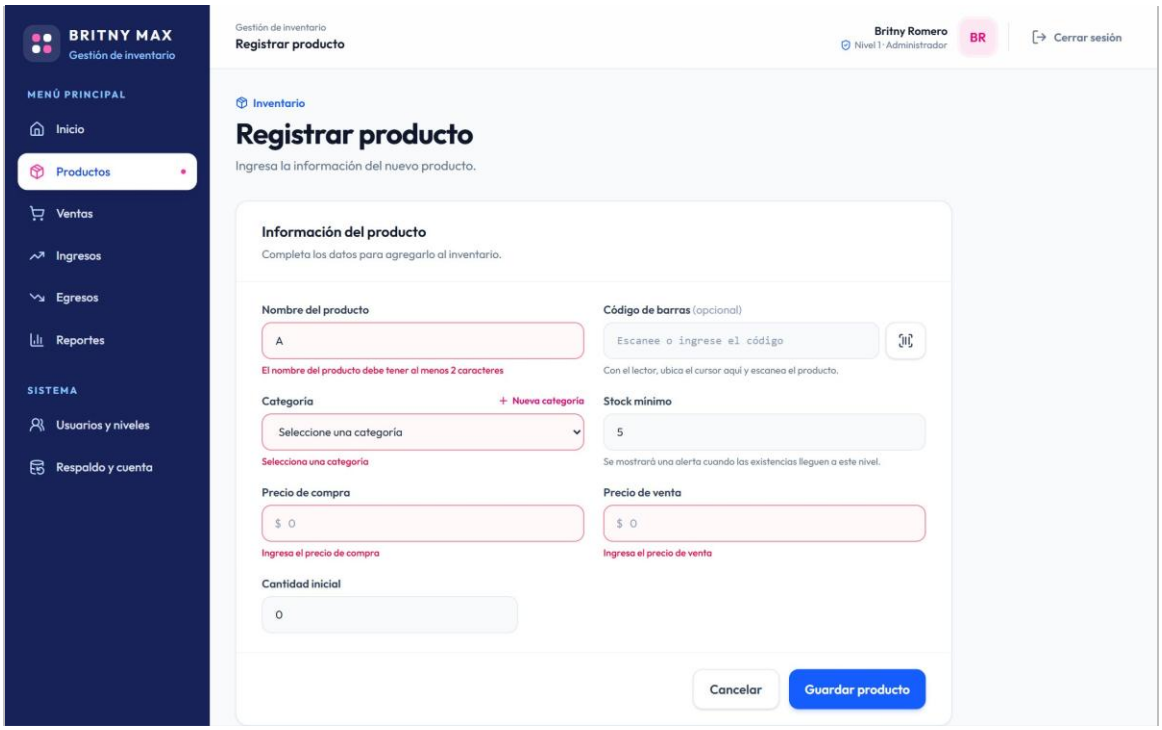


Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo).

Elaboración propia.

Figura 20

Registro de Producto con Mensajes de Validación



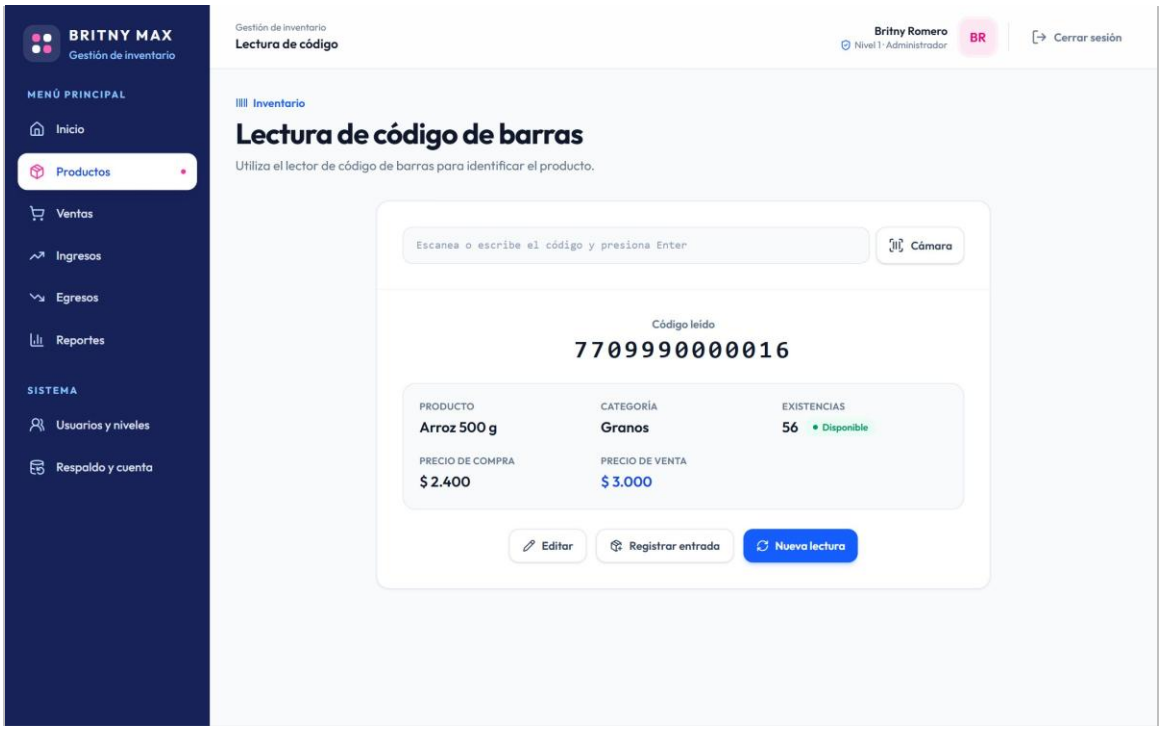
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Lectura de Código de Barras

La lectura de códigos (RF05) acepta el lector USB, que escribe el código y presiona Enter, o la cámara del celular o del computador. Al leer un código, el sistema muestra el producto, su categoría, precios y existencias, y ofrece accesos directos para editarlo o registrar una entrada (Figura 21). El mismo campo de lectura está disponible en ventas, entradas de mercancía y conteo físico.

Figura 21

Lectura de Código de Barras



Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

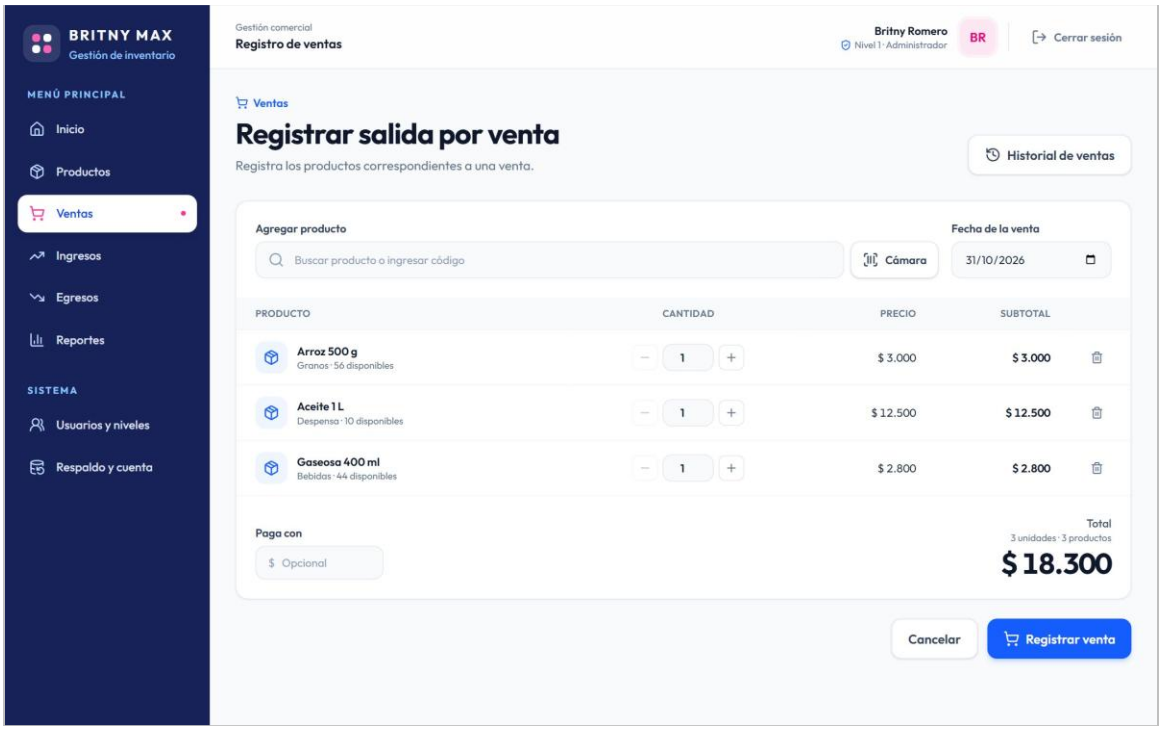
Registro de Ventas

El registro de salidas por venta (RF07) permite agregar productos por búsqueda o lectura de código, ajustar las cantidades, calcular el cambio y registrar la venta (Figura 22). Al confirmar, el sistema descuenta las existencias, guarda el detalle, registra el movimiento de inventario y genera el ingreso en una sola operación; si alguna cantidad supera las existencias, la venta se

rechaza sin modificar nada. El historial (Figura 23) muestra las ventas del período y permite anularlas, lo que devuelve las unidades al inventario y retira el ingreso.

Figura 22

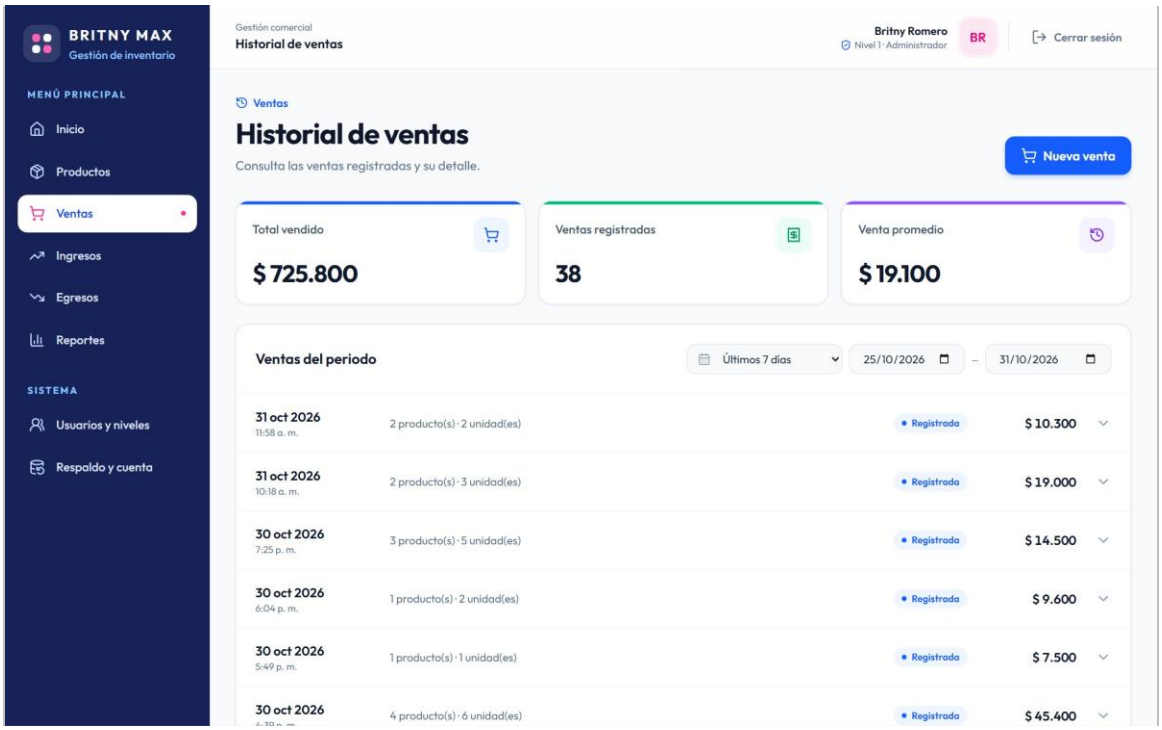
Registro de Salida por Venta



Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Figura 23

Historial de Ventas



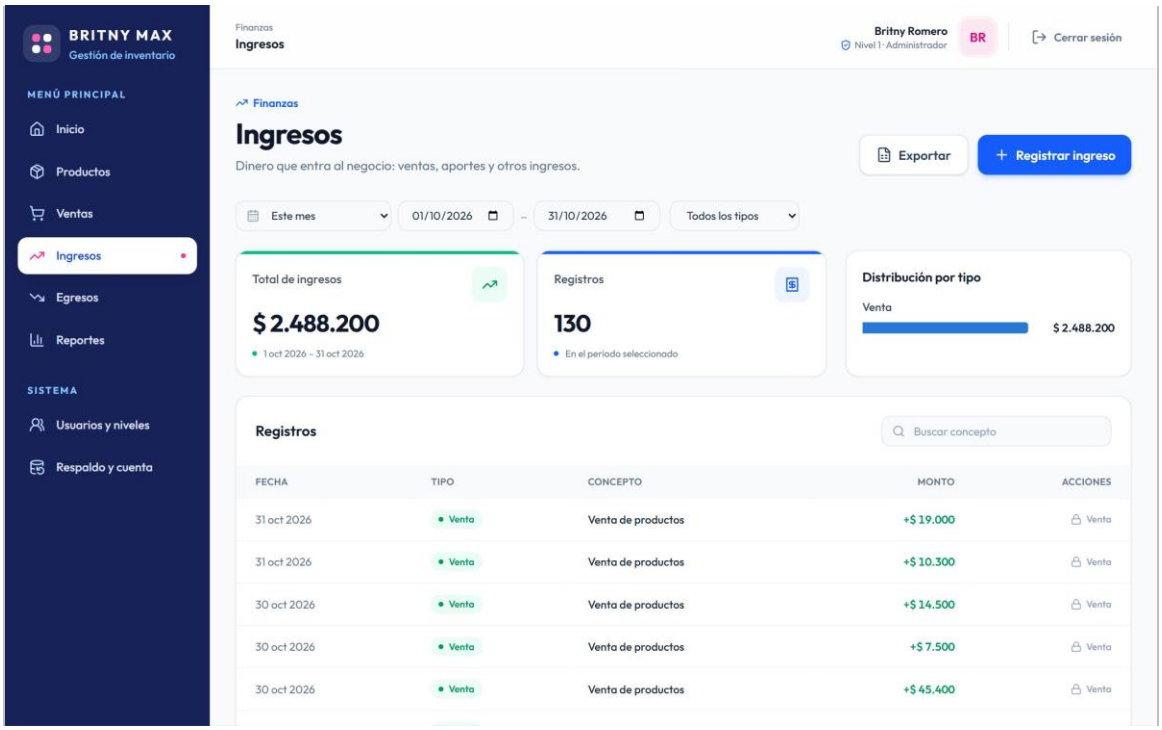
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Ingresos y Egresos

Los módulos de ingresos y egresos (RF11) presentan por separado los movimientos económicos del período, con totales, distribución por tipo y opciones para registrar, editar, filtrar y exportar (Figuras 24 y 25). Los ingresos por ventas se generan automáticamente y no se editan a mano, para que coincidan siempre con las ventas registradas.

Figura 24

Módulo de Ingresos

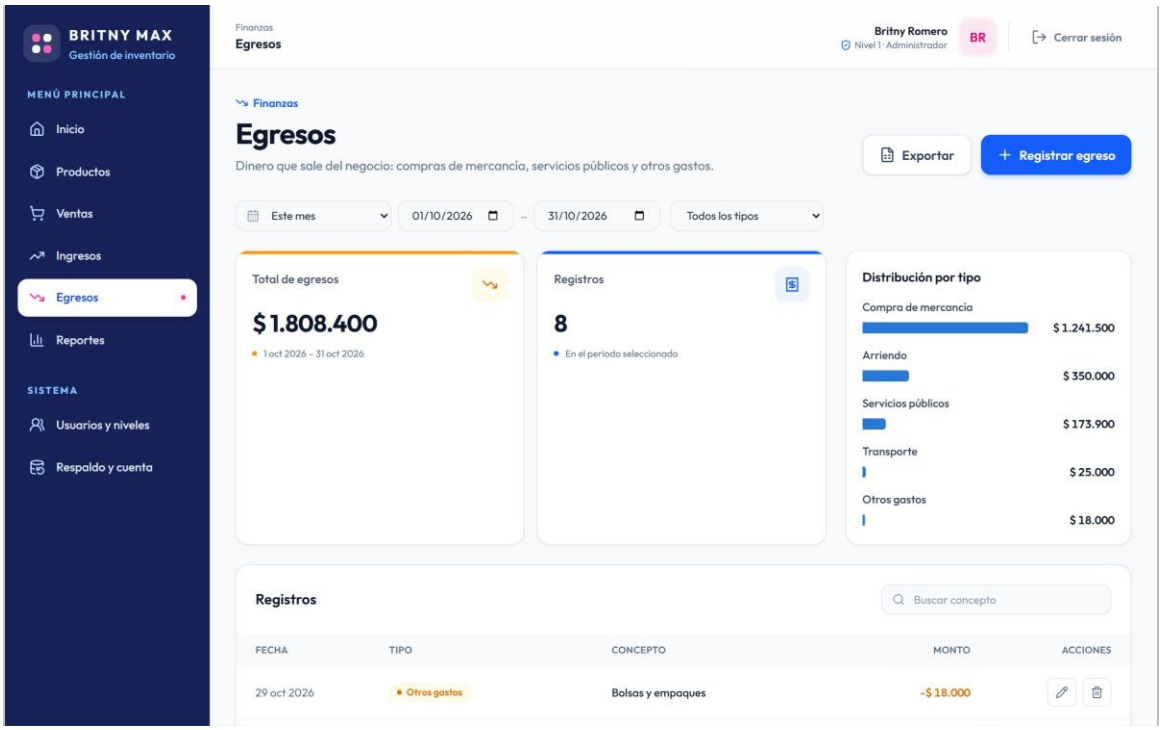


Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo).

Elaboración propia.

Figura 25

Módulo de Egresos



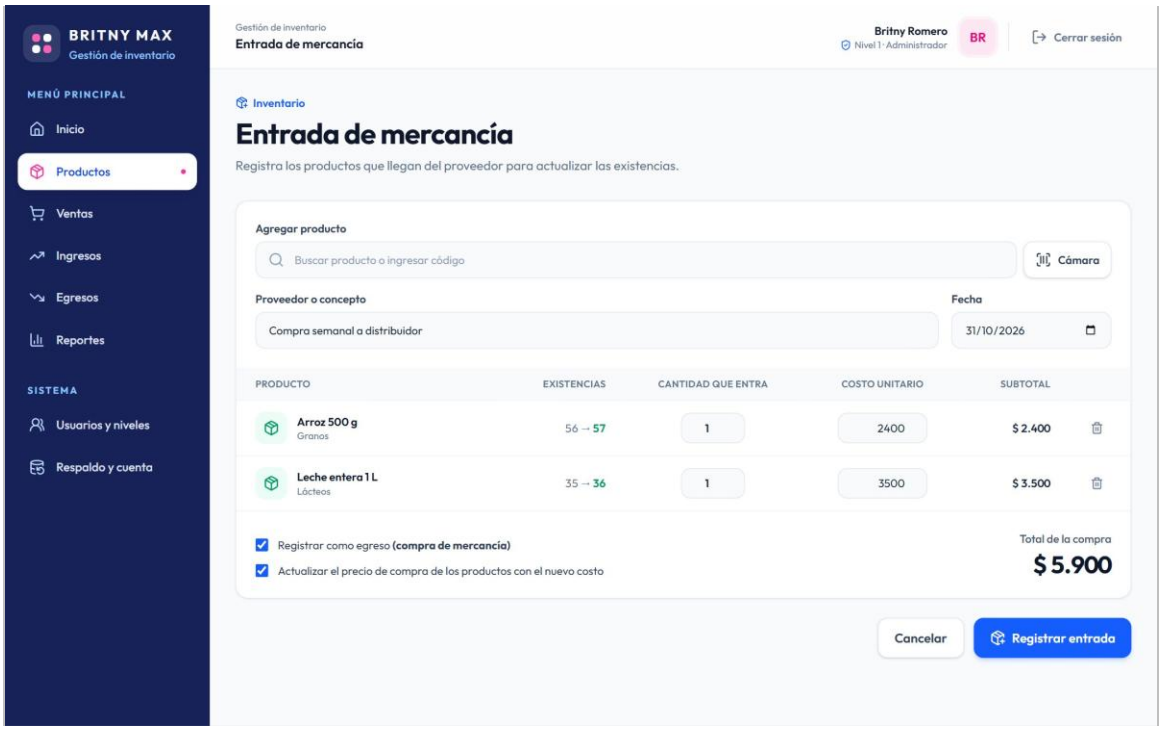
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Entradas de Mercancía y Conteo Físico

Las entradas de mercancía (Figura 26) suman las unidades recibidas del proveedor, pueden actualizar el costo de compra y registrar el egreso correspondiente. El conteo físico (Figura 27) responde al objetivo de detectar discrepancias: el usuario registra las unidades contadas en la estantería y el sistema calcula la diferencia frente a lo registrado y su valor; al guardar, solo se ajustan los productos con diferencia y queda el motivo del ajuste en los movimientos de inventario.

Figura 26

Registro de Entrada de Mercancía

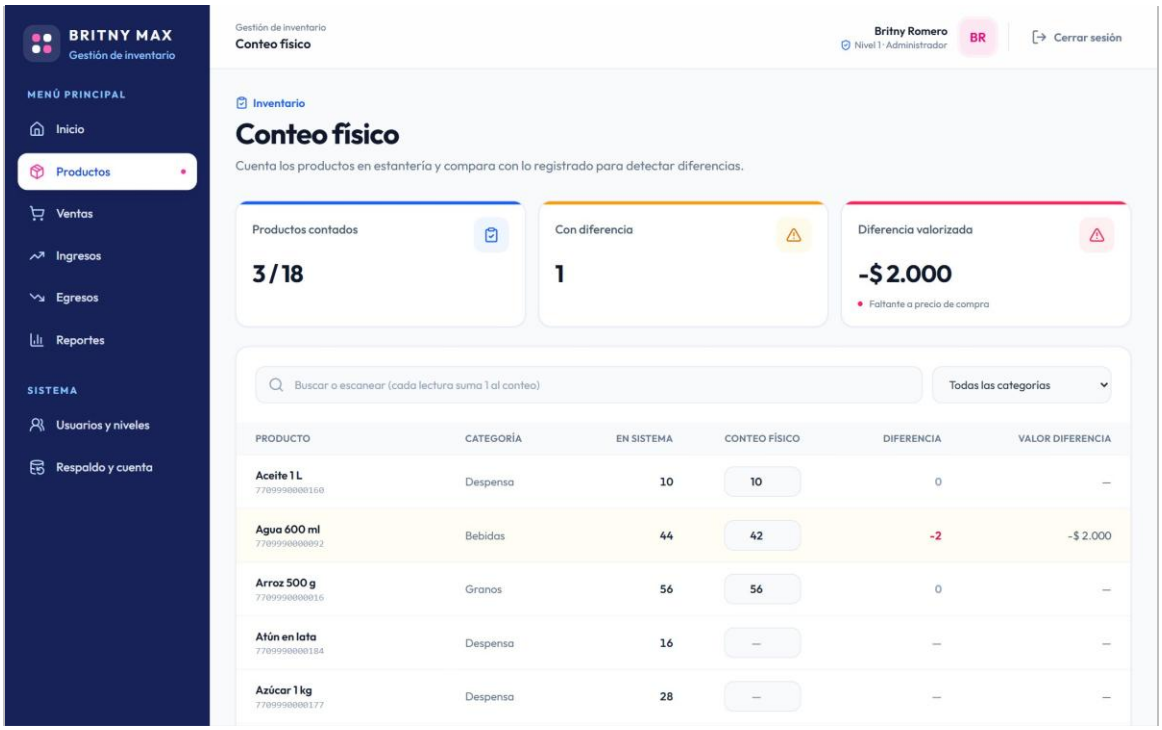


Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo).

Elaboración propia.

Figura 27

Conteo Físico con Detección de Diferencias



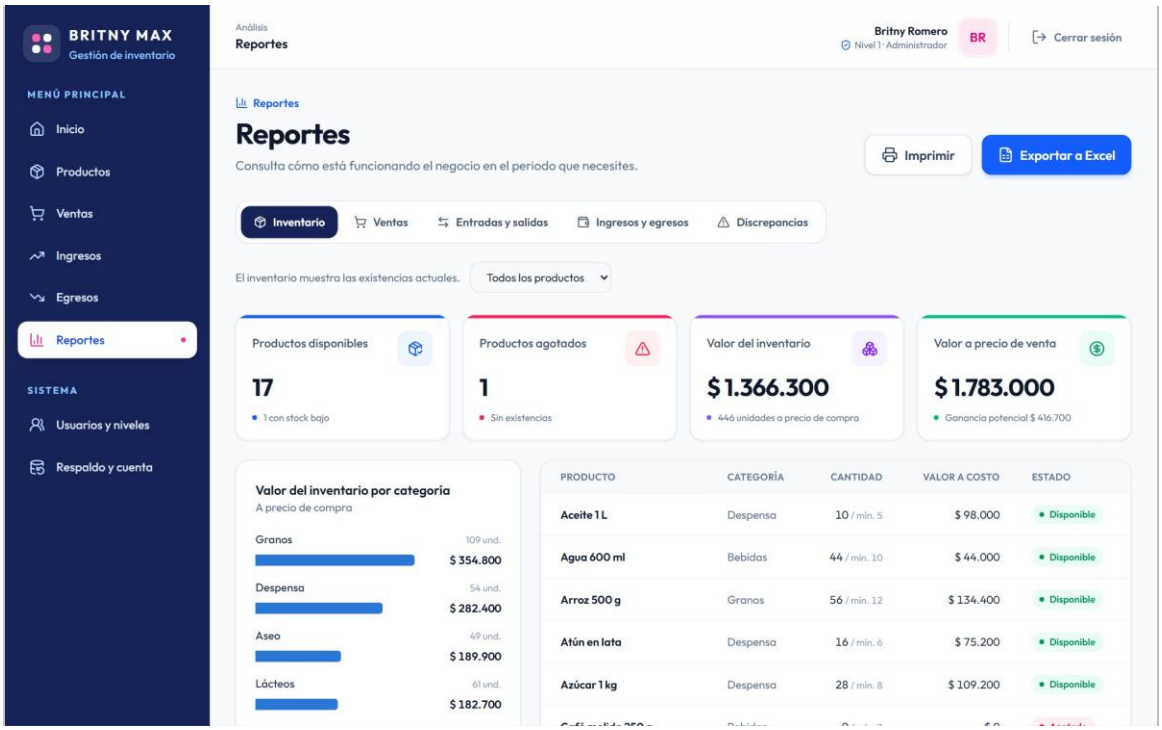
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Reportes

El módulo de reportes (RF15) agrupa la información en inventario, ventas, entradas y salidas, ingresos y egresos, y discrepancias, con filtros por período y opciones para imprimir o exportar a Excel (Figura 28).

Figura 28

Módulo de Reportes



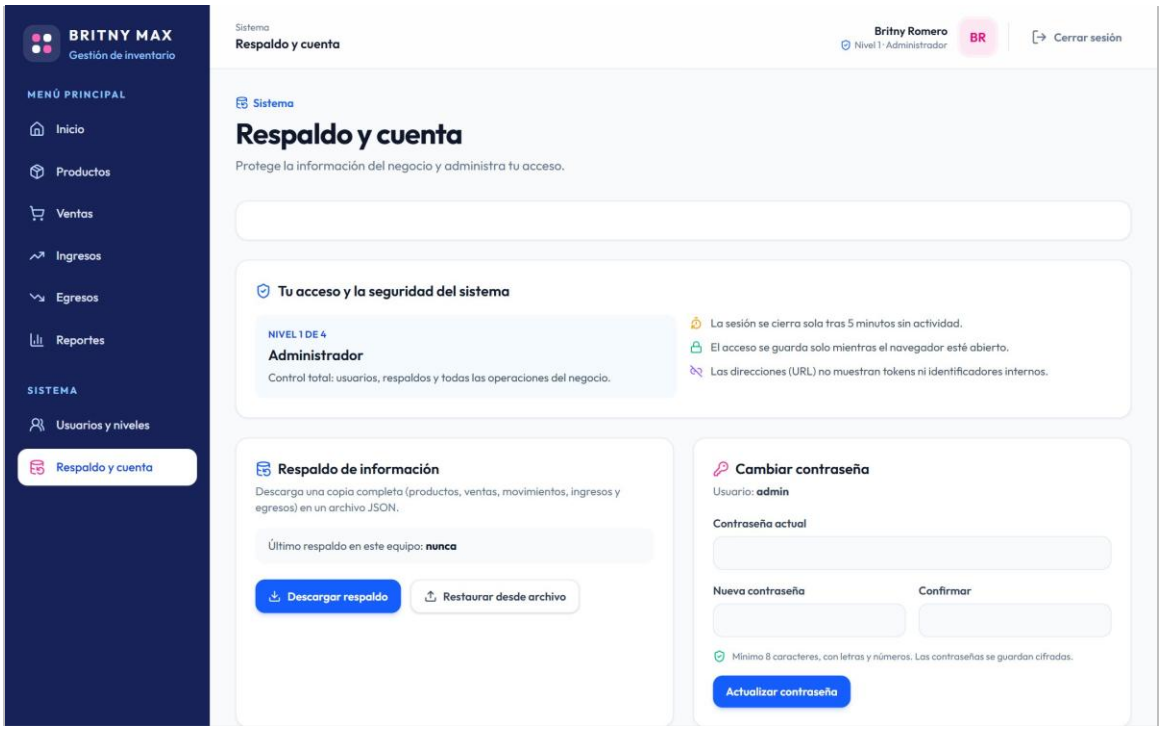
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Respaldo de Información

En cumplimiento del RNF09, los niveles administrador y supervisor pueden descargar una copia completa de la información en un archivo, y solo el administrador puede restaurarla (Figura 29). En la versión conectada, los datos se conservan además en la base de datos gestionada por Supabase. El mismo módulo resume las medidas de seguridad de la cuenta y permite cambiar la contraseña.

Figura 29

Módulo de Respaldo y Cuenta



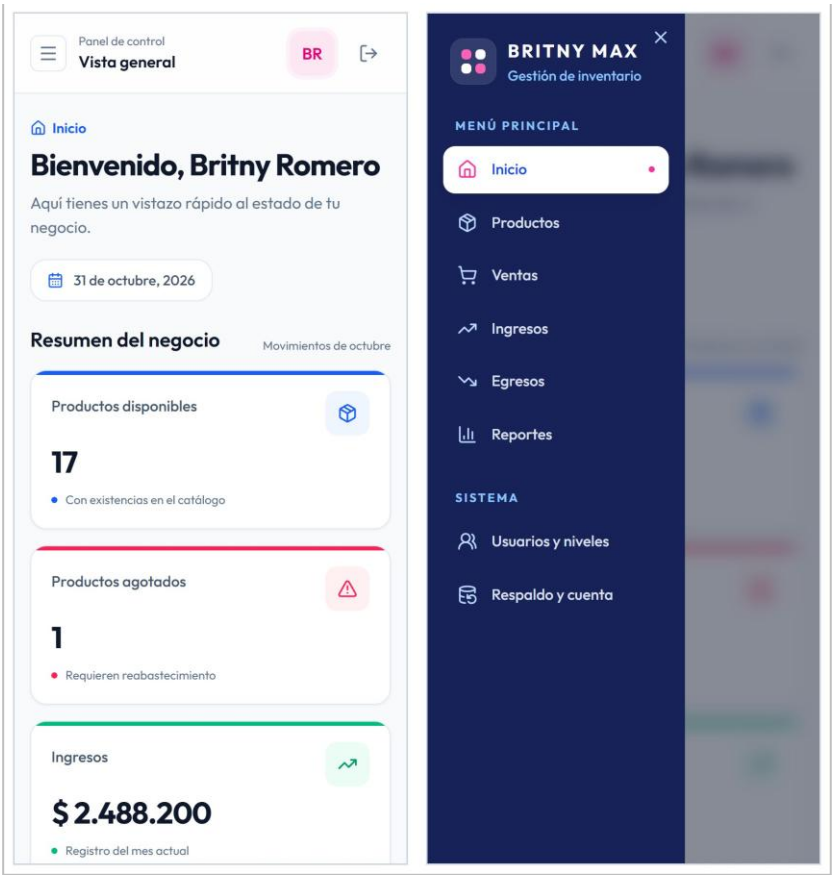
Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Diseño Adaptable

Para cumplir el RNF05, la interfaz se adapta al tamaño de la pantalla: en el celular, el menú se oculta en un panel lateral y las tablas se desplazan horizontalmente, lo que permite consultar el inventario o registrar ventas desde el teléfono (Figura 30).

Figura 30

Vista del Sistema en un Celular



Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo).

Elaboración propia.

Plan de Pruebas y Resultados

El plan de pruebas se elaboró con base en los conceptos de la norma ISO/IEC/IEEE 29119-1 (ISO, 2022) y en las características de calidad de la norma ISO/IEC 25010 (ISO, 2023). Su propósito es verificar que el sistema cumple los requerimientos funcionales y no funcionales especificados y aportar la evidencia necesaria para el tercer objetivo específico: evaluar el funcionamiento del sistema mediante pruebas que validen el control de existencias y la detección de discrepancias de inventario.

Objetivo y Alcance

El alcance comprende los módulos de autenticación y niveles de acceso, productos, lectura de códigos de barras, ventas, entradas de mercancía, conteo físico, ingresos y egresos, reportes y respaldo, así como los requerimientos no funcionales de tiempo de respuesta, seguridad, facilidad de uso, disponibilidad, compatibilidad, integridad, escalabilidad y respaldo. Quedan por fuera del alcance la facturación electrónica, los pagos electrónicos y la integración con proveedores, que no hacen parte de los requerimientos del proyecto.

Niveles y Tipos de Prueba

Las pruebas se organizaron en cinco niveles, de lo más particular a lo más general (Tabla 22). Las pruebas automatizadas se ejecutan con un solo comando cada vez que se modifica el sistema, lo que permite repetirlas sin esfuerzo adicional antes de cada entrega. Al repetirlas el 28 de octubre, una prueba de base de datos falló después de las 7 p. m. porque calculaba la fecha de

«hoy» en hora UTC y no en la de Bogotá; la aplicación y la base de datos ya usaban la hora de Colombia, así que se corrigió la prueba y las 56 volvieron a aprobarse.

Tabla 22

Niveles y Tipos de Prueba del Plan

Nivel	Qué se verifica	Técnica y herramienta	Casos
Unitarias	Validaciones de datos, matriz de permisos, inactividad, limpieza de URL, configuración y operaciones del modo demostración	Automatizada, node:test (npm test)	37
Integración con la base de datos	Esquema, restricciones, políticas RLS por nivel, funciones transaccionales y disparadores	Automatizada, PGlite con roles de Supabase simulados (npm run test:db)	24
Sistema (funcionales)	Recorrido de las pantallas con cada nivel de acceso, mensajes de validación, aviso y cierre por inactividad	Automatizada en Chrome y Edge sin interfaz	22 pasos
No funcionales	Tiempo de respuesta, seguridad sobre el proyecto real de Supabase, compatibilidad y diseño adaptable	Medición con el navegador y consultas a la API	4 tipos
Aceptación	Operación real en la tienda con la propietaria: registro diario y conteo físico frente al sistema	Manual, formato de registro (Tabla 28)	2 casos

Nota. Elaboración propia.

Entorno y Datos de Prueba

Tabla 23

Entorno de Ejecución de las Pruebas

Elemento	Configuración
Equipo	Computador con procesador AMD Ryzen 7 5700X y 16 GB de memoria RAM
Sistema operativo	Microsoft Windows 11 Pro
Navegadores	Google Chrome 156.0.8078.48 y Microsoft Edge 156.0.4314.50 (escritorio y vista de celular de 390 × 844 px)
Entorno de ejecución	Node.js 24.18.0; versión de producción compilada con Vite 8.3.5
Base de datos en la nube	Proyecto de Supabase en la región us-east-1 con el esquema, las políticas y los datos de ejemplo cargados
Sitio publicado	https://britny-max.vercel.app (Vercel), conectado al mismo proyecto de Supabase
Datos de prueba	18 productos en 7 categorías, 130 ventas con sus ingresos, 8 egresos y una cuenta por cada nivel de acceso
Fecha de ejecución	23 al 31 de octubre de 2026

Nota. Los datos de prueba son de ejemplo y deben eliminarse antes de iniciar la operación real de la tienda. Elaboración propia.

Criterios de Aceptación

Se definieron los siguientes criterios para dar por superada la fase de pruebas: el 100 % de las pruebas automatizadas debe aprobarse; ningún caso de prueba de prioridad alta puede quedar con defectos abiertos; las operaciones principales deben responder en menos de 2 segundos, según el RNF01; ningún usuario debe poder ejecutar una operación que su nivel no permite, ni desde la interfaz ni desde la API; y, en la prueba de aceptación, la diferencia entre el conteo físico y el inventario del sistema debe quedar explicada por los ajustes registrados. Las pruebas se suspenden

si el entorno no está disponible o si un defecto impide continuar, y se reanudan después de corregirlo, repitiendo las pruebas automatizadas completas.

Casos de Prueba y Resultados

La Tabla 24 relaciona cada caso de prueba con el requerimiento que verifica, el resultado esperado y el resultado obtenido. Las evidencias corresponden a las pruebas automatizadas (Anexo A) y a las capturas presentadas en el capítulo de desarrollo.

Tabla 24

Casos de Prueba y Resultados Obtenidos

ID	Req.	Caso de prueba	Resultado esperado	Resultado obtenido	Estado
CP01	RF01	Iniciar sesión con usuario y contraseña válidos en cada nivel	Accede al panel con el menú de su nivel	Los cuatro niveles accedieron con su menú (Figuras 18 y 15)	Aprobado
CP02	RF01 RNF02	Iniciar sesión con campos vacíos o contraseña incorrecta	No accede y muestra el error junto al campo	Se mostraron los mensajes («Usuario o contraseña incorrectos») y no se abrió la sesión	Aprobado
CP03	RF02	Registrar producto con nombre de un carácter, sin categoría y sin precios	No guarda y señala cada campo inválido	Cada campo inválido se marcó con su mensaje y no se guardó (Figura 20)	Aprobado
CP04	RF02	Registrar producto válido con cantidad inicial	Crea el producto y su movimiento de inventario inicial	Producto y movimiento creados (prueba automatizada)	Aprobado
CP05	RF03	Buscar productos por nombre, categoría y código	Filtra el catálogo en menos de 2 s	Filtró en 8 ms (mediana)	Aprobado
CP06	RF04	Editar producto con código de barras repetido y cambiar existencias directamente	Rechaza el código duplicado y el cambio directo de existencias	Ambas operaciones fueron rechazadas por la base de datos	Aprobado
CP07	RF05	Leer un código de barras existente con el lector (teclado)	Muestra el producto y sus existencias	Mostró el producto leído (Figura 21)	Aprobado
CP08	RF07	Registrar una venta de varios productos	Descuenta existencias, guarda el detalle y genera el ingreso	Existencias, detalle, movimiento e ingreso correctos	Aprobado

ID	Req.	Caso de prueba	Resultado esperado	Resultado obtenido	Estado
CP09	RF07 RNF07	Registrar una venta con cantidad mayor a las existencias	Rechaza la venta sin modificar nada	Venta rechazada; inventario sin cambios	Aprobado
CP10	RF07	Anular una venta registrada	Devuelve las unidades y retira el ingreso	Unidades devueltas e ingreso retirado	Aprobado
CP11	RF11	Registrar ingresos y egresos con fecha futura, monto negativo o concepto vacío	Rechaza los datos inválidos y acepta los válidos	Validaciones aplicadas en la interfaz y en la base de datos	Aprobado
CP12	RF11	Registrar una entrada de mercancía con egreso	Suma existencias, actualiza el costo y genera el egreso	Existencias, costo y egreso correctos (Figura 26)	Aprobado
CP13	RF15	Consultar los reportes por período y exportarlos	Muestra los totales del período seleccionado	Reportes generados y exportables (Figura 28)	Aprobado
CP14	RF15 Obj. 3	Registrar un conteo físico con un producto faltante	Calcula la diferencia y ajusta solo ese producto	Diferencia de 2 unidades valorizada; solo se ajustó ese producto (Figura 27)	Aprobado
CP15	RNF02	Abrir funciones no permitidas con los niveles cajero y consulta	Bloquea el acceso en la interfaz y en la base de datos	Acceso bloqueado en ambas capas (Figura 15)	Aprobado
CP16	RNF02	Dejar la sesión sin actividad durante 5 minutos	Avisa 30 s antes y cierra la sesión	Aviso con cuenta regresiva y cierre automático (Figura 16)	Aprobado
CP17	RNF02	Consultar la API de Supabase sin sesión y abrir URL con tokens	Rechaza la consulta y elimina los tokens de la URL	Respuesta HTTP 401 sin datos; tokens eliminados	Aprobado
CP18	RNF01	Medir el tiempo de respuesta de las operaciones principales	Menos de 2 s	Máximo de 245 ms (Tabla 26)	Aprobado
CP19	RNF05	Usar el sistema en Chrome, Edge, Firefox y en pantalla de celular	Funciona igual en cada navegador y tamaño	Recorrido completo sin errores en Chrome y Edge (Figura 30)	Aprobado
CP20	RNF09	Descargar un respaldo y restaurarlo	Restaura la información en orden y sin duplicados	Restauración correcta y repetible (prueba automatizada)	Aprobado
CP21	RNF04	Consultar repetidamente la API de Supabase y el servicio local	Ambos responden todas las solicitudes	20 de 20 consultas respondidas; mediana de 116 ms	Aprobado
CP22	RNF02 RNF04	Publicar en Vercel sin las variables de Supabase y luego con ellas	Sin variables la compilación se detiene; con ellas el sitio queda conectado a Supabase	Compilación detenida con el aviso de variables faltantes; luego, sitio publicado (HTTP 200) con la clave pública del proyecto	Aprobado

ID	Req.	Caso de prueba	Resultado esperado	Resultado obtenido	Estado
CP23	RNF02	Quitar y dar permisos a los niveles 2 a 4 desde «Usuarios y niveles»	Cada nivel ve y puede hacer solo lo que tiene asignado, también estando conectado	Supervisor sin «editar productos» y consulta sin «finanzas» quedaron bloqueados; un permiso dado apareció al cambiar de pantalla (Figura 33)	Aprobado
CP24	RF01 RNF02	Ingresar al sitio publicado con las cuatro cuentas reales	Cada cuenta entra con el menú de su nivel	Verificado y aceptado por el usuario el 31 de octubre de 2026	Aprobado
CP25	RNF03	Ejecutar las tareas principales con la propietaria tras la capacitación	Completa las tareas sin ayuda	La propietaria completó las tareas sin ayuda (reportado por el equipo)	Aprobado
CP26	Obj. 3	Comparar el conteo físico de la tienda con el sistema durante dos semanas	Las diferencias quedan detectadas y explicadas	Conteos conformes en las dos semanas (reportado por el equipo)	Aprobado

Nota. Req. = requerimiento verificado; Obj. 3 = tercer objetivo específico. Elaboración propia.

Resultados de las Pruebas Automatizadas

Las 61 pruebas automatizadas se aprobaron en su totalidad (Tabla 25). Las pruebas de base de datos se ejecutan sobre una instancia de PostgreSQL embebida que simula los roles de Supabase y el usuario autenticado, por lo que verifican las mismas políticas y funciones instaladas en la nube sin modificar los datos reales. Las Figuras 31 y 32 muestran la salida de la consola de ambas ejecuciones.

Tabla 25

Resultados de las Pruebas Automatizadas

Conjunto de pruebas	Pruebas	Aprobadas	Fallidas	Duración
Unitarias (npm test)	37	37	0	0,55 s
Base de datos (npm run test:db)	24	24	0	15,10 s
Total	61	61	0	15,65 s

Nota. Ejecución del 31 de octubre de 2026. Elaboración propia.

Figura 31

Ejecución de las Pruebas Unitarias



Nota. Salida de la consola; se omiten las líneas de resumen con valor cero. Elaboración propia.

Figura 32

Ejecución de las Pruebas de Base de Datos



Nota. Salida de la consola; se omiten las líneas de resumen con valor cero. Elaboración propia.

Resultados de Rendimiento

El tiempo de respuesta se midió en la versión de producción, con el navegador Chrome sin interfaz, repitiendo cada operación nueve veces con la caché vacía, y la latencia de la API de

Supabase con veinte consultas desde el mismo equipo (Tabla 26). Todas las operaciones quedaron muy por debajo del límite de 2 segundos del RNF01; la operación más lenta fue la carga inicial de la aplicación, con un máximo de 245 ms. En la versión conectada, a cada operación que consulta o guarda información se suma la latencia de la API, cuya mediana fue de 116 ms.

Tabla 26

Tiempos de Respuesta Medidos

Operación	Mínimo	Mediana	Percentil 95	Límite RNF01
Carga inicial de la aplicación	110 ms	153 ms	245 ms	2.000 ms
Inicio de sesión hasta ver el panel	85 ms	92 ms	146 ms	2.000 ms
Apertura del módulo de productos	66 ms	74 ms	145 ms	2.000 ms
Búsqueda de un producto	7 ms	8 ms	17 ms	2.000 ms
Registro de una venta	22 ms	24 ms	30 ms	2.000 ms
Apertura de reportes	35 ms	43 ms	53 ms	2.000 ms
Consulta a la API de Supabase (nube)	109 ms	116 ms	172 ms	2.000 ms

Nota. Mediciones del 27 de octubre de 2026 en el entorno de la Tabla 23; las operaciones de la interfaz se midieron con los datos de ejemplo del modo de demostración. Elaboración propia.

Resultados de Seguridad y Compatibilidad

Sobre el proyecto real de Supabase se verificó, sin escribir datos, que el registro público de usuarios está desactivado y que un visitante sin sesión no puede leer productos ni registrar ventas: la API respondió con el código HTTP 401 y sin información. Además, el 23 de octubre de 2026 se comprobó cada nivel de acceso sobre la base de datos real dentro de una transacción que se revirtió al final, de modo que no quedaron cambios; los resultados coincidieron con la matriz de permisos. En compatibilidad, el recorrido completo de 22 pasos se ejecutó sin errores en Google

Chrome y en Microsoft Edge, en escritorio y en la vista de celular, y el equipo del proyecto lo verificó también en Mozilla Firefox sin encontrar diferencias.

Pruebas de Aceptación del Usuario

Durante la revisión del sistema publicado, el usuario solicitó que la administradora pudiera poner y quitar permisos a los demás niveles y que cada persona viera únicamente lo que se le asigna. El cambio se implementó en la interfaz y en la base de datos, y se validó con los escenarios de la Tabla 27, ejecutados el 31 de octubre de 2026 en el modo de demostración y verificados en el proyecto real de Supabase. La Figura 33 muestra la matriz durante un cambio, antes de guardarlo.

Tabla 27

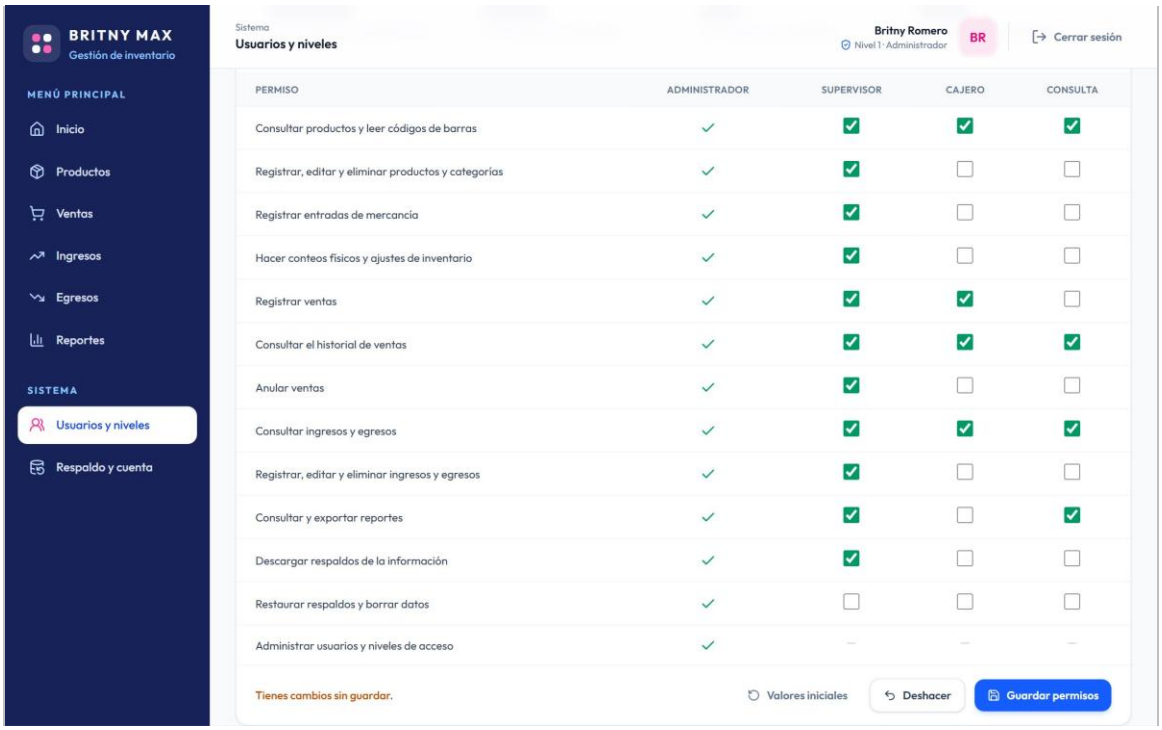
Escenarios de la Prueba de Aceptación del Usuario

N.º	Escenario	Resultado observado	Evidencia	Estado
1	La administradora quita a supervisor «registrar, editar y eliminar productos» y «anular ventas», y guarda	El supervisor ya no ve «Registrar producto» ni los botones de edición; al abrir /productos/nuevo recibe «Tu nivel de acceso (Supervisor) no permite abrir…»	Recorrido en el navegador	Aprobado
2	La administradora quita a consulta «consultar ingresos y egresos»	Ingresos y Egresos desaparecen del menú y la dirección /egresos queda bloqueada	Recorrido en el navegador	Aprobado
3	La administradora da un permiso mientras la otra persona sigue conectada	A la persona de consulta le aparecieron Ingresos y Egresos al cambiar de pantalla, sin cerrar su sesión	Dos pestañas simultáneas	Aprobado
4	Un nivel distinto del administrador intenta cambiar la matriz o asignarse «administrar usuarios»	La base de datos rechaza la operación y la matriz queda sin cambios	Prueba automatizada	Aprobado
5	Revisión final del sistema con las cuatro cuentas reales	El usuario aceptó el funcionamiento del sistema	Aceptación del 31 de octubre de 2026	Aprobado

Nota. Elaboración propia.

Figura 33

Cambio de Permisos Pendiente de Guardar



Nota. Captura del sistema en funcionamiento (modo de demostración con datos de ejemplo). Elaboración propia.

Prueba de Aceptación en la Tienda

La prueba de aceptación completó la evaluación en el entorno operativo real y se realizó con la propietaria durante dos semanas, después de eliminar los datos de ejemplo y cargar el inventario real. El procedimiento es el siguiente: primero, capacitar a la propietaria en una sesión y verificar que complete sin ayuda el registro de un producto, una venta, una entrada y un egreso (CP25); segundo, registrar en el sistema todas las ventas y compras del período; tercero, hacer

cada semana un conteo físico de los productos y compararlo con el sistema mediante el módulo de conteo (CP26); y cuarto, consignar los resultados en la Tabla 28 y analizar las diferencias encontradas. Según el reporte del equipo del proyecto, la propietaria completó las tareas sin ayuda y los conteos de ambas semanas fueron conformes, con las diferencias detectadas por el sistema y explicadas mediante sus ajustes.

Tabla 28

Resultados de la Prueba de Aceptación en la Tienda

Semana	Productos contados	Productos con diferencia	Diferencia valorizada	Observaciones
Semana 1	Inventario completo	Detectadas por el sistema	Explicada por ajustes registrados	Conforme
Semana 2	Inventario completo	Detectadas por el sistema	Explicada por ajustes registrados	Conforme

Nota. Resultados reportados por el equipo del proyecto el 31 de octubre de 2026. Elaboración propia.

Análisis de Resultados

Los 26 casos de prueba definidos se ejecutaron y se aprobaron, incluidos el cambio de permisos solicitado por el usuario, su aceptación del sistema y la prueba de aceptación en la tienda con la propietaria. Las pruebas confirman que el sistema mantiene la integridad del inventario —las existencias solo cambian mediante operaciones que registran su movimiento—, que los permisos de cada nivel, incluso cuando la administradora los cambia, se respetan tanto en la interfaz como en la base de datos y que el tiempo de respuesta es holgado frente al requerimiento.

El módulo de conteo físico detectó y valorizó correctamente la diferencia introducida en la prueba, lo que demuestra la capacidad del sistema para identificar discrepancias entre el inventario físico y el registrado.

Conclusiones

El análisis del proceso manual de la tienda Britny Max permitió establecer ocho requerimientos funcionales y ocho no funcionales que orientaron el diseño y la construcción del sistema, con lo cual se cumplió el primer objetivo específico. El levantamiento confirmó que el principal problema no era la falta de información, sino su dispersión en un cuaderno sin respaldo, sin actualización inmediata de las existencias y sin forma ágil de consultar el estado del negocio.

Se implementaron los módulos de registro de productos, lectura de códigos de barras, ventas, entradas de mercancía, conteo físico, ingresos y egresos, reportes y respaldo, con lo cual se cumplió el segundo objetivo específico. La decisión de repetir las reglas de negocio en la base de datos, mediante funciones transaccionales y políticas por nivel, hace que la información se mantenga consistente aunque falle la conexión o se intente operar por fuera de la interfaz.

El plan de pruebas verificó técnicamente el sistema: las 61 pruebas automatizadas se aprobaron, los 26 casos de prueba se ejecutaron con éxito y las operaciones principales respondieron en menos de 0,3 segundos, muy por debajo del límite de 2 segundos. Con la prueba de aceptación de dos semanas en la tienda, en la que la propietaria operó el sistema y los conteos físicos fueron conformes, se cumplió el tercer objetivo específico.

Los cuatro niveles de acceso con permisos que la administradora ajusta sin modificar el sistema, el cierre de sesión por inactividad y la protección de las direcciones web elevan la seguridad de la información frente al cuaderno físico y permiten delegar tareas como la venta en el mostrador sin exponer las finanzas ni la configuración del negocio.

Finalmente, el proyecto muestra que una tienda de barrio puede contar con una herramienta tecnológica sencilla, de bajo costo y adaptada a su operación, lo que coincide con lo planteado por Pardo Morales et al. (2026) y Rueda Vera et al. (2022) sobre el aporte de los sistemas de información al control de inventarios en pequeños negocios.

Recomendaciones

Antes de iniciar la operación real, se recomienda eliminar los datos de ejemplo con el archivo supabase/vaciar-datos.sql, cargar el inventario real a partir de un conteo físico completo y cambiar las contraseñas iniciales de las cuatro cuentas desde el módulo «Respaldo y cuenta».

Se recomienda mantener el conteo físico al menos una vez por semana durante los primeros meses y descargar un respaldo de la información cada semana.

Como trabajo futuro, se sugiere evaluar la integración con la facturación electrónica de la DIAN, el registro de proveedores y la generación de pedidos sugeridos a partir del stock mínimo y de la rotación de los productos.

Referencias

Ccallo Rodríguez, C., Villena-Reyes, D. y Salvador-Reyes, R. (2024). Lean Warehousing para la optimización de la gestión de inventarios en PYMES: una revisión sistemática. En *Proceedings of the 4th LACCEI International Multiconference on Entrepreneurship, Innovation and Regional Development (LEIRD 2024)*. LACCEI. https://doi.org/10.18687/LEIRD2024.1.1.797

Corella Parra, L. M. y Olea Miranda, J. (2023). Desarrollo de un sistema de control de inventario para una empresa comercializadora de sistemas de riego. *Ingeniería Investigación y Tecnología*, *24*(1), 1–10. https://doi.org/10.22201/fi.25940732e.2023.24.1.006

Decreto 1377 de 2013. Por el cual se reglamenta parcialmente la Ley 1581 de 2012. 27 de junio de 2013. D.O. No. 48834.

Departamento Administrativo Nacional de Estadística [DANE]. (2025). *Boletín técnico: Encuesta de Micronegocios (EMICRON) 2024*. https://www.dane.gov.co/files/operaciones/EMICRON/bol-EMICRON-2024.pdf

Departamento Administrativo Nacional de Estadística [DANE]. (2026). *Boletín técnico: Encuesta de Micronegocios (EMICRON). Panaderías y tiendas de barrio 2020-2024*. https://www.dane.gov.co/files/operaciones/EMICRON/bol-EMICRON-PanaderiasTiendasBarrio-2024.pdf

Flores Tapia, C. E. y Flores Cevallos, K. L. (2023). Optimización de inventarios aplicando investigación de operaciones. *RECAI Revista de Estudios en Contaduría, Administración e Informática, 12*(34), 1–15. https://doi.org/10.36677/recai.v12i34.19628

Galaviz-Magallanes, L. E., Rodríguez-Morachis, M. A. y Gutiérrez-Hernández, M. G. (2024). Implementación de un método para el control de inventario en una microempresa familiar. *Cultura Científica y Tecnológica, 21*(3), 80–89. https://doi.org/10.20983/culcyt.2024.3.2.8

Guerra-Naranjo, L. M. (2024). Evaluación de la influencia de la gestión de inventarios en el rendimiento empresarial: un estudio de caso. *Reflexiones Contables, 7*(2), 15–28. https://doi.org/10.22463/26655543.4547

International Organization for Standardization. (2022). *Software and systems engineering — Software testing — Part 1: General concepts* (ISO/IEC/IEEE Standard No. 29119-1:2022).

International Organization for Standardization. (2023). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model* (ISO/IEC Standard No. 25010:2023).

International Telecommunication Union [ITU]. (2024). *Measuring digital development: Facts and figures 2024*. https://www.itu.int/itu-d/reports/statistics/facts-figures-2024/

Ley 1581 de 2012. Por la cual se dictan disposiciones generales para la protección de datos personales. 17 de octubre de 2012. D.O. No. 48587.

Ley 527 de 1999. Por medio de la cual se define y reglamenta el acceso y uso de los mensajes de datos, del comercio electrónico y de las firmas digitales, y se establecen las entidades de certificación y se dictan otras disposiciones. 18 de agosto de 1999. D.O. No. 43673.

Maheshwari, P., Kamble, S., Pundir, A., Belhadi, A., Ndubisi, N. O. y Tiwari, S. (2025). Internet of things for perishable inventory management systems: An application and managerial insights for micro, small and medium enterprises. *Annals of Operations Research, 350*(2), 395–423. https://doi.org/10.1007/s10479-021-04277-9

Nirmala, D. A. R., Kannan, V., Thanalakshmi, M., Gnanaraj, S. J. P. y Appadurai, M. (2022). Inventory management and control system using ABC and VED analysis. *Materials Today: Proceedings, 60*, 922–925. https://doi.org/10.1016/j.matpr.2021.10.315

OWASP Foundation. (2025). *OWASP Application Security Verification Standard* (Versión 5.0.0). https://owasp.org/www-project-application-security-verification-standard/

Palma Cardoso, E., Acebedo Molina, D. G., Morales Lugo, R. E. y Guzmán, R. A. (2023). Gestión de inventarios en microempresas del sector farmacéutico, Colombia. *Revista Venezolana de Gerencia, 28*(Especial 9), 464–481. https://doi.org/10.52080/rvgluz.28.e9.29

Pardo Morales, J. R., Alvarado Caicedo, J. D. y Mendivelso Fajardo, C. A. (2026). Diseño participativo de un prototipo web para la gestión de inventarios en tiendas de barrio: experiencia de apropiación tecnológica con tenderos del sector Balmoral de Fusagasugá.

Revista Electrónica de Investigación de Tecnologías Educativas ITFIPVIRTUAL, 8(1), 29–42. https://doi.org/10.54198/ywb0pv18

Pinia. (2025). *Pinia* (Versión 3.0.4) [Software]. https://pinia.vuejs.org/

PostgreSQL Global Development Group. (2025). *PostgreSQL 18 documentation: Row security policies*. https://www.postgresql.org/docs/18/ddl-rowsecurity.html

Rueda Vera, G., Avendaño Castro, W. R. y Parada Trujillo, A. E. (2022). Sistemas de información y control de inventarios en micro, pequeñas y medianas empresas - Mipymes de la ciudad de Cúcuta, Colombia. *Saber, Ciencia y Libertad, 17*(2), 328–351. https://doi.org/10.18041/2382-3240/saber.2022v17n2.9295

Sarmiento Suárez, J. E., Gutiérrez Navas, E. B. y Ramírez Montañez, J. C. (2024). Oportunidades y desafíos para la digitalización de las mipymes en Colombia. *Pensamiento & Gestión*, (57), 128–154. https://doi.org/10.14482/pege.57.240.855

Sbai, N., Benabbou, L. y Berrado, A. (2022). Multi-echelon inventory system selection: Case of distribution systems. *International Journal of Supply and Operations Management, 9*(1), 108–125.

Supabase. (2026). *supabase-js* (Versión 2.118.0) [Software]. GitHub. https://github.com/supabase/supabase-js

Tailwind Labs. (2026). *Tailwind CSS* (Versión 4.3.3) [Software]. https://tailwindcss.com/

Vite. (2026). *Vite* (Versión 8.3.5) [Software]. https://vite.dev/

Vue.js. (2026). *Vue.js* (Versión 3.5.44) [Software]. https://vuejs.org/

ZXing-js. (2024). *@zxing/browser* (Versión 0.1.5) [Software]. GitHub. https://github.com/zxing-js/browser

Anexo A. Listado de Pruebas Automatizadas

La Tabla 29 relaciona las 61 pruebas automatizadas del proyecto, todas aprobadas en la ejecución del 31 de octubre de 2026. Se ejecutan con los comandos npm test y npm run test:db desde la carpeta del proyecto.

Tabla 29

Pruebas Automatizadas del Sistema

N.º	Conjunto	Prueba	Resultado
1	Unitarias	Sin variables: modo demo solo si se permite, si no es un error	Aprobada
2	Unitarias	Configuración válida con clave anon o publishable	Aprobada
3	Unitarias	Detecta variables incompletas o incorrectas	Aprobada
4	Unitarias	Rechaza la clave secreta service_role	Aprobada
5	Unitarias	Crear producto registra el inventario inicial	Aprobada
6	Unitarias	venta: descuenta stock, guarda detalle, movimiento e ingreso	Aprobada
7	Unitarias	Venta con stock insuficiente no cambia nada (todo o nada)	Aprobada
8	Unitarias	Anular venta devuelve existencias y retira el ingreso	Aprobada
9	Unitarias	Entrada de mercancía con egreso y concepto corto	Aprobada
10	Unitarias	Conteo físico ajusta solo los productos con diferencia	Aprobada
11	Unitarias	Los datos de ejemplo son consistentes	Aprobada
12	Unitarias	Hay exactamente 4 niveles, numerados del 1 al 4	Aprobada
13	Unitarias	Matriz de permisos	Aprobada
14	Unitarias	Permisos desconocidos o sin nivel se niegan	Aprobada
15	Unitarias	Permisos editables: el administrador asigna y quita, y se respeta	Aprobada
16	Unitarias	Permisos editables: requisitos y validación de la lista	Aprobada
17	Unitarias	La matriz inicial del código es la misma que carga schema.sql	Aprobada
18	Unitarias	La sesión vence a los 5 minutos sin actividad	Aprobada
19	Unitarias	Quita tokens de la consulta y del fragmento de la URL	Aprobada
20	Unitarias	Solo redirige a rutas internas después del login	Aprobada
21	Unitarias	Validación de usuarios	Aprobada
22	Unitarias	Modo demo: crea una cuenta de ejemplo por nivel	Aprobada

N.º	Conjunto	Prueba	Resultado
23	Unitarias	Modo demo: el cajero vende pero no ve finanzas ni edita	Aprobada
24	Unitarias	Modo demo: consulta ve finanzas pero no modifica	Aprobada
25	Unitarias	Modo demo: el administrador cambia los permisos y el cajero los recibe	Aprobada
26	Unitarias	Modo demo: sin sesión no se opera	Aprobada
27	Unitarias	Modo demo: el administrador gestiona usuarios con protecciones	Aprobada
28	Unitarias	categoría: nombre requerido, longitud y duplicados sin importar mayúsculas	Aprobada
29	Unitarias	Producto válido no tiene errores	Aprobada
30	Unitarias	producto: precios vacíos, negativos, cero, enormes o con muchos decimales	Aprobada
31	Unitarias	producto: cantidades enteras, código de barras y categoría	Aprobada
32	Unitarias	venta: stock, cantidades, fecha y productos retirados	Aprobada
33	Unitarias	Entrada y ajuste de inventario	Aprobada
34	Unitarias	Ingresos y egresos: fecha, tipo, concepto y monto	Aprobada
35	Unitarias	Contraseñas e inicio de sesión	Aprobada
36	Unitarias	respaldos: estructura mínima	Aprobada
37	Unitarias	exigirValido lanza ErrorValidacion con el mapa de errores	Aprobada
38	Base de datos	Los visitantes anónimos no pueden leer ni operar	Aprobada
39	Base de datos	Un usuario autenticado sin autorización no ve ni modifica datos	Aprobada
40	Base de datos	Cada nivel ve solo su perfil; el administrador ve todos	Aprobada
41	Base de datos	categorías: nombre obligatorio, único (sin importar mayúsculas) y normalizado	Aprobada
42	Base de datos	No se elimina una categoría con productos activos	Aprobada
43	Base de datos	crear_producto registra el inventario inicial	Aprobada
44	Base de datos	productos: validaciones de nombre, código, precios y existencias	Aprobada
45	Base de datos	El código de barras es único entre productos activos	Aprobada
46	Base de datos	Las existencias no se pueden modificar directamente	Aprobada
47	Base de datos	registrar_venta descuenta existencias, guarda el detalle y genera el ingreso	Aprobada
48	Base de datos	registrar_venta valida stock, cantidades, fecha y productos	Aprobada
49	Base de datos	ventas, detalles y movimientos no se escriben por fuera de las operaciones	Aprobada
50	Base de datos	La marca de operación no sobrevive fuera de la función	Aprobada
51	Base de datos	anular_venta devuelve existencias y retira el ingreso	Aprobada
52	Base de datos	registrar_entrada suma existencias, actualiza costo y genera el egreso	Aprobada
53	Base de datos	ajustar_inventario registra las diferencias del conteo físico	Aprobada
54	Base de datos	Ingresos y egresos: fecha, concepto, monto y tipo	Aprobada

N.º	Conjunto	Prueba	Resultado
55	Base de datos	importar_respaldo restaura en orden y puede repetirse	Aprobada
56	Base de datos	Nivel 3 (cajero): vende, pero no edita inventario ni ve finanzas	Aprobada
57	Base de datos	Nivel 4 (consulta): ve todo, no modifica nada	Aprobada
58	Base de datos	Nivel 2 (supervisor): opera el negocio, sin respaldos ni usuarios	Aprobada
59	Base de datos	Nivel 1 (administrador): gestiona usuarios con protecciones	Aprobada
60	Base de datos	Las funciones de nivel no se pueden llamar desde la API	Aprobada
61	Base de datos	El administrador pone y quita permisos, y la base de datos los respeta	Aprobada

Nota. Elaboración propia a partir de la salida de node:test.

Anexo B. Instalación y Puesta en Marcha

1. Instalar Node.js 24 o superior en el computador de la tienda y copiar la carpeta del proyecto britny-max.

2. Crear el archivo .env con la dirección del proyecto de Supabase y la clave publicable (nunca la clave secreta).

3. Ejecutar el archivo supabase/schema.sql en el editor SQL de Supabase y, antes de la operación real, supabase/vaciar-datos.sql para eliminar los datos de ejemplo.

4. Crear la cuenta de la propietaria en Supabase Auth, autorizarla como administradora y crear desde el módulo «Usuarios y niveles» las cuentas de los demás usuarios.

5. Para publicarlo en la web: subir el proyecto a un repositorio de GitHub, importarlo en Vercel con el preajuste Vite y registrar las variables VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY y VITE_AUTH_DOMAIN antes de desplegar. Si falta alguna, la compilación se detiene con un mensaje que indica cuál.

6. Para usarlo también sin publicar: hacer doble clic en iniciar-britny-max.bat; el servicio queda en segundo plano en http://localhost:4173 y se detiene con detener-britny-max.bat.

7. Verificar la instalación con npm run verificar:supabase y ejecutar npm test y npm run test:db después de cualquier modificación.

Anexo C. Script de la Base de Datos

Este es el contenido completo del archivo supabase/schema.sql, que crea en PostgreSQL (Supabase) las tablas, las restricciones, la seguridad a nivel de fila, las funciones transaccionales, la matriz inicial de permisos y la gestión de usuarios. Se ejecuta en Supabase → SQL Editor y puede volver a ejecutarse sin borrar la información ni los permisos configurados. El modelo resultante se presenta en la Figura 12.

```
-- ========================================================================
--  Britny Max · Sistema de gestión de inventarios, ingresos y egresos
--  Esquema para Supabase (PostgreSQL 15+).
--
--  Ejecutar completo en: Supabase > SQL Editor > New query > Run.
--  Es seguro volver a ejecutarlo: actualiza funciones, políticas y
--  disparadores sin borrar datos.
--
--  Los límites coinciden con src/lib/validaciones.js (RNF07).
-- ========================================================================

-- Esquema privado (no expuesto por la API) para funciones auxiliares.
create schema if not exists privado;

-- ------------------------------------------------------------------------
--  1. Usuarios autorizados (clase Usuario del diagrama)
--     La cuenta se crea en Supabase Auth; aquí se autoriza su acceso.
-- ------------------------------------------------------------------------

create table if not exists public.usuarios (
  id         uuid primary key references auth.users (id) on delete cascade,
  nombre     text not null constraint usuarios_nombre_longitud check (char_length(btrim(nombre)) between 2 and
80),
  rol        text not null default 'consulta' constraint usuarios_rol_valido
             check (rol in ('administrador', 'supervisor', 'cajero', 'consulta')),
  activo     boolean not null default true,
  creado_en  timestamptz not null default now()
);

-- Niveles de seguridad: 1 administrador · 2 supervisor · 3 cajero · 4 consulta
-- (misma matriz que src/lib/permisos.js). Actualiza bases de la versión anterior.
alter table public.usuarios drop constraint if exists usuarios_rol_valido;
update public.usuarios set rol = 'cajero' where rol not in ('administrador', 'supervisor', 'cajero', 'consulta');
alter table public.usuarios add constraint usuarios_rol_valido
  check (rol in ('administrador', 'supervisor', 'cajero', 'consulta'));

-- Permisos de los niveles 2 a 4: el administrador los pone y los quita desde
-- «Usuarios y niveles». El administrador los tiene todos siempre, y
-- «usuarios.gestionar» es solo suyo (por eso no está en la lista).
create table if not exists public.permisos_rol (
```

```
  rol     text not null constraint permisos_rol_nivel check (rol in ('supervisor', 'cajero', 'consulta')),
  permiso text not null constraint permisos_rol_permiso check (permiso in (
    'inventario.ver', 'inventario.editar', 'inventario.entradas', 'inventario.conteo',
    'ventas.registrar', 'ventas.ver', 'ventas.anular', 'finanzas.ver', 'finanzas.editar',
    'reportes.ver', 'respaldo.descargar', 'respaldo.restaurar')),
  primary key (rol, permiso)
);

-- Matriz inicial (la misma de src/lib/permisos.js); solo se carga si la tabla está vacía,
-- así volver a ejecutar este archivo no borra lo que el administrador configuró.
insert into public.permisos_rol (rol, permiso)
select v.rol, v.permiso
from (values
  ('supervisor', 'inventario.ver'), ('supervisor', 'inventario.editar'), ('supervisor', 'inventario.entradas'),
  ('supervisor', 'inventario.conteo'), ('supervisor', 'ventas.registrar'), ('supervisor', 'ventas.ver'),
  ('supervisor', 'ventas.anular'), ('supervisor', 'finanzas.ver'), ('supervisor', 'finanzas.editar'),
  ('supervisor', 'reportes.ver'), ('supervisor', 'respaldo.descargar'),
  ('cajero', 'inventario.ver'), ('cajero', 'ventas.registrar'), ('cajero', 'ventas.ver'),
  ('consulta', 'inventario.ver'), ('consulta', 'ventas.ver'), ('consulta', 'finanzas.ver'), ('consulta',
'reportes.ver')
) as v(rol, permiso)
where not exists (select 1 from public.permisos_rol);

-- ---------------------------------------------------------------------
--  2. Tablas de negocio
-- ---------------------------------------------------------------------

create table if not exists public.categorias (
  id           uuid primary key default gen_random_uuid(),
  nombre       text not null constraint categorias_nombre_longitud check (char_length(btrim(nombre)) between 2
and 60),
  descripcion  text not null default '' constraint categorias_descripcion_longitud check
(char_length(descripcion) <= 120),
  creado_en    timestamptz not null default now()
);
create unique index if not exists categorias_nombre_unico on public.categorias (lower(btrim(nombre)));

create table if not exists public.productos (
  id             uuid primary key default gen_random_uuid(),
  nombre         text not null constraint productos_nombre_longitud check (char_length(btrim(nombre)) between 2
and 120),
  codigo_barras  text constraint productos_codigo_formato check (codigo_barras is null or codigo_barras ~ '^[A-
Za-z0-9-]{4,40}$'),
  categoria_id   uuid references public.categorias (id) on delete set null,
  precio_compra  numeric(14,2) not null default 0 constraint productos_precio_compra_rango check (precio_compra
between 0 and 100000000),
  precio_venta   numeric(14,2) not null constraint productos_precio_venta_rango check (precio_venta > 0 and
precio_venta <= 100000000),
  cantidad       integer not null default 0 constraint productos_cantidad_rango check (cantidad between 0 and
1000000),
  stock_minimo   integer not null default 0 constraint productos_stock_minimo_rango check (stock_minimo between 0
and 100000),
  activo         boolean not null default true,
  creado_en      timestamptz not null default now()
);
-- Un código de barras no puede repetirse entre productos activos.
create unique index if not exists productos_codigo_unico
  on public.productos (codigo_barras)
```

```
  where activo and codigo_barras is not null;
create index if not exists productos_categoria_idx on public.productos (categoria_id);

create table if not exists public.ventas (
  id          uuid primary key default gen_random_uuid(),
  fecha       timestamptz not null default now(),
  total       numeric(14,2) not null default 0 constraint ventas_total_rango check (total >= 0),
  estado      text not null default 'registrada' constraint ventas_estado_valido check (estado in ('registrada',
'anulada')),
  usuario_id  uuid default auth.uid() references auth.users (id) on delete set null,
  creado_en   timestamptz not null default now()
);
create index if not exists ventas_fecha_idx on public.ventas (fecha);

create table if not exists public.detalle_venta (
  id               uuid primary key default gen_random_uuid(),
  venta_id         uuid not null references public.ventas (id) on delete cascade,
  producto_id      uuid references public.productos (id) on delete set null,
  nombre_producto  text not null,
  cantidad         integer not null constraint detalle_cantidad_rango check (cantidad between 1 and 1000000),
  precio_unitario  numeric(14,2) not null constraint detalle_precio_rango check (precio_unitario between 0 and
100000000),
  subtotal         numeric(14,2) not null,
  constraint detalle_subtotal_correcto check (subtotal = cantidad * precio_unitario)
);
create index if not exists detalle_venta_venta_idx on public.detalle_venta (venta_id);

create table if not exists public.movimientos_inventario (
  id                uuid primary key default gen_random_uuid(),
  fecha             timestamptz not null default now(),
  producto_id       uuid references public.productos (id) on delete set null,
  nombre_producto   text not null,
  tipo              text not null constraint movimientos_tipo_valido check (tipo in ('entrada', 'salida',
'ajuste')),
  cantidad          integer not null constraint movimientos_cantidad_no_cero check (cantidad <> 0),  -- + entra,
- sale
  stock_resultante  integer not null constraint movimientos_stock_rango check (stock_resultante between 0 and
1000000),
  motivo            text not null default '' constraint movimientos_motivo_longitud check (char_length(motivo) <=
120),
  referencia_id     uuid,
  constraint movimientos_signo_coherente check (
    (tipo = 'entrada' and cantidad > 0) or (tipo = 'salida' and cantidad < 0) or tipo = 'ajuste'
  )
);
create index if not exists movimientos_fecha_idx on public.movimientos_inventario (fecha);
create index if not exists movimientos_producto_idx on public.movimientos_inventario (producto_id);

create table if not exists public.ingresos (
  id          uuid primary key default gen_random_uuid(),
  fecha       date not null default current_date constraint ingresos_fecha_minima check (fecha >= date '2000-01-
01'),
  concepto    text not null constraint ingresos_concepto_longitud check (char_length(btrim(concepto)) between 3
and 140),
  monto       numeric(14,2) not null constraint ingresos_monto_rango check (monto > 0 and monto <= 1000000000),
  tipo        text not null default 'otro' constraint ingresos_tipo_valido check (tipo in ('venta', 'aporte',
'otro')),
  venta_id    uuid references public.ventas (id) on delete set null,
```

```
  usuario_id  uuid default auth.uid() references auth.users (id) on delete set null,
  creado_en   timestamptz not null default now()
);
create index if not exists ingresos_fecha_idx on public.ingresos (fecha);
create index if not exists ingresos_venta_idx on public.ingresos (venta_id);

create table if not exists public.egresos (
  id             uuid primary key default gen_random_uuid(),
  fecha          date not null default current_date constraint egresos_fecha_minima check (fecha >= date '2000-
01-01'),
  concepto       text not null constraint egresos_concepto_longitud check (char_length(btrim(concepto)) between 3
and 140),
  monto          numeric(14,2) not null constraint egresos_monto_rango check (monto > 0 and monto <= 1000000000),
  tipo           text not null default 'otros' constraint egresos_tipo_valido
                 check (tipo in ('compra_mercancia', 'servicios_publicos', 'arriendo', 'nomina', 'transporte',
'otros')),
  referencia_id  uuid,
  usuario_id     uuid default auth.uid() references auth.users (id) on delete set null,
  creado_en      timestamptz not null default now()
);
create index if not exists egresos_fecha_idx on public.egresos (fecha);

-- ---------------------------------------------------------------------
--  3. Funciones auxiliares (esquema privado)
-- ---------------------------------------------------------------------

-- ¿El usuario de la sesión está autorizado y activo?
create or replace function privado.es_usuario_activo()
returns boolean
language sql stable security definer
set search_path = ''
as $$
  select exists (select 1 from public.usuarios u where u.id = auth.uid() and u.activo)
$$;

-- ¿La sentencia se ejecuta dentro de una operación del sistema (venta,
-- entrada, conteo, restauración)? Solo las funciones de este archivo
-- activan la marca; la API no permite modificarla.
create or replace function privado.en_operacion()
returns boolean
language sql stable
set search_path = ''
as $$
  select coalesce(current_setting('britnymax.operacion', true), '') = 'si'
$$;

-- Nivel del usuario de la sesión (null si no está autorizado o está inactivo).
create or replace function privado.rol_actual()
returns text
language sql stable security definer
set search_path = ''
as $$
  select u.rol from public.usuarios u where u.id = auth.uid() and u.activo
$$;

-- ¿El usuario de la sesión tiene alguno de estos niveles?
create or replace function privado.tiene_rol(variadic p_roles text[])
returns boolean
```

```
language sql stable security definer
set search_path = ''
as $$
  select coalesce(privado.rol_actual() = any (p_roles), false)
$$;

create or replace function privado.nombre_rol(p_rol text)
returns text
language sql immutable
set search_path = ''
as $$
  select case p_rol
    when 'administrador' then 'Administrador'
    when 'supervisor' then 'Supervisor'
    when 'cajero' then 'Cajero'
    when 'consulta' then 'Consulta'
    else 'Sin nivel'
  end
$$;

-- Corta la operación si el usuario no está autorizado o si su nivel no alcanza.
-- Cada función pública activa la marca de operación con set_config(..., true)
-- y la apaga antes de terminar.
create or replace function privado.exigir_rol(variadic p_roles text[])
returns void
language plpgsql stable
set search_path = ''
as $$
declare
  v_rol text := privado.rol_actual();
begin
  if v_rol is null then
    raise exception 'Tu usuario no está autorizado para usar Britny Max' using errcode = '42501';
  end if;
  if not (v_rol = any (p_roles)) then
    raise exception 'Tu nivel de acceso (%) no permite esta operación', privado.nombre_rol(v_rol) using errcode =
'42501';
  end if;
end $$;

-- ¿El usuario de la sesión tiene este permiso? El administrador los tiene todos.
create or replace function privado.tiene_permiso(p_permiso text)
returns boolean
language sql stable security definer
set search_path = ''
as $$
  select coalesce(
    privado.rol_actual() = 'administrador'
    or exists (select 1 from public.permisos_rol p where p.rol = privado.rol_actual() and p.permiso = p_permiso),
    false)
$$;

-- Corta la operación si el usuario no está autorizado o si su nivel no tiene el permiso.
create or replace function privado.exigir_permiso(p_permiso text)
returns void
language plpgsql stable
set search_path = ''
as $$
```

```
declare
  v_rol text := privado.rol_actual();
begin
  if v_rol is null then
    raise exception 'Tu usuario no está autorizado para usar Britny Max' using errcode = '42501';
  end if;
  if not privado.tiene_permiso(p_permiso) then
    raise exception 'Tu nivel de acceso (%) no permite esta operación', privado.nombre_rol(v_rol) using errcode =
'42501';
  end if;
end $$;

create or replace function privado.validar_fecha_operacion(p_fecha timestamptz)
returns timestamptz
language plpgsql stable
set search_path = ''
as $$
declare
  v timestamptz := coalesce(p_fecha, now());
begin
  if v > now() + interval '5 minutes' then raise exception 'La fecha no puede ser posterior a hoy'; end if;
  if v < timestamptz '2000-01-01 00:00:00-05' then raise exception 'La fecha es demasiado antigua'; end if;
  return v;
end $$;

create or replace function privado.dia_bogota(p_momento timestamptz)
returns date
language sql immutable
set search_path = ''
as $$
  select (p_momento at time zone 'America/Bogota')::date
$$;

grant usage on schema privado to authenticated;
revoke all on all functions in schema privado from public, anon;
grant execute on function privado.es_usuario_activo() to authenticated;
grant execute on function privado.en_operacion() to authenticated;
grant execute on function privado.rol_actual() to authenticated;
grant execute on function privado.tiene_rol(text[]) to authenticated;
grant execute on function privado.nombre_rol(text) to authenticated;
grant execute on function privado.exigir_rol(text[]) to authenticated;
grant execute on function privado.tiene_permiso(text) to authenticated;
grant execute on function privado.exigir_permiso(text) to authenticated;
grant execute on function privado.validar_fecha_operacion(timestamptz) to authenticated;
grant execute on function privado.dia_bogota(timestamptz) to authenticated;

-- ---------------------------------------------------------------------
--  4. Disparadores de integridad
-- ---------------------------------------------------------------------

-- Las existencias solo cambian con ventas, entradas, conteos o restauraciones.
create or replace function privado.proteger_existencias()
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  if not privado.en_operacion() then
```

```
    if tg_op = 'INSERT' and new.cantidad <> 0 then
      raise exception 'Las existencias iniciales se registran con la función crear_producto';
    elsif tg_op = 'UPDATE' and new.cantidad is distinct from old.cantidad then
      raise exception 'Las existencias solo cambian con ventas, entradas de mercancía o conteos físicos';
    end if;
  end if;
  return new;
end $$;

drop trigger if exists productos_proteger_existencias on public.productos;
create trigger productos_proteger_existencias
  before insert or update on public.productos
  for each row execute function privado.proteger_existencias();

-- Normaliza textos (sin espacios sobrantes; código vacío = sin código).
create or replace function privado.normalizar_producto()
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  new.nombre := btrim(new.nombre);
  new.codigo_barras := nullif(btrim(new.codigo_barras), '');
  return new;
end $$;

drop trigger if exists productos_normalizar on public.productos;
create trigger productos_normalizar
  before insert or update on public.productos
  for each row execute function privado.normalizar_producto();

create or replace function privado.normalizar_categoria()
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  new.nombre := btrim(new.nombre);
  new.descripcion := btrim(coalesce(new.descripcion, ''));
  return new;
end $$;

drop trigger if exists categorias_normalizar on public.categorias;
create trigger categorias_normalizar
  before insert or update on public.categorias
  for each row execute function privado.normalizar_categoria();

-- No se elimina una categoría con productos activos.
create or replace function privado.validar_eliminar_categoria()
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  if exists (select 1 from public.productos p where p.categoria_id = old.id and p.activo) then
    raise exception 'No se puede eliminar: la categoría tiene productos activos';
  end if;
  return old;
```

```
end $$;

drop trigger if exists categorias_validar_eliminar on public.categorias;
create trigger categorias_validar_eliminar
  before delete on public.categorias
  for each row execute function privado.validar_eliminar_categoria();

-- Ingresos y egresos: concepto sin espacios sobrantes y fecha no futura.
create or replace function privado.validar_movimiento_economico()
returns trigger
language plpgsql
set search_path = ''
as $$
begin
  new.concepto := btrim(new.concepto);
  if new.fecha > privado.dia_bogota(now()) and not privado.en_operacion() then
    raise exception 'La fecha no puede ser posterior a hoy';
  end if;
  return new;
end $$;

drop trigger if exists ingresos_validar on public.ingresos;
create trigger ingresos_validar
  before insert or update on public.ingresos
  for each row execute function privado.validar_movimiento_economico();

drop trigger if exists egresos_validar on public.egresos;
create trigger egresos_validar
  before insert or update on public.egresos
  for each row execute function privado.validar_movimiento_economico();

-- ------------------------------------------------------------------------
--  5. Seguridad (RNF02): Row Level Security con 4 niveles de acceso
--     1 administrador · 2 supervisor · 3 cajero · 4 consulta
--     · Solo usuarios autorizados y activos ven datos.
--     · Ventas, detalles y movimientos solo se escriben desde las
--       funciones de la sección 6, que verifican el nivel.
-- ------------------------------------------------------------------------

alter table public.usuarios               enable row level security;
alter table public.permisos_rol           enable row level security;
alter table public.categorias             enable row level security;
alter table public.productos              enable row level security;
alter table public.ventas                 enable row level security;
alter table public.detalle_venta          enable row level security;
alter table public.movimientos_inventario enable row level security;
alter table public.ingresos               enable row level security;
alter table public.egresos                enable row level security;

-- Elimina las políticas anteriores para poder re-ejecutar este archivo.
do $$
declare r record;
begin
  for r in
    select policyname, tablename from pg_policies
    where schemaname = 'public'
      and tablename in ('usuarios', 'permisos_rol', 'categorias', 'productos', 'ventas', 'detalle_venta',
                        'movimientos_inventario', 'ingresos', 'egresos')
```

```
  loop
    execute format('drop policy if exists %I on public.%I', r.policyname, r.tablename);
  end loop;
end $$;

-- Cada usuario ve su perfil; el administrador ve todos. Los cambios se hacen
-- solo con las funciones de la sección 7.
create policy usuarios_ver on public.usuarios
  for select to authenticated
  using (id = (select auth.uid()) or (select privado.tiene_rol('administrador')));

-- Inventario: todos los niveles consultan; edita quien tenga «inventario.editar».
create policy categorias_ver on public.categorias
  for select to authenticated using ((select privado.es_usuario_activo()));
create policy categorias_crear on public.categorias
  for insert to authenticated with check ((select privado.tiene_permiso('inventario.editar')));
create policy categorias_editar on public.categorias
  for update to authenticated
  using ((select privado.tiene_permiso('inventario.editar')))
  with check ((select privado.tiene_permiso('inventario.editar')));
create policy categorias_eliminar on public.categorias
  for delete to authenticated using ((select privado.tiene_permiso('inventario.editar')));

-- Productos: sin eliminación física (se retiran con activo = false). Las
-- existencias de una venta del cajero se descuentan dentro de registrar_venta.
create policy productos_ver on public.productos
  for select to authenticated using ((select privado.es_usuario_activo()));
create policy productos_crear on public.productos
  for insert to authenticated with check ((select privado.tiene_permiso('inventario.editar')));
create policy productos_editar on public.productos
  for update to authenticated
  using ((select privado.tiene_permiso('inventario.editar'))
         or ((select privado.es_usuario_activo()) and (select privado.en_operacion())))
  with check ((select privado.tiene_permiso('inventario.editar'))
         or ((select privado.es_usuario_activo()) and (select privado.en_operacion())));

-- Ventas, detalles y movimientos: lectura para todos los niveles; escritura
-- solo dentro de las funciones de la sección 6.
create policy ventas_ver on public.ventas
  for select to authenticated using ((select privado.es_usuario_activo()));
create policy ventas_operacion_crear on public.ventas
  for insert to authenticated with check ((select privado.es_usuario_activo()) and (select
privado.en_operacion()));
create policy ventas_operacion_editar on public.ventas
  for update to authenticated using ((select privado.es_usuario_activo()) and (select privado.en_operacion()));

create policy detalle_ver on public.detalle_venta
  for select to authenticated using ((select privado.es_usuario_activo()));
create policy detalle_operacion_crear on public.detalle_venta
  for insert to authenticated with check ((select privado.es_usuario_activo()) and (select
privado.en_operacion()));

create policy movimientos_ver on public.movimientos_inventario
  for select to authenticated using ((select privado.es_usuario_activo()));
create policy movimientos_operacion_crear on public.movimientos_inventario
  for insert to authenticated with check ((select privado.es_usuario_activo()) and (select
privado.en_operacion()));
```

```
-- Finanzas: las ve quien tenga «finanzas.ver» y las modifica quien tenga
-- «finanzas.editar». Los ingresos y egresos que generan las operaciones
-- (ventas, anulaciones, entradas) se escriben dentro de ellas, que ya
-- comprueban su propio permiso; el ingreso de una venta solo se retira al anularla.
create policy ingresos_ver on public.ingresos
  for select to authenticated using ((select privado.tiene_permiso('finanzas.ver')));
create policy ingresos_crear on public.ingresos
  for insert to authenticated
  with check (
    ((select privado.tiene_permiso('finanzas.editar')) and venta_id is null)
    or ((select privado.es_usuario_activo()) and (select privado.en_operacion()))
  );
create policy ingresos_editar on public.ingresos
  for update to authenticated
  using (((select privado.tiene_permiso('finanzas.editar')) and venta_id is null)
         or ((select privado.es_usuario_activo()) and (select privado.en_operacion())))
  with check (((select privado.tiene_permiso('finanzas.editar')) and venta_id is null)
         or ((select privado.es_usuario_activo()) and (select privado.en_operacion())));
create policy ingresos_eliminar on public.ingresos
  for delete to authenticated
  using (((select privado.tiene_permiso('finanzas.editar')) and venta_id is null)
         or ((select privado.es_usuario_activo()) and (select privado.en_operacion())));

create policy egresos_ver on public.egresos
  for select to authenticated using ((select privado.tiene_permiso('finanzas.ver')));
create policy egresos_crear on public.egresos
  for insert to authenticated
  with check ((select privado.tiene_permiso('finanzas.editar'))
              or ((select privado.es_usuario_activo()) and (select privado.en_operacion())));
create policy egresos_editar on public.egresos
  for update to authenticated
  using ((select privado.tiene_permiso('finanzas.editar')))
  with check ((select privado.tiene_permiso('finanzas.editar')));
create policy egresos_eliminar on public.egresos
  for delete to authenticated using ((select privado.tiene_permiso('finanzas.editar')));

-- Permisos por nivel: todos los usuarios activos leen la matriz (la app la usa
-- para mostrar el menú); solo se modifica con guardar_permisos (administrador).
create policy permisos_ver on public.permisos_rol
  for select to authenticated using ((select privado.es_usuario_activo()));

-- Privilegios: los visitantes anónimos no acceden a nada.
revoke all on all tables in schema public from anon;
grant select, insert, update, delete on all tables in schema public to authenticated;

-- ---------------------------------------------------------------------
--  6. Operaciones transaccionales (RNF07 – integridad de los datos)
--     Cada función se ejecuta completa o no aplica ningún cambio.
-- ---------------------------------------------------------------------

-- RF02: crea el producto y registra su inventario inicial.
create or replace function public.crear_producto(p_producto jsonb)
returns uuid
language plpgsql
set search_path = ''
as $$
declare
  v_id       uuid;
```

```
  v_cantidad integer;
begin
  perform privado.exigir_permiso('inventario.editar');
  perform set_config('britnymax.operacion', 'si', true);
  if jsonb_typeof(p_producto) is distinct from 'object' then raise exception 'Datos del producto inválidos'; end
if;

  v_cantidad := coalesce((p_producto ->> 'cantidad')::integer, 0);
  if v_cantidad < 0 or v_cantidad > 1000000 then raise exception 'La cantidad inicial no es válida'; end if;
  if nullif(p_producto ->> 'categoria_id', '') is null then raise exception 'Selecciona una categoría'; end if;

  insert into public.productos (nombre, codigo_barras, categoria_id, precio_compra, precio_venta, cantidad,
stock_minimo)
  values (
    p_producto ->> 'nombre',
    p_producto ->> 'codigo_barras',
    (p_producto ->> 'categoria_id')::uuid,
    coalesce((p_producto ->> 'precio_compra')::numeric, 0),
    (p_producto ->> 'precio_venta')::numeric,
    v_cantidad,
    coalesce((p_producto ->> 'stock_minimo')::integer, 0)
  )
  returning id into v_id;

  if v_cantidad > 0 then
    insert into public.movimientos_inventario (producto_id, nombre_producto, tipo, cantidad, stock_resultante,
motivo, referencia_id)
    select v_id, p.nombre, 'entrada', v_cantidad, v_cantidad, 'Inventario inicial', v_id
    from public.productos p where p.id = v_id;
  end if;
  perform set_config('britnymax.operacion', '', true);
  return v_id;
end $$;

-- RF07: registra la venta, descuenta existencias y genera el ingreso.
-- p_items: [{ "producto_id": uuid, "cantidad": int, "precio_unitario": numeric }]
create or replace function public.registrar_venta(p_fecha timestamptz, p_items jsonb)
returns uuid
language plpgsql
set search_path = ''
as $$
declare
  v_fecha  timestamptz;
  v_venta  uuid;
  v_total  numeric := 0;
  v_cant   integer;
  v_precio numeric;
  it       jsonb;
  p        public.productos%rowtype;
begin
  perform privado.exigir_permiso('ventas.registrar');
  perform set_config('britnymax.operacion', 'si', true);
  v_fecha := privado.validar_fecha_operacion(p_fecha);
  if jsonb_typeof(p_items) is distinct from 'array' or jsonb_array_length(p_items) = 0 then
    raise exception 'La venta no tiene productos';
  end if;
  if jsonb_array_length(p_items) > 200 then raise exception 'Una venta admite máximo 200 productos'; end if;
```

```
  insert into public.ventas (fecha, total) values (v_fecha, 0) returning id into v_venta;

  for it in select * from jsonb_array_elements(p_items) loop
    select * into p from public.productos where id = (it ->> 'producto_id')::uuid and activo for update;
    if not found then raise exception 'Uno de los productos ya no existe en el catálogo'; end if;

    v_cant := (it ->> 'cantidad')::integer;
    if v_cant is null or v_cant <= 0 then raise exception 'Cantidad inválida para %', p.nombre; end if;
    if p.cantidad < v_cant then
      raise exception 'Stock insuficiente para % (disponible: %)', p.nombre, p.cantidad;
    end if;
    v_precio := coalesce((it ->> 'precio_unitario')::numeric, p.precio_venta);
    if v_precio < 0 then raise exception 'Precio inválido para %', p.nombre; end if;

    update public.productos set cantidad = cantidad - v_cant where id = p.id;

    insert into public.detalle_venta (venta_id, producto_id, nombre_producto, cantidad, precio_unitario,
subtotal)
    values (v_venta, p.id, p.nombre, v_cant, v_precio, v_cant * v_precio);

    insert into public.movimientos_inventario (fecha, producto_id, nombre_producto, tipo, cantidad,
stock_resultante, motivo, referencia_id)
    values (v_fecha, p.id, p.nombre, 'salida', -v_cant, p.cantidad - v_cant, 'Venta', v_venta);

    v_total := v_total + v_cant * v_precio;
  end loop;

  update public.ventas set total = v_total where id = v_venta;

  if v_total > 0 then
    insert into public.ingresos (fecha, concepto, monto, tipo, venta_id)
    values (privado.dia_bogota(v_fecha), 'Venta de productos', v_total, 'venta', v_venta);
  end if;
  perform set_config('britnymax.operacion', '', true);
  return v_venta;
end $$;

-- Anula una venta: devuelve existencias y retira el ingreso asociado.
create or replace function public.anular_venta(p_venta uuid)
returns void
language plpgsql
set search_path = ''
as $$
declare
  v       public.ventas%rowtype;
  d       public.detalle_venta%rowtype;
  v_stock integer;
begin
  perform privado.exigir_permiso('ventas.anular');
  perform set_config('britnymax.operacion', 'si', true);
  select * into v from public.ventas where id = p_venta for update;
  if not found then raise exception 'Venta no encontrada'; end if;
  if v.estado = 'anulada' then raise exception 'La venta ya fue anulada'; end if;

  for d in select * from public.detalle_venta where venta_id = p_venta loop
    if d.producto_id is not null then
      update public.productos set cantidad = cantidad + d.cantidad where id = d.producto_id
      returning cantidad into v_stock;
```

```
      insert into public.movimientos_inventario (producto_id, nombre_producto, tipo, cantidad, stock_resultante,
motivo, referencia_id)
      values (d.producto_id, d.nombre_producto, 'entrada', d.cantidad, v_stock, 'Anulación de venta', p_venta);
    end if;
  end loop;

  update public.ventas set estado = 'anulada' where id = p_venta;
  delete from public.ingresos where venta_id = p_venta;
  perform set_config('britnymax.operacion', '', true);
end $$;

-- Entrada de mercancía: suma existencias y, si se indica, registra el egreso.
-- p_items: [{ "producto_id": uuid, "cantidad": int, "costo_unitario": numeric }]
create or replace function public.registrar_entrada(
  p_fecha timestamptz,
  p_concepto text,
  p_items jsonb,
  p_registrar_egreso boolean,
  p_actualizar_precio boolean
) returns uuid
language plpgsql
set search_path = ''
as $$
declare
  v_fecha    timestamptz;
  v_ref      uuid := gen_random_uuid();
  v_concepto text := btrim(coalesce(p_concepto, ''));
  v_total    numeric := 0;
  v_cant     integer;
  v_costo    numeric;
  it         jsonb;
  p          public.productos%rowtype;
begin
  perform privado.exigir_permiso('inventario.entradas');
  perform set_config('britnymax.operacion', 'si', true);
  v_fecha := privado.validar_fecha_operacion(p_fecha);
  if char_length(v_concepto) > 120 then raise exception 'El concepto admite máximo 120 caracteres'; end if;
  if jsonb_typeof(p_items) is distinct from 'array' or jsonb_array_length(p_items) = 0 then
    raise exception 'La entrada no tiene productos';
  end if;
  if jsonb_array_length(p_items) > 200 then raise exception 'Una entrada admite máximo 200 productos'; end if;

  for it in select * from jsonb_array_elements(p_items) loop
    select * into p from public.productos where id = (it ->> 'producto_id')::uuid and activo for update;
    if not found then raise exception 'Uno de los productos ya no existe en el catálogo'; end if;

    v_cant := (it ->> 'cantidad')::integer;
    if v_cant is null or v_cant <= 0 then raise exception 'Cantidad inválida para %', p.nombre; end if;
    v_costo := coalesce((it ->> 'costo_unitario')::numeric, 0);
    if v_costo < 0 or v_costo > 100000000 then raise exception 'Costo inválido para %', p.nombre; end if;

    update public.productos
       set cantidad = cantidad + v_cant,
           precio_compra = case when p_actualizar_precio and v_costo > 0 then v_costo else precio_compra end
     where id = p.id;

    insert into public.movimientos_inventario (fecha, producto_id, nombre_producto, tipo, cantidad,
stock_resultante, motivo, referencia_id)
```

```
    values (v_fecha, p.id, p.nombre, 'entrada', v_cant, p.cantidad + v_cant,
            case when v_concepto = '' then 'Entrada de mercancía' else v_concepto end, v_ref);

    v_total := v_total + v_cant * v_costo;
  end loop;

  if coalesce(p_registrar_egreso, false) and v_total > 0 then
    insert into public.egresos (fecha, concepto, monto, tipo, referencia_id)
    values (privado.dia_bogota(v_fecha),
            case when char_length(v_concepto) >= 3 then v_concepto else 'Compra de mercancía' end,
            v_total, 'compra_mercancia', v_ref);
  end if;
  perform set_config('britnymax.operacion', '', true);
  return v_ref;
end $$;

-- Conteo físico: fija la cantidad contada y deja el ajuste registrado.
-- p_items: [{ "producto_id": uuid, "cantidad_fisica": int }]
create or replace function public.ajustar_inventario(p_fecha timestamptz, p_motivo text, p_items jsonb)
returns integer
language plpgsql
set search_path = ''
as $$
declare
  v_fecha   timestamptz;
  v_ref     uuid := gen_random_uuid();
  v_motivo  text := btrim(coalesce(p_motivo, ''));
  v_fisica  integer;
  v_dif     integer;
  v_ajustes integer := 0;
  it        jsonb;
  p         public.productos%rowtype;
begin
  perform privado.exigir_permiso('inventario.conteo');
  perform set_config('britnymax.operacion', 'si', true);
  v_fecha := privado.validar_fecha_operacion(p_fecha);
  if char_length(v_motivo) < 3 then v_motivo := 'Conteo físico'; end if;
  if char_length(v_motivo) > 120 then raise exception 'El motivo admite máximo 120 caracteres'; end if;
  if jsonb_typeof(coalesce(p_items, '[]'::jsonb)) <> 'array' then raise exception 'Datos del conteo inválidos';
end if;

  for it in select * from jsonb_array_elements(coalesce(p_items, '[]'::jsonb)) loop
    select * into p from public.productos where id = (it ->> 'producto_id')::uuid and activo for update;
    if not found then raise exception 'Uno de los productos ya no existe en el catálogo'; end if;

    v_fisica := (it ->> 'cantidad_fisica')::integer;
    if v_fisica is null or v_fisica < 0 or v_fisica > 1000000 then
      raise exception 'Cantidad inválida para %', p.nombre;
    end if;

    v_dif := v_fisica - p.cantidad;
    if v_dif <> 0 then
      update public.productos set cantidad = v_fisica where id = p.id;
      insert into public.movimientos_inventario (fecha, producto_id, nombre_producto, tipo, cantidad,
stock_resultante, motivo, referencia_id)
      values (v_fecha, p.id, p.nombre, 'ajuste', v_dif, v_fisica, v_motivo, v_ref);
      v_ajustes := v_ajustes + 1;
    end if;
```

```
  end loop;
  perform set_config('britnymax.operacion', '', true);
  return v_ajustes;
end $$;

-- RNF09: restaura un bloque de registros desde un archivo de respaldo.
-- Se llama tabla por tabla, en este orden: categorias, productos, ventas,
-- detalle_venta, movimientos, ingresos, egresos. Es idempotente.
create or replace function public.importar_respaldo(p_tabla text, p_filas jsonb)
returns integer
language plpgsql
set search_path = ''
as $$
declare
  v_filas integer := 0;
begin
  perform privado.exigir_permiso('respaldo.restaurar');
  perform set_config('britnymax.operacion', 'si', true);
  if jsonb_typeof(p_filas) is distinct from 'array' then raise exception 'Formato de respaldo inválido'; end if;
  if jsonb_array_length(p_filas) > 1000 then raise exception 'Bloque de respaldo demasiado grande'; end if;

  case p_tabla
    when 'categorias' then
      insert into public.categorias (id, nombre, descripcion, creado_en)
      select r.id, r.nombre, coalesce(r.descripcion, ''), coalesce(r.creado_en, now())
      from jsonb_populate_recordset(null::public.categorias, p_filas) r
      on conflict (id) do update set nombre = excluded.nombre, descripcion = excluded.descripcion;

    when 'productos' then
      insert into public.productos (id, nombre, codigo_barras, categoria_id, precio_compra, precio_venta,
cantidad, stock_minimo, activo, creado_en)
      select r.id, r.nombre, r.codigo_barras, r.categoria_id, coalesce(r.precio_compra, 0), r.precio_venta,
             coalesce(r.cantidad, 0), coalesce(r.stock_minimo, 0), coalesce(r.activo, true),
coalesce(r.creado_en, now())
      from jsonb_populate_recordset(null::public.productos, p_filas) r
      on conflict (id) do update set
        nombre = excluded.nombre, codigo_barras = excluded.codigo_barras, categoria_id = excluded.categoria_id,
        precio_compra = excluded.precio_compra, precio_venta = excluded.precio_venta, cantidad =
excluded.cantidad,
        stock_minimo = excluded.stock_minimo, activo = excluded.activo;

    when 'ventas' then
      insert into public.ventas (id, fecha, total, estado, creado_en)
      select r.id, r.fecha, coalesce(r.total, 0), coalesce(r.estado, 'registrada'), coalesce(r.creado_en, now())
      from jsonb_populate_recordset(null::public.ventas, p_filas) r
      on conflict (id) do update set total = excluded.total, estado = excluded.estado;

    when 'detalle_venta' then
      insert into public.detalle_venta (id, venta_id, producto_id, nombre_producto, cantidad, precio_unitario,
subtotal)
      select r.id, r.venta_id, r.producto_id, r.nombre_producto, r.cantidad, r.precio_unitario, r.subtotal
      from jsonb_populate_recordset(null::public.detalle_venta, p_filas) r
      on conflict (id) do nothing;

    when 'movimientos' then
      insert into public.movimientos_inventario (id, fecha, producto_id, nombre_producto, tipo, cantidad,
stock_resultante, motivo, referencia_id)
```

```
      select r.id, r.fecha, r.producto_id, r.nombre_producto, r.tipo, r.cantidad, r.stock_resultante,
coalesce(r.motivo, ''), r.referencia_id
      from jsonb_populate_recordset(null::public.movimientos_inventario, p_filas) r
      on conflict (id) do nothing;

    when 'ingresos' then
      insert into public.ingresos (id, fecha, concepto, monto, tipo, venta_id, creado_en)
      select r.id, r.fecha, r.concepto, r.monto, coalesce(r.tipo, 'otro'), r.venta_id, coalesce(r.creado_en,
now())
      from jsonb_populate_recordset(null::public.ingresos, p_filas) r
      on conflict (id) do update set fecha = excluded.fecha, concepto = excluded.concepto, monto =
excluded.monto, tipo = excluded.tipo;

    when 'egresos' then
      insert into public.egresos (id, fecha, concepto, monto, tipo, referencia_id, creado_en)
      select r.id, r.fecha, r.concepto, r.monto, coalesce(r.tipo, 'otros'), r.referencia_id,
coalesce(r.creado_en, now())
      from jsonb_populate_recordset(null::public.egresos, p_filas) r
      on conflict (id) do update set fecha = excluded.fecha, concepto = excluded.concepto, monto =
excluded.monto, tipo = excluded.tipo;

    else
      raise exception 'Tabla de respaldo desconocida: %', p_tabla;
  end case;

  get diagnostics v_filas = row_count;
  perform set_config('britnymax.operacion', '', true);
  return v_filas;
end $$;

-- Solo usuarios autenticados ejecutan las operaciones (y dentro de ellas se
-- verifica que estén autorizados en public.usuarios).
revoke execute on function public.crear_producto(jsonb) from public, anon;
revoke execute on function public.registrar_venta(timestamptz, jsonb) from public, anon;
revoke execute on function public.anular_venta(uuid) from public, anon;
revoke execute on function public.registrar_entrada(timestamptz, text, jsonb, boolean, boolean) from public,
anon;
revoke execute on function public.ajustar_inventario(timestamptz, text, jsonb) from public, anon;
revoke execute on function public.importar_respaldo(text, jsonb) from public, anon;

grant execute on function public.crear_producto(jsonb) to authenticated;
grant execute on function public.registrar_venta(timestamptz, jsonb) to authenticated;
grant execute on function public.anular_venta(uuid) to authenticated;
grant execute on function public.registrar_entrada(timestamptz, text, jsonb, boolean, boolean) to authenticated;
grant execute on function public.ajustar_inventario(timestamptz, text, jsonb) to authenticated;
grant execute on function public.importar_respaldo(text, jsonb) to authenticated;

-- ---------------------------------------------------------------------
--  7. Gestión de usuarios (nivel 1: administrador)
--     La cuenta se crea en Supabase Auth; estas funciones la autorizan,
--     cambian su nivel o la desactivan. Son SECURITY DEFINER porque leen
--     auth.users, y por eso verifican el nivel antes de cualquier cosa.
-- ---------------------------------------------------------------------

create or replace function public.listar_usuarios()
returns table (id uuid, nombre text, correo text, rol text, activo boolean, creado_en timestamptz, ultimo_ingreso
timestamptz)
language plpgsql stable security definer
```

```
set search_path = ''
as $$
begin
  perform privado.exigir_rol('administrador');
  return query
    select u.id, u.nombre, a.email::text, u.rol, u.activo, u.creado_en, a.last_sign_in_at
    from public.usuarios u
    join auth.users a on a.id = u.id
    order by u.activo desc, u.nombre;
end $$;

create or replace function public.autorizar_usuario(p_correo text, p_nombre text, p_rol text)
returns uuid
language plpgsql security definer
set search_path = ''
as $$
declare
  v_id uuid;
begin
  perform privado.exigir_rol('administrador');
  if p_rol is null or p_rol not in ('administrador', 'supervisor', 'cajero', 'consulta') then
    raise exception 'Selecciona un nivel de acceso válido';
  end if;
  select a.id into v_id from auth.users a where lower(a.email) = lower(btrim(p_correo));
  if v_id is null then
    raise exception 'No existe una cuenta con el correo %. Créala primero en Supabase → Authentication → Users',
btrim(p_correo);
  end if;
  if exists (select 1 from public.usuarios u where u.id = v_id) then
    raise exception 'Esa cuenta ya está registrada en el sistema';
  end if;
  insert into public.usuarios (id, nombre, rol) values (v_id, btrim(p_nombre), p_rol);
  return v_id;
end $$;

create or replace function public.actualizar_usuario(p_id uuid, p_nombre text, p_rol text, p_activo boolean)
returns void
language plpgsql security definer
set search_path = ''
as $$
declare
  v_actual public.usuarios%rowtype;
  v_activo boolean := coalesce(p_activo, true);
begin
  perform privado.exigir_rol('administrador');
  if p_rol is null or p_rol not in ('administrador', 'supervisor', 'cajero', 'consulta') then
    raise exception 'Selecciona un nivel de acceso válido';
  end if;
  select * into v_actual from public.usuarios u where u.id = p_id for update;
  if not found then raise exception 'Usuario no encontrado'; end if;

  if p_id = auth.uid() and (p_rol <> v_actual.rol or not v_activo) then
    raise exception 'No puedes cambiar tu propio nivel de acceso ni desactivarte';
  end if;
  if v_actual.rol = 'administrador' and v_actual.activo
     and (p_rol <> 'administrador' or not v_activo)
     and (select count(*) from public.usuarios u where u.rol = 'administrador' and u.activo) <= 1 then
    raise exception 'Debe quedar al menos un administrador activo';
```

```
  end if;

  update public.usuarios u set nombre = btrim(p_nombre), rol = p_rol, activo = v_activo where u.id = p_id;
end $$;

-- Reemplaza los permisos de los niveles 2 a 4 por la lista recibida
-- ([{ "rol": "cajero", "permiso": "finanzas.ver" }, …]). Las restricciones de la
-- tabla rechazan niveles o permisos inválidos, incluido «usuarios.gestionar».
create or replace function public.guardar_permisos(p_permisos jsonb)
returns void
language plpgsql security definer
set search_path = ''
as $$
begin
  perform privado.exigir_rol('administrador');
  if jsonb_typeof(p_permisos) is distinct from 'array' then
    raise exception 'Formato de permisos inválido';
  end if;
  delete from public.permisos_rol where rol is not null;
  insert into public.permisos_rol (rol, permiso)
  select distinct x ->> 'rol', x ->> 'permiso' from jsonb_array_elements(p_permisos) as x;
end $$;

revoke execute on function public.guardar_permisos(jsonb) from public, anon;
grant execute on function public.guardar_permisos(jsonb) to authenticated;

revoke execute on function public.listar_usuarios() from public, anon;
revoke execute on function public.autorizar_usuario(text, text, text) from public, anon;
revoke execute on function public.actualizar_usuario(uuid, text, text, boolean) from public, anon;
grant execute on function public.listar_usuarios() to authenticated;
grant execute on function public.autorizar_usuario(text, text, text) to authenticated;
grant execute on function public.actualizar_usuario(uuid, text, text, boolean) to authenticated;

-- Función de la versión anterior, reemplazada por privado.exigir_rol.
drop function if exists privado.exigir_autorizacion();

-- ======================================================================
--  Después de ejecutar este archivo por primera vez, autoriza a la
--  propietaria como administradora (ver README):
--
--    insert into public.usuarios (id, nombre, rol)
--    select id, 'Britny Romero', 'administrador'
--    from auth.users where email = 'admin@britnymax.app';
--
--  Luego, desde la app (Usuarios y niveles) ella autoriza a las demás
--  cuentas con su nivel: supervisor, cajero o consulta.
-- ======================================================================
```

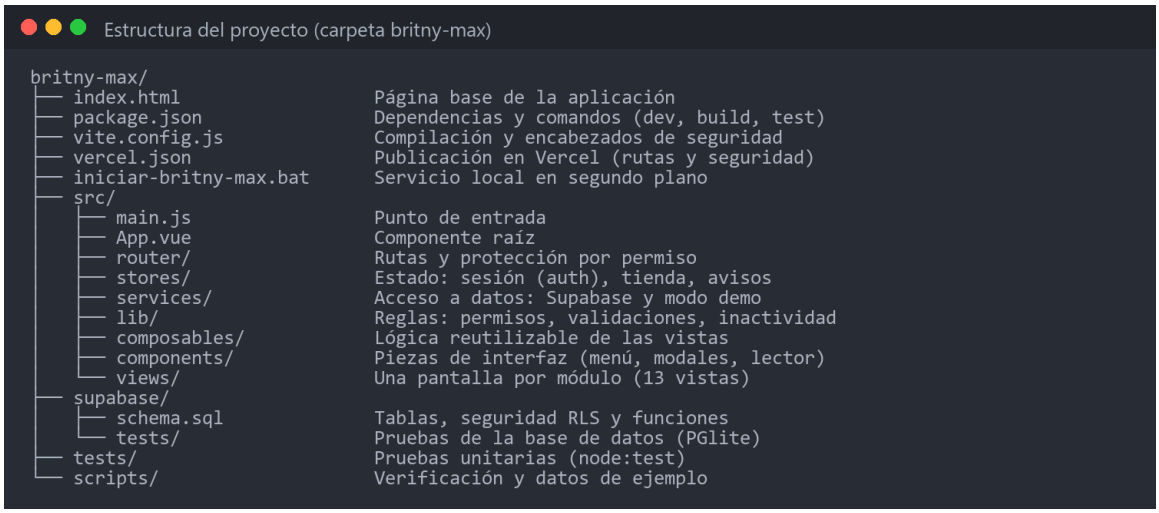
Anexo D. Código Fuente y Trazabilidad

El sistema publicado puede consultarse en https://britny-max.vercel.app con las cuentas descritas en la Tabla 21; las contraseñas se entregan por separado. El código completo se conserva en el repositorio privado del proyecto en GitHub, desde el cual Vercel publica cada versión, y el script de la base de datos está en el Anexo C. Este anexo explica la estructura del proyecto y presenta, con capturas, los módulos más importantes; la Tabla 30 relaciona cada requerimiento con los archivos que lo implementan y las pruebas que lo verifican.

Estructura del proyecto. La aplicación sigue la organización habitual de un proyecto Vue: las pantallas (views) usan componentes reutilizables (components) y toman los datos de los almacenes de estado (stores); los almacenes aplican las reglas de negocio (lib) y llaman a los servicios (services), que hablan con Supabase o, en el modo de demostración, con el navegador. La base de datos y sus pruebas están en la carpeta supabase (Figura 34).

Figura 34

Estructura de Carpetas del Proyecto



Nota. Elaboración propia.

Tabla 30

Matriz de Trazabilidad de Requerimientos

Req.	Archivos que lo implementan	Base de datos (schema.sql)	Casos y pruebas
RF01	views/LoginView.vue, stores/auth.js, services/supabaseBackend.js, router/index.js	usuarios, Supabase Auth	CP01, CP02, CP24
RF02	views/ProductoFormView.vue, stores/tienda.js, lib/validaciones.js	crear_producto, productos	CP03, CP04
RF03	views/ProductosView.vue, components/ProductPicker.vue	productos (lectura)	CP05
RF04	views/ProductosView.vue, views/ProductoFormView.vue	productos, proteger_existencias	CP06
RF05	views/LecturaCodigoView.vue, components/BarcodeCamera.vue, components/ProductPicker.vue	productos.codigo_barras	CP07
RF07	views/VentasView.vue, views/HistorialVentasView.vue, stores/tienda.js	registrar_venta, anular_venta, ventas, detalle_venta	CP08, CP09, CP10
RF11	views/FinanzasView.vue, views/EntradaMercanciaView.vue	ingresos, egresos, registrar_entrada	CP11, CP12

Req.	Archivos que lo implementan	Base de datos (schema.sql)	Casos y pruebas
RF15	views/ReportesView.vue, views/ConteoInventarioView.vue, lib/archivos.js	ajustar_inventario, movimientos_inventario	CP13, CP14
RNF01	router/index.js (carga diferida), stores/tienda.js	Índices por fecha	CP18
RNF02	lib/permisos.js, lib/inactividad.js, lib/url-segura.js, lib/limpiar-url.js, views/UsuariosView.vue, vite.config.js, vercel.json	Políticas RLS, permisos_rol, guardar_permisos	CP15, CP16, CP17, CP22, CP23
RNF03	components/, *views/*	—	CP25
RNF04	iniciar-britny-max.bat, vercel.json	Supabase (nube)	CP21
RNF05	components/AppLayout.vue, style.css (Tailwind)	—	CP19
RNF07	lib/validaciones.js, services/operaciones.js	Restricciones y funciones transaccionales	CP09
RNF08	services/supabaseBackend.js	PostgreSQL en Supabase	CP21
RNF09	views/ConfiguracionView.vue, lib/archivos.js	importar_respaldo	CP20

Nota. Las rutas son relativas a la carpeta src, salvo los archivos de la raíz del proyecto.

Elaboración propia.

Punto de entrada. Antes que cualquier otra cosa importa limpiar-url.js, que quita de la barra de direcciones los tokens o contraseñas que pudieran llegar en la URL. Luego crea la aplicación con Pinia (estado) y Vue Router (pantallas) y, si Supabase cierra la sesión en otra pestaña, devuelve a todas al inicio de sesión (Figura 35).

Figura 35

Punto de Entrada de la Aplicación (src/main.js)

src/main.js · líneas 1–22

```
// Primero: si la URL trae tokens, se quitan de la barra de direcciones y del
// historial antes de que el router la lea (no se leen ni se usan).
import './lib/limpiar-url.js'
import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'
import router from './router'
import backend from './services'
import { useAuthStore } from './stores/auth'
import './style.css'

const app = createApp(App).use(createPinia()).use(router)

// Si la sesión de Supabase vence o se cierra en otra pestaña, vuelve al login.
backend.alCerrarSesion?.(() => {
  const auth = useAuthStore()
  if (!auth.usuario) return
  auth.sesionTerminada()
  router.replace({ name: 'login' })
})

app.mount('#app')
```

Nota. Captura del código fuente del proyecto. Elaboración propia.

Rutas protegidas. Antes de abrir cualquier pantalla, el router aplica cuatro controles en orden: limpia la URL, cierra la sesión si pasaron cinco minutos sin actividad, exige haber iniciado sesión y, con los permisos vigentes que asignó la administradora, impide abrir una pantalla sin permiso y muestra el aviso correspondiente (RNF02) (Figura 36).

Figura 36

Protección de las Pantallas (src/router/index.js)

src/router/index.js · líneas 75–105

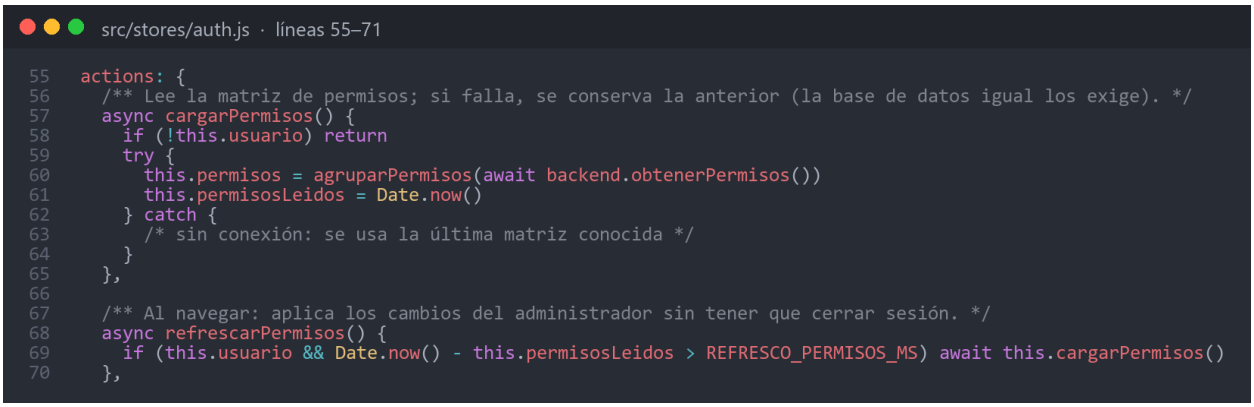
```
75 router.beforeEach(async (to) => {
76   // 1. Nunca deja tokens ni contraseñas en la barra de direcciones.
77   if (consultaConTokens(to.query) || fragmentoConTokens(to.hash)) {
78     const query = Object.fromEntries(
79       Object.entries(to.query).filter(([k]) => !PARAMETROS_SENSIBLES.includes(k.toLowerCase())),
80     )
81     return { path: to.path, query, hash: fragmentoConTokens(to.hash) ? '' : to.hash, replace: true }
82   }
83
84   const auth = useAuthStore()
85   await auth.verificar()
86
87   // 2. Una sesión inactiva por más de 5 minutos se cierra al intentar navegar.
88   if (auth.usuario && sesionVencida()) {
89     await auth.cerrarSesion('inactividad')
90     return { name: 'login', replace: true }
91   }
92
93   if (!to.meta.publica && !auth.usuario) {
94     recordarDestino(to.fullPath)
95     return { name: 'login', replace: true }
96   }
97   if (to.name === 'login' && auth.usuario) return { name: 'inicio' }
98
99   // 3. Nivel de seguridad de la pantalla, con los permisos vigentes que asignó el administrador.
100  await auth.refrescarPermisos()
101  if (to.meta.permiso && !auth.puede(to.meta.permiso)) {
102    useUiStore().error(`Tu nivel de acceso (${auth.nombreRol}) no permite abrir «${to.meta.titulo}»`)
103    return { name: 'inicio', replace: true }
104  }
105})
```

Nota. Captura del código fuente del proyecto. Elaboración propia.

Sesión y permisos. El almacén de sesión lee la matriz de permisos de la base de datos al iniciar sesión y la vuelve a leer, como máximo cada 30 segundos, al cambiar de pantalla; así un cambio de la administradora llega a las personas conectadas sin que tengan que salir. Si no hay conexión, conserva la última matriz conocida (Figura 37).

Figura 37

Carga de Permisos en la Sesión (src/stores/auth.js)



Nota. Captura del código fuente del proyecto. Elaboración propia.

Reglas de permisos. normalizarPermisos valida la matriz que guarda la administradora y agrega los permisos que cada uno necesita; puede decide si un nivel puede realizar una acción (el administrador siempre puede y «administrar usuarios» es solo suyo), y exigirPermiso detiene la operación con un mensaje claro cuando no está permitida (Figura 38).

Figura 38

Reglas de Permisos (src/lib/permisos.js)



Nota. Captura del código fuente del proyecto. Elaboración propia.

Cierre por inactividad. Revisa cada segundo cuánto falta para los cinco minutos sin actividad, registra el teclado, el mouse, el toque y el lector como actividad, y cierra la sesión al vencer el tiempo. Como usa la hora real del equipo, también la cierra si el computador se suspendió; treinta segundos antes, la pantalla muestra el aviso «¿Sigues ahí?» (Figura 39).

Figura 39

Cierre de Sesión por Inactividad (src/composables/useInactividad.js)

src/composables/useInactividad.js · líneas 10–48

```
10 export function useInactividad(alVencer) {
11   const restante = ref(LIMITE_MS)
12   let intervalo = null
13   let ultimaEscritura = 0
14   let vencida = false
15
16   function revisar() {
17     if (vencida) return
18     const r = tiempoRestante()
19     restante.value = r ?? 0
20     if (r === null || r <= 0) {
21       vencida = true
22       detener()
23       alVencer()
24     }
25   }
26
27   function actividad() {
28     if (vencida) return
29     if (sesionVencida()) return revisar()
30     const t = Date.now()
31     // Escribe como máximo cada 2 s para no saturar el almacenamiento con mousemove.
32     if (t - ultimaEscritura > 2000) {
33       ultimaEscritura = t
34       registrarActividad(t)
35       restante.value = LIMITE_MS
36     }
37   }
38
39   function seguirConectado() {
40     ultimaEscritura = 0
41     actividad()
42     revisar()
43   }
44
45   function detener() {
46     for (const e of EVENTOS) window.removeEventListener(e, actividad, { capture: true })
47     document.removeEventListener('visibilitychange', revisar)
48     clearInterval(intervalo)
```

Nota. Captura del código fuente del proyecto. Elaboración propia.

Acceso a datos. La aplicación no escribe las ventas directamente en las tablas: llama a funciones de PostgreSQL (RPC) que hacen toda la operación en una sola transacción. El modo de demostración tiene un archivo equivalente (localBackend.js) con las mismas reglas, para presentar el sistema sin conexión (Figura 40).

Figura 40

Llamadas a las Operaciones de la Base de Datos (src/services/supabaseBackend.js)



Nota. Captura del código fuente del proyecto. Elaboración propia.

Venta transaccional. La función registrar_venta comprueba el permiso, la fecha y cada cantidad; bloquea el producto mientras lo usa (for update) para que dos ventas simultáneas no dejen existencias negativas; descuenta el inventario, guarda el detalle y el movimiento, y genera el ingreso. Si algo falla, no se guarda nada (RF07 y RNF07) (Figura 41).

Figura 41

Registro Transaccional de una Venta (supabase/schema.sql)

supabase/schema.sql · líneas 569–624

```sql
create or replace function public.registrar_venta(p_fecha timestamptz, p_items jsonb)
returns uuid
language plpgsql
set search_path = ''
as $$
declare
  v_fecha  timestamptz;
  v_venta  uuid;
  v_total  numeric := 0;
  v_cant   integer;
  v_precio numeric;
  it       jsonb;
  p        public.productos%rowtype;
begin
  perform privado.exigir_permiso('ventas.registrar');
  perform set_config('britnymax.operacion', 'si', true);
  v_fecha := privado.validar_fecha_operacion(p_fecha);
  if jsonb_typeof(p_items) is distinct from 'array' or jsonb_array_length(p_items) = 0 then
    raise exception 'La venta no tiene productos';
  end if;
  if jsonb_array_length(p_items) > 200 then raise exception 'Una venta admite máximo 200 productos'; end if;

  insert into public.ventas (fecha, total) values (v_fecha, 0) returning id into v_venta;

  for it in select * from jsonb_array_elements(p_items) loop
    select * into p from public.productos where id = (it ->> 'producto_id')::uuid and activo for update;
    if not found then raise exception 'Uno de los productos ya no existe en el catálogo'; end if;

    v_cant := (it ->> 'cantidad')::integer;
    if v_cant is null or v_cant <= 0 then raise exception 'Cantidad inválida para %', p.nombre; end if;
    if p.cantidad < v_cant then
      raise exception 'Stock insuficiente para % (disponible: %)', p.nombre, p.cantidad;
    end if;
    v_precio := coalesce((it ->> 'precio_unitario')::numeric, p.precio_venta);
    if v_precio < 0 then raise exception 'Precio inválido para %', p.nombre; end if;

    update public.productos set cantidad = cantidad - v_cant where id = p.id;

    insert into public.detalle_venta (venta_id, producto_id, nombre_producto, cantidad, precio_unitario, subtotal)
    values (v_venta, p.id, p.nombre, v_cant, v_precio, v_cant * v_precio);

    insert into public.movimientos_inventario (fecha, producto_id, nombre_producto, tipo, cantidad, stock_resultante, motivo, referencia_id)
    values (v_fecha, p.id, p.nombre, 'salida', -v_cant, p.cantidad - v_cant, 'Venta', v_venta);

    v_total := v_total + v_cant * v_precio;
  end loop;

  update public.ventas set total = v_total where id = v_venta;

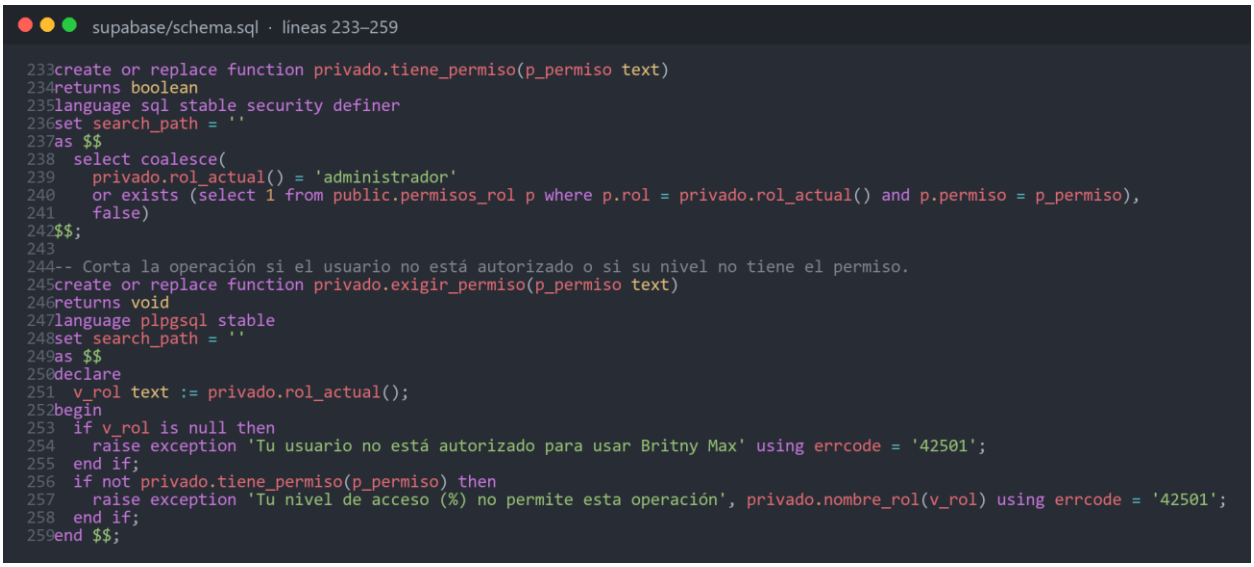
  if v_total > 0 then
    insert into public.ingresos (fecha, concepto, monto, tipo, venta_id)
    values (privado.dia_bogota(v_fecha), 'Venta de productos', v_total, 'venta', v_venta);
  end if;
  perform set_config('britnymax.operacion', '', true);
  return v_venta;
end $$;
```

Nota. Captura del código fuente del proyecto. Elaboración propia.

Permisos en la base de datos. tiene_permiso consulta la tabla permisos_rol (el administrador los tiene todos) y exigir_permiso corta la operación con el mismo mensaje que muestra la aplicación. Las políticas de seguridad y las funciones las usan, por lo que un permiso retirado se bloquea también para quien intente usar la API directamente (Figura 42).

Figura 42

Verificación de Permisos en la Base de Datos (supabase/schema.sql)

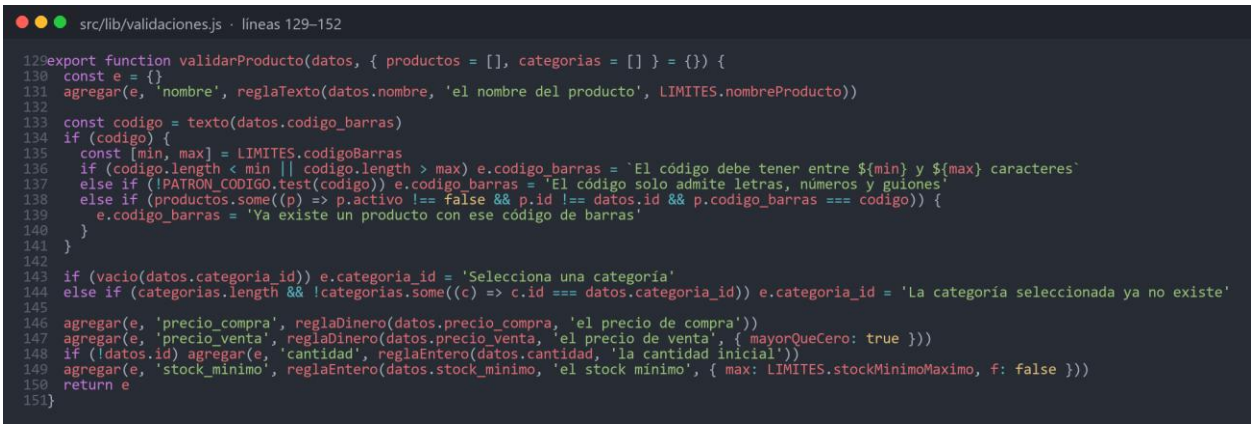


Nota. Captura del código fuente del proyecto. Elaboración propia.

Validaciones. Las reglas de cada formulario están en un solo archivo, que usan tanto las pantallas (para mostrar el error junto al campo) como el almacén de la tienda (para no enviar datos inválidos). Los mismos límites existen como restricciones en la base de datos, de modo que ningún dato incorrecto llega a guardarse (Figura 43).

Figura 43

Validación de Productos (src/lib/validaciones.js)



Nota. Captura del código fuente del proyecto. Elaboración propia.

CONTROL DE CAMBIOS

FECHA	VERSIÓN INICIAL	DESCRIPCIÓN DEL CAMBIO	VERSIÓN FINAL
26/08/2025	00	Actualización Formato	00